

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Dương Minh Hoàng

**PHƯƠNG PHÁP NỘI SUY HỖ TRỢ GIẢM ĐỘ TRỄ
TRONG GAME TRỰC TUYẾN**

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Công nghệ thông tin

HÀ NỘI - 2026

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



DƯƠNG MINH HOÀNG

**PHƯƠNG PHÁP NỘI SUY HỖ TRỢ GIẢM ĐỘ TRỄ
TRONG GAME TRỰC TUYẾN**

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Công nghệ thông tin

Cán bộ hướng dẫn: ThS. Trần Mạnh Cường

Cán bộ đồng hướng dẫn: TS. Nguyễn Thị Thủy

HÀ NỘI - 2026

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



Duong Minh Hoang

**INTERPOLATION - BASED METHOD FOR REDUCING
LATENCY IN ONLINE GAMES**

BACHELOR'S THESIS

Major: Information Technology

Supervisor: M.Sc. Tran Manh Cuong

Co-Supervisor: Dr. Nguyen Thi Thuy

HANOI - 2026

Lời cảm ơn

Lời đầu tiên, tôi xin gửi lời cảm ơn chân thành và lòng biết ơn sâu sắc nhất đến cán bộ hướng dẫn ThS. Trần Mạnh Cường và TS. Nguyễn Thị Thủy đã tận tình chỉ đạo, định hướng và động viên tôi trong toàn bộ quá trình thực hiện khóa luận tốt nghiệp này.

Tôi cũng xin gửi lời cảm ơn tới các thầy cô giáo tại trường Đại học Công nghệ - Đại học Quốc gia Hà Nội đã giảng dạy và cung cấp cho tôi các kiến thức chuyên môn quan trọng và quý báu để tôi có được nền tảng như ngày hôm nay.

Trong quá trình làm khóa luận tốt nghiệp, do trình độ và kinh nghiệm thực tiễn còn hạn chế, khóa luận không thể tránh khỏi những thiếu sót. Tôi rất mong được sự chỉ dẫn và góp ý từ các thầy cô nhằm khắc phục những thiếu sót này.

Tôi xin chân thành cảm ơn!

Lời cam đoan

Tôi là Dương Minh Hoàng, sinh viên lớp QH-2022-I/CQ-I-IT1. Tôi xin xác nhận và cam kết rằng khóa luận tốt nghiệp được thực hiện và hoàn thành bởi cá nhân tôi, dưới sự chỉ dẫn của ThS. Trần Mạnh Cường và TS. Nguyễn Thị Thủy.

Trong quá trình thực hiện, tôi đã tuân thủ nghiêm túc quy định về trích dẫn và sử dụng tài liệu tham khảo, không sao chép hay đạo văn từ bất kỳ công trình nào mà không ghi rõ nguồn gốc. Tất cả dữ liệu, phân tích và kết quả nghiên cứu đều được thu thập và trình bày một cách trung thực, khách quan. Tôi xin cam đoan rằng trong quá trình thực hiện khóa luận, tôi không lạm dụng trí tuệ nhân tạo để thay thế cho việc nghiên cứu, lập trình và viết nội dung khóa luận. Các công cụ trí tuệ nhân tạo được sử dụng chỉ đóng vai trò hỗ trợ tham khảo, gợi ý và rà soát một số nội dung, không quyết định thay cho quá trình tư duy, phân tích và triển khai của cá nhân tôi. Tôi chấp nhận mọi hình thức xử lý theo quy định của Khoa và Nhà trường nếu phát hiện bất kỳ vi phạm nào về tính trung thực của khóa luận này.

Hà Nội, ngày tháng năm

Sinh viên

Dương Minh Hoàng

Tóm tắt

Trong bối cảnh ngành công nghiệp trò chơi điện tử đang phát triển mạnh theo hướng trực tuyến nhiều người chơi thời gian thực, bài toán đồng bộ hóa trạng thái giữa Server và Client trở thành một thách thức kỹ thuật quan trọng. Sự tồn tại của độ trễ mạng và độ biến thiên độ trễ trong hạ tầng Internet thường gây ra các hiện tượng như chuyển động giật cục, sai lệch vị trí hoặc dịch chuyển tức thời, làm suy giảm đáng kể chất lượng trải nghiệm của người chơi. Để giảm thiểu ảnh hưởng của các yếu tố này, nhiều phương pháp đã được đề xuất nhằm che giấu độ trễ thông qua cơ chế dự đoán và nội suy trạng thái. Trong số đó, Dead Reckoning là một phương pháp phổ biến, dựa trên việc dự đoán trạng thái tương lai của đối tượng từ các thông tin đã biết, kết hợp với nội suy để đảm bảo tính liên tục của chuyển động. Tuy nhiên, phương pháp này vẫn tồn tại một số hạn chế như độ chính xác dự đoán chưa cao trong các tình huống phức tạp, độ mượt chuyển động chưa ổn định, và khó xử lý các trường hợp liên quan đến hành vi người chơi hoặc bất đồng bộ tương tác.

Khóa luận này tập trung phân tích phương pháp Dead Reckoning, từ đó đề xuất các cải tiến về mô hình dự đoán, kỹ thuật nội suy và cơ chế đồng bộ thời gian giữa Server và Client, đồng thời xây dựng cơ chế thích ứng với độ biến thiên độ trễ nhằm nâng cao độ chính xác và tính ổn định của hệ thống. Bên cạnh đó, khóa luận cũng nghiên cứu và tích hợp các phương pháp bổ trợ như Client-side Prediction, Server Reconciliation và Lag Compensation nhằm khắc phục các hạn chế còn tồn tại của Dead Reckoning trong các kịch bản cụ thể.

Về mặt thực nghiệm, khóa luận xây dựng một hệ thống thử nghiệm trên nền tảng Unity Engine, sử dụng thư viện Netcode for GameObjects để triển khai các phương pháp đề xuất. Hiệu quả của hệ thống được đánh giá thông qua các chỉ số như độ giật chuyển động, độ chính xác, số khung hình trên giây và mức sử dụng bộ nhớ. Kết quả thực nghiệm cho thấy các cải tiến đề xuất có khả năng nâng cao chất lượng đồng bộ trạng thái, đồng thời làm rõ sự đánh đổi giữa độ mượt chuyển động, tính chính xác của dữ liệu và chi phí tài nguyên, qua đó đề xuất các cấu hình phù hợp cho từng loại trò chơi và điều kiện mạng khác nhau.

Từ khóa: Dead Reckoning, Làm mượt chuyển động, Game trực tuyến nhiều người chơi

Abstract

In the context of the video game industry’s rapid development toward real-time online multiplayer experiences, the problem of state synchronization between the Server and the Client has become an important technical challenge. The presence of network latency and latency variation in Internet infrastructure often causes phenomena such as jerky movement, positional discrepancies, or instantaneous displacement, significantly reducing the quality of the player experience. To mitigate the impact of these factors, many methods have been proposed to mask latency through state prediction and interpolation mechanisms. Among them, Dead Reckoning is a widely used method, based on predicting the future state of an object from known information, combined with interpolation to ensure motion continuity. However, this method still has several limitations, such as low prediction accuracy in complex situations, unstable motion smoothness, and difficulty in handling cases related to player behavior or collision desynchronization.

This thesis focuses on analyzing the Dead Reckoning method, thereby proposing improvements to the prediction model, interpolation techniques, and time synchronization mechanism between the Server and the Client. It also develops a mechanism that adapts to latency variation in order to improve the accuracy and stability of the system. In addition, the thesis studies and integrates supporting methods such as Client-side Prediction, Server Reconciliation, and Lag Compensation to overcome the remaining limitations of Dead Reckoning in specific scenarios.

From an experimental perspective, the thesis builds a testing system on the Unity Engine platform, using the Netcode for GameObjects library to implement the proposed methods. The effectiveness of the system is evaluated through metrics such as motion jerk, accuracy, frames per second, and memory usage. The experimental results show that the proposed improvements can enhance the quality of state synchronization, while also clarifying the trade-offs between motion smoothness, data accuracy, and resource cost. Based on these findings, the thesis proposes suitable configurations for different types of games and network conditions.

Keywords: Dead Reckoning, Motion Smoothing, Online Multiplayer Game

Mục lục

Lời cảm ơn

Lời cam đoan i

Tóm tắt ii

Abstract iii

Mục lục iv

Danh sách hình vẽ vii

Danh sách bảng viii

Danh mục các từ viết tắt ix

Chương 1 Đặt vấn đề 1

1.1 Bối cảnh và lý do chọn đề tài 1

1.2 Mô tả bài toán 2

1.3 Mục tiêu và phạm vi nghiên cứu 3

1.4 Cấu trúc khóa luận 4

Chương 2 Cơ sở lý thuyết 5

2.1 Tổng quan về lập trình game trực tuyến 5

2.1.1 Kiến trúc mạng 5

2.1.2 Các khái niệm ảnh hưởng tới đồng bộ 8

2.1.3 Vòng lặp game 11

2.2 Cơ sở vật lý 14

2.2.1 Chuyển động thẳng đều 15

2.2.2 Chuyển động thẳng biến đổi đều 16

2.2.3 Jerk 17

2.2.4 Độ mượt chuyển động	18
2.3 Dead Reckoning	19
2.3.1 Giới thiệu	19
2.3.2 Nguyên lý hoạt động	20
2.3.3 Mã giả thuật toán	22
2.3.4 Ưu điểm và hạn chế	23
2.3.5 Ứng dụng	24
2.3.6 Một số trò chơi sử dụng Dead Reckoning	25
2.3.7 Ý nghĩa của Dead Reckoning trong khóa luận	25
Chương 3 Phương pháp đề xuất	26
3.1 Phân tích hạn chế của Dead Reckoning	26
3.2 Kiến trúc tổng thể của phương pháp đề xuất	27
3.3 Hạ tầng thời gian và lưu trữ trạng thái	30
3.3.1 Time Stamping	30
3.3.2 Tick Management	33
3.3.3 Circular Buffer	36
3.4 Cải tiến DR	38
3.4.1 Mô hình dự đoán bậc hai	38
3.4.2 Adaptive Threshold	40
3.4.3 Làm mượt giảm chấn tối hạn SmoothDamp	42
3.5 Phương pháp hỗ trợ trong mô hình Server Authoritative	45
3.5.1 Client-side Prediction	45
3.5.2 Server Reconciliation	48
3.5.3 Lag Compensation	50
Chương 4 Thực nghiệm và đánh giá	54
4.1 Câu hỏi nghiên cứu	54
4.2 Môi trường và thiết lập thực nghiệm	54
4.2.1 Tổng quan	54
4.2.2 Công nghệ cốt lõi	55
4.2.3 Các tính năng chính của hệ thống	55
4.2.4 Quy trình vận hành demo	56
4.3 Chỉ số đánh giá	60
4.3.1 Chỉ số Jerk	60

4.3.2 Tỷ lệ chính xác	60
4.3.3 Mức sử dụng CPU	61
4.3.4 Mức sử dụng bộ nhớ RAM	62
4.3.5 Thu thập dữ liệu CPU và RAM	62
4.4 Kết quả thực nghiệm	65
4.4.1 Tổng quan	65
4.4.2 Đánh giá mô hình dự đoán và Time Stamping	65
4.4.3 Đánh giá độ mượt chuyển động	66
4.4.4 Đánh giá Client-side Prediction và Server Reconciliation	69
4.4.5 Đánh giá Lag Compensation	71
4.4.6 Tổng hợp kết quả	73
Kết luận	74
Tài liệu tham khảo	76

Danh sách hình vẽ

3.1	Luồng xử lý của phương pháp đề xuất	28
4.1	Giao diện menu chính trong game	56
4.2	Giao diện Lobby trong game	57
4.3	Giao diện gameplay	58
4.4	Các waypoint mà bot sẽ di chuyển tới trong quá trình thực nghiệm .	59
4.5	Công cụ Unity Profiler	63
4.6	Công cụ Unity Profile Analyzer	64

Danh sách bảng

4.1	Kết quả thực nghiệm mô hình dự đoán và Time Stamping	65
4.2	Kết quả thực nghiệm Nội suy và Adaptive Threshold	66
4.3	Kết quả thực nghiệm Adaptive Threshold trong điều kiện Jitter . . .	67
4.4	Kết quả thực nghiệm tham số Adaptive Threshold	68
4.5	Kết quả thực nghiệm Client-side Prediction và Server Reconciliation	69
4.6	Kết quả thực nghiệm Server Reconciliation	70
4.7	Kết quả thực nghiệm Lag Compensation	71
4.8	Kết quả thực nghiệm tham số Lag Compensation	72

Danh mục các từ viết tắt

Từ viết tắt	Cụm từ đầy đủ	Ý nghĩa / Mô tả
DR	Dead Reckoning	Phương pháp làm mượt chuyển động bằng cách nội suy trạng thái vật thể dựa trên dự đoán từ dữ kiện quá khứ
FPS	Frames Per Second	Số lượng khung hình mỗi giây
NPC	Non-Player Character	Nhân vật trong game mà không phải người chơi điều khiển
RTT	Round Trip Time	Là khoảng thời gian cần thiết để một gói dữ liệu được gửi từ máy tính của Client đến Server và nhận lại phản hồi
CSP	Client-side Prediction	Phương pháp dự đoán trạng thái game bằng cách áp dụng input từ người chơi ngay lập tức
SR	Server Reconciliation	Phương pháp đồng bộ trạng thái với Server thông qua việc ghi đè trạng thái mới và lặp lại input trong bộ đệm
CPU	Central Processing Unit	Bộ xử lý trung tâm, thành phần thực hiện các phép tính và xử lý lệnh trong hệ thống
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên, được sử dụng để lưu trữ tạm thời dữ liệu trong quá trình chương trình hoạt động
NGO	Netcode for GameObjects	Thư viện mạng của Unity hỗ trợ xây dựng và đồng bộ các đối tượng trong game trực tuyến nhiều người chơi

Chương 1

Đặt vấn đề

1.1 Bối cảnh và lý do chọn đề tài

Trò chơi trực tuyến là một thị trường có quy mô rất lớn. Năm 2024, game Roblox (một game trực tuyến nhiều người chơi) ghi nhận có tới hơn 70 triệu người chơi mỗi ngày [1], con số này tương đương với dân số Thái Lan trong cùng năm đó [2]. Người chơi ngày nay không chỉ kỳ vọng vào chất lượng đồ họa mà còn yêu cầu khả năng tương tác gần như tức thời với nhiều người chơi khác trong cùng một môi trường ảo. Tuy nhiên, sự gia tăng về quy mô hệ thống và khoảng cách địa lý giữa các người chơi đã đặt ra nhiều thách thức kỹ thuật trong việc đảm bảo tính ổn định và đồng bộ trạng thái trò chơi.

Một trong những rào cản lớn nhất ảnh hưởng đến trải nghiệm trong môi trường trực tuyến là độ trễ mạng. Độ trễ mạng là khoảng thời gian cần thiết để một gói tin di chuyển từ Client đến Server và phản hồi trở lại. Khi độ trễ xuất hiện, trạng thái mà người chơi quan sát được thường là trạng thái đã xảy ra trong quá khứ so với thời điểm hiện tại trên Server. Sự chênh lệch này có thể gây ra hiện tượng hiển thị không mượt, sai lệch vị trí thực thể hoặc các hành vi vật lý kém tự nhiên, từ đó làm giảm tính liên tục của trải nghiệm chơi game [3]. Trong thực tiễn, điều này không chỉ ảnh hưởng đến chất lượng hệ thống mà còn làm giảm mức độ công bằng trong các trò chơi có tính cạnh tranh cao, chẳng hạn như các tình huống bắn trúng nhưng không được ghi nhận. Trong một game bắn súng góc nhìn thứ nhất, độ trễ chỉ khoảng 100 ms cũng có thể làm giảm hiệu suất của người chơi [4]. Bên cạnh đó, sự thiếu công bằng do độ trễ là một trong những nguyên nhân quan trọng khiến người chơi rời bỏ trò chơi.

Theo Grand View Research, quy mô thị trường game trực tuyến toàn cầu năm 2024 đạt khoảng 127,6 tỷ USD [5]. Trong khi đó, báo cáo Online Gaming Connectivity Report của Edgemarket cho thấy 34% người chơi có xu hướng rời bỏ hoàn toàn game khi gặp các vấn đề do độ trễ mạng [6]. Từ đó, phần doanh thu tiềm năng chưa được khai thác do các vấn đề liên quan đến độ trễ mạng có thể tính được lên tới

khoảng 65,7 tỷ USD. Qua đó phản ánh tác động nghiêm trọng của độ trễ mạng đối với tiềm năng doanh thu của game trực tuyến.

Từ thực tiễn đó, việc giảm thiểu ảnh hưởng của độ trễ trong game trực tuyến trở thành một vấn đề quan trọng. Tuy nhiên, việc loại bỏ hoàn toàn độ trễ là không khả thi do các giới hạn vật lý của mạng, đặc biệt là khoảng cách địa lý giữa các người chơi và Server. Nếu như khoảng cách trên là khoảng 7000 km (bằng từ Hà Nội tới Moscow, Nga) thì độ trễ khi dùng mạng 5G sẽ là khoảng 181 ms [3], độ trễ sẽ còn tăng thêm nếu tính thêm cả thời gian Server xử lý thông tin. Bên cạnh đó, việc cài đặt Server tại nhiều nơi nhằm giảm khoảng cách địa lý cũng không phải phương án tối ưu do chi phí cài đặt Server không hề nhỏ. Do đó, thay vì cố gắng triệt tiêu độ trễ, nhiều phương pháp đã được đề xuất nhằm giảm cảm nhận độ trễ từ phía người chơi thông qua việc che giấu hoặc bù đắp các sai lệch trong quá trình hiển thị và tương tác.

Trong số đó, các kỹ thuật dự đoán và nội suy đóng vai trò quan trọng trong việc cải thiện tính liên tục của trạng thái trò chơi. Một phương pháp phổ biến là Dead Reckoning, cho phép máy khách dự đoán trạng thái tương lai của đối tượng dựa trên các thông tin đã biết từ máy chủ. Bên cạnh đó, kỹ thuật nội suy được sử dụng để ước lượng các trạng thái trung gian giữa các mốc thời gian nhận được, từ đó giúp làm mượt chuyển động và giảm thiểu sự gián đoạn trong hiển thị. Tuy nhiên, các phương pháp này vẫn tồn tại những hạn chế nhất định, đặc biệt trong các tình huống chuyển động phức tạp hoặc khi xảy ra sai lệch lớn giữa trạng thái dự đoán và trạng thái thực tế. Do đó, việc nghiên cứu và cải tiến các phương pháp dự đoán và nội suy vẫn là một hướng đi có ý nghĩa cả về lý thuyết và thực tiễn [7].

1.2 Mô tả bài toán

Xuất phát từ những vấn đề thực tiễn đã được đề cập ở phần trên, khóa luận sẽ tập trung vào giải quyết bài toán đồng bộ trạng thái của các đối tượng chuyển động trong trò chơi trực tuyến thời gian thực trong điều kiện tồn tại độ trễ mạng. Đầu vào bài toán là trạng thái rời rạc của đối tượng được gửi từ Server theo từng khoảng thời gian, bao gồm vị trí, vận tốc và các thông tin liên quan, độ trễ mạng và độ biến thiên độ trễ giữa Client và Server. Đầu ra là trạng thái liên tục của đối tượng tại phía Client, bao gồm vị trí và hướng chuyển động, được ước lượng sao

cho đảm bảo tính mượt mà và gần với trạng thái thực tế tại Server. Phạm vi bài toán tập trung vào các đối tượng chuyển động trong môi trường trò chơi thời gian thực sử dụng kiến trúc Client-Server. Các tương tác vật lý phức tạp và va chạm được xem xét ở mức cơ bản trong phạm vi đánh giá.

1.3 Mục tiêu và phạm vi nghiên cứu

Mục tiêu chính của khóa luận là nghiên cứu và đề xuất các cải tiến cho phương pháp Dead Reckoning trong bài toán đồng bộ trạng thái đối tượng chuyển động trong trò chơi trực tuyến thời gian thực. Khóa luận lựa chọn hướng tiếp cận dựa trên mô hình động học của đối tượng, trong đó các đại lượng vật lý như vị trí, vận tốc và gia tốc được khai thác nhằm nâng cao độ chính xác của quá trình dự đoán trạng thái tại phía Client.

Trên cơ sở đó, khóa luận tập trung vào các mục tiêu cụ thể sau. Thứ nhất, phân tích nguyên lý hoạt động và các hạn chế của phương pháp Dead Reckoning trong điều kiện tồn tại độ trễ và độ biến thiên độ trễ trong mạng. Thứ hai, đề xuất các cải tiến cho mô hình dự đoán trạng thái dựa trên các quy luật động học, nhằm giảm sai lệch giữa trạng thái dự đoán và trạng thái thực tế từ Server. Thứ ba, nghiên cứu và áp dụng các kỹ thuật nội suy theo hướng tận dụng tính liên tục của chuyển động vật lý, qua đó cải thiện độ mượt trong quá trình hiển thị trạng thái. Thứ tư, xây dựng cơ chế đồng bộ thời gian và xử lý độ biến thiên độ trễ nhằm tăng tính ổn định và nhất quán của hệ thống dự đoán.

Bên cạnh phương pháp Dead Reckoning được cải tiến, khóa luận cũng xem xét tích hợp cơ chế hỗ trợ như Client-side Prediction, Server Reconciliation và Lag Compensation. Các phương pháp này được sử dụng nhằm xử lý các sai lệch phát sinh trong tương tác thời gian thực, đặc biệt trong các tình huống điều khiển nhân vật và va chạm, vốn không thể được giải quyết hoàn toàn chỉ bằng Dead Reckoning. Việc tích hợp các cơ chế này được thực hiện theo hướng đảm bảo tính nhất quán với mô hình vật lý của đối tượng trong môi trường mô phỏng.

Về phạm vi nghiên cứu, khóa luận tập trung vào các đối tượng chuyển động trong môi trường trò chơi thời gian thực sử dụng kiến trúc Client-Server. Các tương tác vật lý được xem xét ở mức cơ bản, chủ yếu liên quan đến chuyển động và va chạm đơn giản giữa các đối tượng. Khóa luận không đi sâu vào mô phỏng vật lý phức

tạp hoặc các hệ thống vật lý có độ chính xác cao.

Về thực nghiệm, phương pháp đề xuất được triển khai trên nền tảng Unity Engine, sử dụng thư viện Netcode for GameObjects để xây dựng hệ thống kiểm thử. Các kịch bản thử nghiệm được thiết kế nhằm đánh giá hiệu quả của phương pháp theo các tiêu chí như độ mượt của chuyển động, sai lệch vị trí so với trạng thái thực tế và khả năng thích ứng với độ trễ và độ biến thiên độ trễ của mạng. Từ đó, khóa luận đề xuất các tham số và phương thức cài đặt phù hợp cho từng kịch bản cụ thể.

1.4 Cấu trúc khóa luận

Khóa luận được tổ chức thành các chương bao gồm Chương 1 sẽ trình bày bối cảnh, bài toán nghiên cứu, mục tiêu và phạm vi của khóa luận. Chương 2 sẽ giới thiệu các khái niệm liên quan đến mạng trong trò chơi trực tuyến, tổng quan về Dead Reckoning và các ứng dụng thực tế. Chương 3 sẽ trình bày chi tiết cải tiến đối với Dead Reckoning và phương pháp bổ trợ. Chương 4 sẽ mô tả hệ thống thực nghiệm, các kịch bản kiểm thử và phân tích kết quả. Cuối cùng, phần Kết luận sẽ tổng kết kết quả đạt được và đề xuất hướng nghiên cứu trong tương lai.

Chương 2

Cơ sở lý thuyết

2.1 Tổng quan về lập trình game trực tuyến

2.1.1 Kiến trúc mạng

Trong các game trực tuyến nhiều người chơi, kiến trúc mạng quyết định cách dữ liệu được trao đổi, xử lý và đồng bộ giữa các thực thể trong hệ thống. Mỗi hành động của người chơi, chẳng hạn như di chuyển, tấn công, tương tác với vật thể hoặc thay đổi trạng thái nhân vật, đều cần được truyền qua mạng và phản ánh nhất quán trên các máy còn lại. Vì vậy, kiến trúc mạng có ảnh hưởng trực tiếp đến độ trễ điều khiển, độ chính xác của trạng thái trò chơi, khả năng mở rộng hệ thống và tính công bằng giữa người chơi.

Một kiến trúc mạng phù hợp cần giải quyết đồng thời nhiều vấn đề. Thứ nhất, hệ thống phải đảm bảo các Client cùng quan sát một trạng thái trò chơi đủ nhất quán, dù mỗi Client có điều kiện mạng khác nhau. Thứ hai, kiến trúc phải hạn chế khả năng gian lận, đặc biệt trong các trò chơi có tính cạnh tranh. Thứ ba, hệ thống cần duy trì cảm giác phản hồi tức thời cho người chơi, do các trò chơi thời gian thực thường yêu cầu tốc độ phản hồi rất cao. Trong số các kiến trúc được sử dụng hiện nay, mô hình Client-Server là một trong những mô hình phổ biến nhất cho trò chơi nhiều người chơi.

Mô hình Client-Server

Mô hình Client-Server chia hệ thống thành hai thành phần chính: Client và Server. Client chạy trên thiết bị của người chơi, chịu trách nhiệm thu thập input (tức dữ liệu đầu vào từ người chơi), hiển thị hình ảnh, phát âm thanh, xử lý giao diện và gửi các yêu cầu tương tác lên hệ thống. Server đóng vai trò trung tâm, tiếp nhận kết nối từ nhiều Client, xử lý dữ liệu nhận được và phân phối trạng thái trò chơi cần thiết về cho các Client [8].

Trong mô hình này, các Client thường không giao tiếp trực tiếp với nhau mà trao

đổi dữ liệu thông qua Server. Khi một người chơi thực hiện hành động, Client của người chơi đó gửi dữ liệu liên quan đến hành động lên Server. Sau đó, Server xử lý hoặc chuyển tiếp thông tin này tới các Client khác tùy theo thiết kế của trò chơi. Cách tổ chức này giúp việc quản lý kết nối, kiểm soát dữ liệu và đồng bộ trạng thái trở nên tập trung hơn so với mô hình ngang hàng.

Ưu điểm lớn của mô hình Client-Server là Server có thể đóng vai trò điều phối trung tâm. Nhờ đó, hệ thống dễ kiểm soát trạng thái trò chơi, dễ ghi nhận dữ liệu, dễ xử lý các tình huống xung đột và dễ áp dụng các cơ chế chống gian lận hơn. Ngoài ra, do mọi Client đều kết nối tới cùng một điểm trung tâm, việc mở rộng các cơ chế như phòng chơi, ghép trận, quản lý người chơi hoặc lưu trữ tiến trình cũng thuận lợi hơn.

Tuy nhiên, mô hình Client-Server cũng tạo ra một số thách thức. Do mọi dữ liệu quan trọng đều phải đi qua Server, độ trễ mạng giữa Client và Server sẽ ảnh hưởng trực tiếp đến trải nghiệm của người chơi. Nếu Client phải chờ phản hồi từ Server cho mọi hành động, người chơi có thể cảm thấy thao tác bị chậm hoặc thiếu tự nhiên. Ngoài ra, khi số lượng người chơi tăng lên, Server phải xử lý nhiều kết nối và nhiều gói tin hơn, dẫn đến yêu cầu cao hơn về hiệu năng xử lý, băng thông và thiết kế hệ thống.

Cần phân biệt rằng mô hình Client-Server chỉ mô tả kiến trúc giao tiếp mạng, tức là cách các thành phần trong hệ thống kết nối và trao đổi dữ liệu với nhau. Mô hình này không mặc định quy định phía nào có quyền quyết định cuối cùng đối với trạng thái gameplay. Trong một số thiết kế đơn giản, Client có thể tự xử lý một phần logic và gửi kết quả lên Server. Tuy nhiên, cách tiếp cận này tiềm ẩn rủi ro gian lận vì Client nằm trên thiết bị của người chơi và có thể bị can thiệp. Do đó, nhiều trò chơi hiện đại sử dụng biến thể Server Authoritative, trong đó Server giữ quyền xác nhận trạng thái cuối cùng của hệ thống [9].

Mô hình Server Authoritative

Server Authoritative là một cách triển khai của mô hình Client-Server, trong đó Server được xem là nguồn sự thật trung tâm của toàn bộ hệ thống. Thay vì tin tưởng hoàn toàn vào kết quả do Client gửi lên, Server chỉ tiếp nhận input hoặc yêu cầu hành động từ Client, sau đó tự xử lý logic gameplay, kiểm tra tính hợp lệ và

cập nhật trạng thái chính thức của trò chơi.

Trong mô hình này, Client không gửi trực tiếp trạng thái cuối cùng, ví dụ nhân vật đã di chuyển đến vị trí mới, mà thường gửi các thông tin đầu vào như hướng di chuyển, phím bấm, thời điểm thực hiện hành động hoặc yêu cầu tương tác. Server dựa trên các input này để mô phỏng trạng thái trò chơi theo luật đã định. Sau khi xử lý xong, Server gửi trạng thái đã được xác nhận về cho các Client để hiển thị.

Các trạng thái quan trọng như vị trí nhân vật, va chạm, sát thương, lượng máu, điểm số, trạng thái vật phẩm hoặc kết quả tương tác thường cần được Server xác nhận trước khi đồng bộ cho các Client khác. Điều này giúp giảm nguy cơ gian lận, vì người chơi không thể dễ dàng tự ý thay đổi các giá trị quan trọng ở phía Client rồi buộc hệ thống chấp nhận. Nếu Client gửi một hành động không hợp lệ, chẳng hạn như di chuyển quá tốc độ cho phép, tấn công ngoài phạm vi hoặc gây sát thương không đúng điều kiện, Server có thể từ chối hoặc hiệu chỉnh lại trạng thái đó [10].

Ưu điểm chính của Server Authoritative là tăng tính công bằng và tính nhất quán của hệ thống. Vì mọi trạng thái quan trọng đều được xác nhận bởi Server, các Client sẽ dần hội tụ về cùng một trạng thái chính thức. Điều này đặc biệt quan trọng đối với các trò chơi thời gian thực, nơi sai lệch nhỏ về vị trí, va chạm hoặc thời điểm thực hiện hành động có thể ảnh hưởng trực tiếp đến kết quả gameplay.

Tuy nhiên, Server Authoritative cũng làm vấn đề độ trễ trở nên rõ ràng hơn. Do Client phải gửi input lên Server và chờ trạng thái xác nhận quay trở lại, hành động của người chơi có thể bị phản hồi chậm nếu hệ thống xử lý một cách tuần tự. Trong môi trường mạng có độ trễ cao hoặc độ biến thiên trễ (jitter) lớn, người chơi có thể quan sát thấy các hiện tượng như nhân vật di chuyển không mượt, vị trí bị kéo ngược lại, va chạm không khớp hoặc trạng thái hiển thị khác với cảm nhận điều khiển.

Vì vậy, trong các trò chơi thời gian thực, Server Authoritative thường không được sử dụng một cách tách biệt mà kết hợp với nhiều kỹ thuật hỗ trợ ở phía Client. Các kỹ thuật như Dead Reckoning hay Client-side Prediction được sử dụng nhằm giảm cảm giác trễ, làm mượt chuyển động và xử lý sai lệch trong dự đoán ở Client. Nói cách khác, Server Authoritative cung cấp nền tảng về tính đúng đắn và chống gian lận, còn các kỹ thuật đồng bộ phía Client giúp cải thiện trải nghiệm điều khiển và hiển thị trong điều kiện mạng không lý tưởng.

Đối với đề tài này, mô hình Server Authoritative được xem là nền tảng kiến trúc chính. Server chịu trách nhiệm duy trì trạng thái chính thức của trò chơi, trong khi Client thực hiện hiển thị, dự đoán và làm mượt chuyển động để giảm tác động của độ trễ mạng. Cách tiếp cận này phù hợp với mục tiêu nghiên cứu các kỹ thuật đồng bộ trong trò chơi trực tuyến, đặc biệt là khi cần đánh giá ảnh hưởng của độ trễ, jitter và sai lệch trạng thái đến trải nghiệm chuyển động của người chơi.

2.1.2 Các khái niệm ảnh hưởng tới đồng bộ

Trong game trực tuyến, quá trình đồng bộ trạng thái giữa Client và Server chịu ảnh hưởng bởi nhiều yếu tố mạng khác nhau. Các yếu tố này quyết định thời điểm dữ liệu được gửi đi, thời điểm dữ liệu được nhận về và mức độ chính xác của trạng thái hiển thị trên Client so với trạng thái chính thức trên Server. Trong đó, độ trễ, jitter, mất gói tin, băng thông và tần suất cập nhật là những khái niệm có ảnh hưởng trực tiếp đến chất lượng đồng bộ.

Đối với các trò chơi thời gian thực, sai lệch nhỏ về thời gian cũng có thể dẫn đến sai lệch đáng kể về vị trí, va chạm hoặc kết quả tương tác. Vì vậy, việc hiểu rõ các yếu tố mạng là cơ sở để thiết kế các kỹ thuật như Dead Reckoning.

Độ trễ

Độ trễ là khoảng thời gian cần thiết để dữ liệu được truyền từ một điểm trong mạng đến một điểm khác. Trong game trực tuyến, độ trễ thường được quan sát thông qua Ping hoặc Round Trip Time (RTT), tức khoảng thời gian để một gói tin đi từ Client đến Server và quay trở lại Client [11]. Có thể mô tả Ping như sau:

$$Ping \approx t_{client \rightarrow server} + t_{server_process} + t_{server \rightarrow client}. \quad (2.1)$$

Trong đó, $t_{client \rightarrow server}$ là thời gian gói tin đi từ Client đến Server, $t_{server_process}$ là thời gian Server xử lý gói tin, và $t_{server \rightarrow client}$ là thời gian dữ liệu từ Server về Client. Có thể thấy, Ping không chỉ phụ thuộc vào thời gian truyền dữ liệu mà còn có thể chịu ảnh hưởng bởi thời gian xử lý phía Server. Trong nhiều trường hợp, thời gian xử lý tại Server nhỏ hơn đáng kể so với thời gian truyền qua mạng, do đó Ping thường được dùng để ước lượng tổng độ trễ hai chiều giữa Client và Server.

Nếu giả sử đường truyền hai chiều tương đối đối xứng, độ trễ một chiều có thể được xấp xỉ như sau:

$$t_{one_way} \approx \frac{Ping}{2}. \quad (2.2)$$

Tuy nhiên, đây chỉ là giá trị xấp xỉ vì đường truyền từ Client đến Server và từ Server về Client không phải lúc nào cũng có độ trễ bằng nhau. Ngoài ra, thời gian xử lý tại Server, hàng đợi gói tin và biến động mạng cũng có thể làm giá trị thực tế thay đổi.

Trong mô hình Server Authoritative, Client thường gửi input hoặc yêu cầu hành động lên Server. Server sau đó kiểm tra, xử lý và gửi trạng thái đã xác nhận trở lại Client. Nếu Client không sử dụng kỹ thuật dự đoán, người chơi có thể phải chờ gần một vòng RTT để thấy kết quả chính thức của hành động. Ví dụ, nếu Ping là 200 ms, phản hồi đã được Server xác nhận có thể đến Client sau khoảng 200 ms. Điều này làm người chơi cảm thấy thao tác bị trễ, đặc biệt trong các hành động yêu cầu phản hồi tức thời như di chuyển, bắn hoặc né tránh.

Đối với trạng thái của các đối tượng khác được gửi từ Server về Client, độ trễ quan sát thường liên quan nhiều hơn đến độ trễ một chiều. Ví dụ, nếu Ping là 200 ms và đường truyền tương đối đối xứng, trạng thái từ Server khi đến Client có thể đã cũ khoảng 100 ms, chưa tính thêm thời gian xử lý cục bộ. Vì vậy, trạng thái mà Client hiển thị thường là trạng thái trong quá khứ so với trạng thái hiện tại trên Server [4].

Độ trễ cao có thể gây ra nhiều vấn đề trong game trực tuyến. Với người chơi điều khiển nhân vật, độ trễ làm thao tác phản hồi chậm và giảm cảm giác kiểm soát. Với các đối tượng từ xa, độ trễ khiến vị trí hiển thị bị chậm so với trạng thái thực tế. Với các tương tác yêu cầu độ chính xác cao, chẳng hạn như va chạm hoặc bắn trúng mục tiêu, độ trễ có thể làm Client và Server đánh giá khác nhau về cùng một sự kiện.

Do không thể loại bỏ hoàn toàn độ trễ mạng, các hệ thống game thời gian thực thường sử dụng các kỹ thuật che giấu độ trễ hoặc bù trễ. Ví dụ, Dead Reckoning giúp duy trì chuyển động mượt mà cho các đối tượng nhận dữ liệu từ xa. Client-side Prediction và Server Reconciliation giúp Client phản hồi input ngay lập tức mà không cần chờ Server.

Jitter

Độ biến thiên trễ (jitter) là sự dao động của độ trễ theo thời gian, tức là các gói tin không đến nơi với khoảng thời gian ổn định [12]. Nếu độ trễ cho biết gói tin mất bao lâu để truyền qua mạng, thì jitter cho biết độ trễ đó thay đổi thất thường đến mức nào. Trong game trực tuyến, jitter là một yếu tố rất quan trọng vì nó ảnh hưởng trực tiếp đến tính đều đặn của dữ liệu đồng bộ.

Giả sử độ trễ của gói tin thứ i là d_i , jitter giữa hai gói tin liên tiếp có thể được biểu diễn đơn giản như sau:

$$J_i = |d_i - d_{i-1}|. \quad (2.3)$$

Nếu các gói tin lần lượt có độ trễ 50 ms, 150 ms và 80 ms, hệ thống đang tồn tại jitter lớn vì khoảng thời gian truyền không ổn định. Khi đó, dù Ping trung bình có thể không quá cao, dữ liệu từ Server vẫn đến Client không đều. Điều này khiến Client khó xác định thời điểm hợp lý để cập nhật hoặc nội suy trạng thái.

Khác với độ trễ ổn định, jitter làm hệ thống khó dự đoán thời điểm nhận dữ liệu mới. Nếu gói tin đến sớm, Client có thể nhận được trạng thái mới trước khi cần hiển thị, dẫn đến hiệu chỉnh sai lệch so với dự kiến. Nếu gói tin đến muộn, Client có thể thiếu dữ liệu để nội suy, buộc phải giữ trạng thái cũ hoặc ngoại suy tạm thời. Khi các gói tin đến không đều, vật thể trong game có thể xuất hiện hiện tượng giật, rung, tăng tốc hoặc giảm tốc bất thường.

Jitter cũng ảnh hưởng trực tiếp đến Dead Reckoning. Kỹ thuật Dead Reckoning thường dự đoán trạng thái hiện tại dựa trên trạng thái gần nhất nhận được từ Server và khoảng thời gian đã trôi qua. Nếu thời điểm nhận trạng thái dao động mạnh, giá trị thời gian dùng để dự đoán cũng thay đổi không ổn định. Điều này có thể làm sai số dự đoán tăng lên, đặc biệt khi đối tượng đổi hướng hoặc thay đổi vận tốc liên tục.

Để giảm ảnh hưởng của jitter, Client thường sử dụng bộ đệm để lưu một số trạng thái trước khi hiển thị. Thay vì hiển thị ngay trạng thái mới nhất, Client có thể cố ý hiển thị trạng thái trễ hơn một khoảng nhỏ. Cách này giúp hệ thống có đủ dữ liệu để nội suy giữa hai trạng thái đã biết, từ đó làm chuyển động mượt hơn. Tuy nhiên, bộ đệm càng lớn thì hiển thị càng ổn định nhưng độ trễ quan sát cũng tăng lên. Vì vậy, hệ thống cần cân bằng giữa độ mượt và độ trễ hiển thị.

2.1.3 Vòng lặp game

Vòng lặp game là cơ chế điều phối toàn bộ quá trình hoạt động của trò chơi theo thời gian. Đây là thành phần cốt lõi giúp trò chơi vận hành liên tục, bao gồm việc nhận input từ người chơi, xử lý logic game, cập nhật vật lý, xử lý dữ liệu mạng và hiển thị khung hình mới. Mỗi lần vòng lặp được thực hiện, trạng thái của trò chơi sẽ được cập nhật thêm một bước, từ đó tạo ra cảm giác thế giới trong game đang vận động liên tục.

Trong trò chơi trực tuyến, vòng lặp game không chỉ xử lý các tác vụ cục bộ trên máy người chơi mà còn phải phối hợp với dữ liệu nhận được từ Server. Do dữ liệu mạng luôn có độ trễ, Client không thể chỉ chờ trạng thái mới từ Server rồi mới cập nhật hiển thị. Thay vào đó, Client cần kết hợp giữa trạng thái cục bộ, dữ liệu mạng gần nhất và các kỹ thuật dự đoán hoặc nội suy để duy trì trải nghiệm mượt mà. Vì vậy, vòng lặp game trong game trực tuyến có vai trò đặc biệt quan trọng đối với cả độ chính xác của mô phỏng lẫn cảm giác điều khiển của người chơi.

Khái niệm vòng lặp game

Vòng lặp game là chuỗi các bước được thực hiện lặp đi lặp lại trong suốt thời gian trò chơi chạy. Ở mỗi vòng lặp, hệ thống thường nhận input, cập nhật trạng thái trò chơi, xử lý vật lý và hiển thị một khung hình mới cho người chơi. Quá trình này diễn ra liên tục với tốc độ cao, tạo nên cảm giác chuyển động theo thời gian thực.

Về cơ bản, một vòng lặp game có thể được mô tả qua ba giai đoạn chính, bao gồm nhận input, cập nhật trạng thái và hiển thị hình ảnh [13]. Giai đoạn nhận input có nhiệm vụ ghi nhận hành động của người chơi, chẳng hạn như nhấn phím, di chuyển chuột hoặc thao tác điều khiển. Giai đoạn cập nhật trạng thái xử lý các thay đổi của thế giới game, bao gồm logic gameplay, vật lý, va chạm, hoạt ảnh và các quy tắc tương tác. Cuối cùng, giai đoạn hiển thị chuyển trạng thái hiện tại của thế giới game thành hình ảnh hiển thị trên màn hình.

Có thể biểu diễn đơn giản một vòng lặp game như sau:

$$\text{Input} \rightarrow \text{Update} \rightarrow \text{Render.} \quad (2.4)$$

Tuy nhiên, trong các hệ thống thực tế, vòng lặp game thường phức tạp hơn do phải xử lý nhiều thành phần khác nhau như âm thanh, hoạt ảnh, vật lý, giao diện, tài nguyên và mạng. Đặc biệt trong game trực tuyến, vòng lặp game còn phải tiếp nhận và xử lý các gói tin từ Server, khiến trạng thái được sử dụng để hiển thị không chỉ phụ thuộc vào tính toán cục bộ mà còn phụ thuộc vào trạng thái mạng.

Vòng lặp game trong game trực tuyến

Trong game trực tuyến, Client không thể chỉ dựa vào dữ liệu hiện tại tại thời điểm hiển thị, vì dữ liệu từ Server luôn đến muộn hơn so với trạng thái thực tế trên Server. Nguyên nhân là do gói tin cần thời gian để truyền qua mạng, ngoài ra còn có thể bị ảnh hưởng bởi jitter, mất gói hoặc biến động băng thông. Vì vậy, tại thời điểm Client hiển thị một khung hình, trạng thái mới nhất mà Client nhận được thường là trạng thái của Server trong quá khứ.

Nếu Client chờ phản hồi từ Server trước khi cập nhật từng hành động, trò chơi sẽ phản hồi chậm và gây cảm giác điều khiển kém tự nhiên. Ví dụ, khi người chơi nhấn phím di chuyển, nhân vật sẽ không di chuyển ngay mà phải chờ input được gửi lên Server, Server xử lý, rồi trạng thái mới được gửi ngược lại Client. Với các trò chơi thời gian thực, cách xử lý này thường không phù hợp vì chỉ một độ trễ nhỏ cũng có thể làm giảm đáng kể trải nghiệm điều khiển.

Do đó, Client thường phải duy trì hiển thị liên tục bằng cách sử dụng dữ liệu gần nhất nhận được từ Server, kết hợp với các kỹ thuật như nội suy, ngoại suy hoặc dự đoán. Nội suy giúp làm mượt chuyển động giữa các trạng thái đã nhận được. Dead Reckoning giúp ước lượng trạng thái hiện tại dựa trên trạng thái cũ, vận tốc hoặc gia tốc. Client-side Prediction giúp Client phản hồi input cục bộ ngay lập tức trước khi nhận xác nhận từ Server.

Ở phía Server, hệ thống cũng có vòng lặp riêng, thường chạy theo tick rate cố định. Trong mỗi tick, Server nhận input từ các Client, kiểm tra tính hợp lệ, cập nhật trạng thái chính thức của trò chơi và gửi dữ liệu đồng bộ về cho các Client. Sau mỗi tick hoặc sau một số tick nhất định, Server có thể gửi gói tin chứa trạng thái của các đối tượng quan trọng. Do đó, hệ thống game trực tuyến thực chất là nhiều vòng lặp hoạt động song song nhưng không đồng bộ hoàn toàn về thời gian [13].

Tick và tick rate trong đồng bộ mạng

Trong game trực tuyến, *tick* là đơn vị cập nhật rời rạc của mô phỏng mạng hoặc mô phỏng Server. Thay vì xử lý trạng thái liên tục theo thời gian thực tuyệt đối, Server thường chia thời gian thành các bước nhỏ có độ dài cố định. Mỗi bước như vậy được gọi là một tick. Số tick được xử lý trong một giây gọi là *tick rate*.

Nếu tick rate của Server là f_{tick} , thời gian giữa hai tick liên tiếp được tính như sau:

$$\Delta t_{tick} = \frac{1}{f_{tick}}. \quad (2.5)$$

Ví dụ, nếu Server chạy ở tick rate 30 tick/giây, mỗi tick tương ứng khoảng 0.033 giây. Nếu Server chạy ở 60 tick/giây, mỗi tick tương ứng khoảng 0.0167 giây. Tick rate càng cao thì Server cập nhật trạng thái càng thường xuyên, giúp giảm sai số giữa các lần đồng bộ. Tuy nhiên, tick rate cao cũng làm tăng chi phí xử lý và băng thông mạng vì Server phải xử lý nhiều bước mô phỏng hơn trong cùng một khoảng thời gian.

Tick rate có ảnh hưởng trực tiếp đến độ chính xác và độ mượt của đồng bộ mạng. Nếu tick rate quá thấp, Client nhận được trạng thái thưa hơn, dẫn đến việc nội suy hoặc dự đoán phải xử lý khoảng cách thời gian lớn hơn giữa hai trạng thái. Điều này có thể làm chuyển động kém mượt hoặc tăng sai số dự đoán. Ngược lại, nếu tick rate quá cao nhưng băng thông hoặc khả năng xử lý không đủ, hệ thống có thể bị nghẽn, gây mất gói hoặc làm tăng độ trễ tổng thể.

Vì vậy, trong thiết kế game trực tuyến, tick rate cần được lựa chọn dựa trên đặc điểm gameplay, yêu cầu phản hồi, số lượng người chơi và khả năng hạ tầng. Các trò chơi có nhịp độ nhanh thường cần tick rate cao hơn để đảm bảo độ chính xác khi xử lý di chuyển, va chạm hoặc hành động tức thời. Trong khi đó, các trò chơi có nhịp độ chậm hơn có thể sử dụng tick rate thấp hơn để giảm chi phí mạng.

Ảnh hưởng của vòng lặp game đến độ mượt và độ chính xác

Vòng lặp game ảnh hưởng trực tiếp đến hai yếu tố quan trọng trong game trực tuyến là độ mượt và độ chính xác. Độ mượt liên quan đến việc chuyển động của vật thể có liên tục và tự nhiên không. Độ chính xác liên quan đến việc trạng thái trên Client có khớp với trạng thái chính thức trên Server không.

Nếu vòng lặp hiển thị hoạt động không ổn định, người chơi có thể thấy hiện tượng giật khung hình, chuyển động không đều hoặc độ trễ hiển thị. Nếu vòng lặp vật lý không ổn định, mô phỏng có thể sinh ra sai số, đặc biệt trong các tình huống va chạm hoặc chuyển động nhanh. Nếu vòng lặp mạng không được xử lý hợp lý, Client có thể hiển thị trạng thái cũ, trạng thái sai hoặc phải hiệu chỉnh đột ngột khi nhận dữ liệu mới từ Server.

Trong môi trường mạng lý tưởng, dữ liệu từ Server đến đều đặn và có độ trễ thấp, việc đồng bộ trạng thái tương đối đơn giản. Tuy nhiên, trong môi trường thực tế, độ trễ và jitter khiến thời điểm nhận trạng thái không đều. Khi đó, vòng lặp game phải có cơ chế xử lý sai lệch thời gian. Các kỹ thuật như bộ đệm, nội suy, dự đoán, hòa giải và làm mượt đều phụ thuộc vào cách vòng lặp game quản lý thời gian và áp dụng trạng thái mới.

Vì vậy, một vòng lặp game tốt trong game trực tuyến không chỉ cần cập nhật nhanh, mà còn cần cập nhật đúng thời điểm. Việc tách riêng thời gian hiển thị, thời gian vật lý và thời gian mạng giúp hệ thống linh hoạt hơn khi xử lý các điều kiện mạng khác nhau. Đây là nền tảng để triển khai các kỹ thuật đồng bộ nâng cao trong các phần tiếp theo.

2.2 Cơ sở vật lý

Trong các hệ thống mô phỏng chuyển động, đặc biệt là game trực tuyến thời gian thực, trạng thái của đối tượng thường được mô tả thông qua vị trí, vận tốc và gia tốc. Các đại lượng này quyết định cách đối tượng di chuyển trong không gian, đồng thời phản ánh mức độ thay đổi của chuyển động theo thời gian. Vì vậy, chúng có ảnh hưởng trực tiếp đến cảm nhận của người chơi về độ mượt và tính tự nhiên của chuyển động.

Trong phạm vi đề tài, các khái niệm vật lý được sử dụng làm cơ sở để phân tích sự thay đổi trạng thái theo thời gian. Việc xem xét các đại lượng như vị trí, vận tốc và gia tốc giúp làm rõ nguyên nhân phát sinh sai lệch hoặc giật cục trong quá trình đồng bộ. Từ đó, các khái niệm này phục vụ việc đánh giá các phương pháp đồng bộ hóa như Dead Reckoning, nội suy trạng thái và hiệu chỉnh sai số.

2.2.1 Chuyển động thẳng đều

Chuyển động thẳng đều là dạng chuyển động đơn giản nhất, trong đó vật chuyển động theo một đường thẳng với vận tốc không đổi theo thời gian. Nếu xét chuyển động một chiều, vị trí của vật tại thời điểm t được xác định bởi:

$$x(t) = x_0 + vt \quad (2.6)$$

trong đó x_0 là vị trí ban đầu của vật tại thời điểm $t = 0$, còn v là vận tốc không đổi [14].

Vận tốc của vật được định nghĩa là đạo hàm bậc nhất của vị trí theo thời gian:

$$v(t) = \frac{dx(t)}{dt} \quad (2.7)$$

Với chuyển động thẳng đều, do v là hằng số nên gia tốc của vật bằng không:

$$a(t) = \frac{dv(t)}{dt} = \frac{d^2x(t)}{dt^2} = 0 \quad (2.8)$$

Điều này cho thấy vật không có sự thay đổi về vận tốc trong suốt quá trình chuyển động. Vì vậy, chuyển động thẳng đều thường tạo ra cảm giác ổn định, dễ dự đoán và không có biến động đột ngột. Trong các hệ thống game, giả thiết chuyển động thẳng đều thường được sử dụng trong các mô hình dự đoán đơn giản, chẳng hạn như Dead Reckoning tuyến tính, khi Client ước lượng vị trí tiếp theo của đối tượng dựa trên vị trí và vận tốc gần nhất nhận được từ Server.

Tuy nhiên, trong thực tế, các đối tượng trong game hiếm khi chỉ di chuyển với vận tốc không đổi. Người chơi có thể tăng tốc, giảm tốc, đổi hướng hoặc dừng lại đột ngột. Khi đó, giả thiết vận tốc không đổi không còn phản ánh chính xác trạng thái chuyển động thực tế của đối tượng. Sai số dự đoán có thể tăng dần theo thời gian, đặc biệt khi khoảng thời gian giữa hai lần cập nhật từ Server lớn hoặc khi đối tượng thay đổi trạng thái chuyển động liên tục. Do đó, mặc dù chuyển động thẳng đều là cơ sở quan trọng để xây dựng mô hình dự đoán, nó vẫn có hạn chế khi mô phỏng các chuyển động phức tạp hơn trong môi trường game trực tuyến.

2.2.2 Chuyển động thẳng biến đổi đều

Chuyển động thẳng biến đổi đều là chuyển động trong đó gia tốc của vật không đổi theo thời gian. Khi đó, vận tốc của vật thay đổi tuyến tính theo thời gian và được xác định bởi:

$$v(t) = v_0 + at \quad (2.9)$$

trong đó v_0 là vận tốc ban đầu và a là gia tốc không đổi [14].

Vị trí của vật tại thời điểm t được xác định bởi:

$$x(t) = x_0 + v_0 t + \frac{1}{2}at^2 \quad (2.10)$$

Công thức trên cho thấy vị trí của vật là một hàm bậc hai theo thời gian. So với chuyển động thẳng đều, mô hình này mô tả được sự thay đổi vận tốc, do đó phù hợp hơn với các trường hợp vật thể tăng tốc hoặc giảm tốc.

Vì gia tốc là hằng số nên đạo hàm của gia tốc theo thời gian bằng không:

$$\frac{da(t)}{dt} = 0 \quad (2.11)$$

Điều này có nghĩa là mặc dù vận tốc thay đổi, mức độ thay đổi vận tốc vẫn ổn định. Vì vậy, chuyển động thẳng biến đổi đều thường tạo cảm giác tự nhiên hơn so với việc thay đổi vận tốc đột ngột. Trong game, các chuyển động có gia tốc ổn định giúp đối tượng có cảm giác quán tính rõ ràng hơn, đặc biệt trong các tình huống như xe tăng tốc, nhân vật bắt đầu di chuyển hoặc vật thể bị tác động bởi lực.

Tuy nhiên, mô hình này vẫn dựa trên giả thiết rằng gia tốc không đổi trong khoảng thời gian dự đoán. Trong thực tế, gia tốc của đối tượng trong game có thể thay đổi do input của người chơi, va chạm, địa hình hoặc các tác động vật lý khác. Nếu gia tốc thay đổi đột ngột giữa các thời điểm khác nhau, kết quả dự đoán vẫn có thể phát sinh sai số và chuyển động vẫn có thể gây cảm giác giật. Để phân tích mức độ thay đổi đột ngột của gia tốc, cần xét đến đại lượng bậc cao hơn là jerk.

2.2.3 Jerk

Jerk là đại lượng biểu diễn tốc độ thay đổi của gia tốc theo thời gian. Nếu vị trí của vật là $x(t)$, vận tốc là $v(t)$ và gia tốc là $a(t)$, thì jerk được định nghĩa như sau:

$$j(t) = \frac{da(t)}{dt} \quad (2.12)$$

Do gia tốc là đạo hàm bậc hai của vị trí theo thời gian, jerk cũng có thể được viết dưới dạng đạo hàm bậc ba của vị trí:

$$j(t) = \frac{d^3x(t)}{dt^3} \quad (2.13)$$

Jerk cho biết gia tốc của vật thay đổi nhanh hay chậm. Nếu gia tốc thay đổi từ từ, jerk có giá trị nhỏ và chuyển động thường tạo cảm giác mềm mại hơn. Ngược lại, nếu gia tốc thay đổi đột ngột trong một khoảng thời gian ngắn, jerk sẽ lớn, khiến chuyển động dễ bị cảm nhận là giật, cứng hoặc thiếu tự nhiên [15].

Trong trường hợp chuyển động thẳng đều, gia tốc luôn bằng không nên jerk cũng bằng không:

$$a(t) = 0 \Rightarrow j(t) = 0 \quad (2.14)$$

Trong trường hợp chuyển động thẳng biến đổi đều, gia tốc là hằng số nên jerk cũng bằng không:

$$a(t) = \text{const} \Rightarrow j(t) = 0 \quad (2.15)$$

Như vậy, jerk không phản ánh việc vật có đang chuyển động nhanh hay chậm, mà phản ánh mức độ đột ngột trong sự thay đổi gia tốc. Đây là lý do jerk thường được sử dụng để đánh giá độ mượt của chuyển động. Một vật có thể di chuyển với vận tốc lớn nhưng vẫn mượt nếu vận tốc và gia tốc thay đổi ổn định. Ngược lại, một vật dù di chuyển chậm vẫn có thể gây cảm giác giật nếu gia tốc thay đổi bất thường. Giá trị jerk càng lớn thì sự thay đổi gia tốc càng mạnh, đồng nghĩa với khả năng chuyển động bị cảm nhận là giật càng cao.

2.2.4 Độ mượt chuyển động

Độ mượt của chuyển động phụ thuộc vào sự liên tục và ổn định của các đại lượng vị trí, vận tốc, gia tốc và jerk theo thời gian. Một chuyển động được xem là mượt khi sự thay đổi trạng thái diễn ra liên tục, không xuất hiện các bước nhảy đột ngột về vị trí, vận tốc hoặc gia tốc.

Nếu vị trí thay đổi đột ngột, người chơi sẽ nhận thấy hiện tượng dịch chuyển tức thời. Nếu vận tốc thay đổi đột ngột, đối tượng có thể tạo cảm giác tăng tốc hoặc dừng lại không tự nhiên. Nếu gia tốc thay đổi quá nhanh, jerk sẽ lớn và chuyển động dễ bị cảm nhận là rung, giật hoặc thô cứng.

Mối quan hệ giữa gia tốc và lực có thể được giải thích thông qua Định luật II Newton:

$$\sum \vec{F} = m\vec{a} \quad (2.16)$$

Trong đó m là khối lượng của vật và $\vec{a}$ là gia tốc. Khi khối lượng không đổi, sự thay đổi của gia tốc tương ứng với sự thay đổi của lực tác dụng lên vật. Nếu gia tốc thay đổi đột ngột trong thời gian rất ngắn, điều đó tương đương với việc vật thể chịu một sự thay đổi lực lớn và bất thường. Vì vậy, các chuyển động có jerk lớn thường tạo cảm giác thiếu tự nhiên, giống như vật thể liên tục bị tác động bởi các ngoại lực mạnh.

Trong game trực tuyến, hiện tượng này thường xuất hiện khi Client nhận được dữ liệu trạng thái mới từ Server nhưng trạng thái này khác biệt đáng kể so với trạng thái đang được hiển thị cục bộ. Nếu Client áp dụng trực tiếp trạng thái mới, đối tượng có thể bị dịch chuyển tức thời đến vị trí mới. Ngay cả khi sai số vị trí không quá lớn, sự thay đổi đột ngột về hướng chuyển động, vận tốc hoặc gia tốc vẫn có thể khiến người chơi nhận thấy chuyển động không mượt.

Do đó, các kỹ thuật như Dead Reckoning hoặc SmoothDamp thường được sử dụng nhằm làm giảm sự thay đổi đột ngột giữa các trạng thái. Thay vì cập nhật vị trí một cách tức thời, hệ thống sẽ đưa đối tượng dần tiến về trạng thái mục tiêu theo thời gian. Quá trình chuyển tiếp này giúp tránh hiện tượng dịch chuyển đột ngột khi Client nhận được trạng thái mới từ Server. Cách tiếp cận này giúp giảm jerk, duy trì tính liên tục và mượt mà của chuyển động, cải thiện trải nghiệm thị giác của người chơi.

Có thể tóm tắt vai trò của các đại lượng vật lý trong đánh giá chuyển động như sau. Vị trí cho biết đối tượng đang ở đâu trong không gian. Vận tốc cho biết đối tượng di chuyển nhanh hay chậm. Gia tốc cho biết vận tốc thay đổi như thế nào. Trong khi đó, jerk phản ánh mức độ đột ngột trong sự thay đổi gia tốc, từ đó cho thấy chuyển động có mượt hay không.

Vì vậy, trong đánh giá các phương pháp đồng bộ hóa chuyển động trong game trực tuyến, việc chỉ xét sai số vị trí là chưa đủ. Một phương pháp có thể cho sai số vị trí nhỏ nhưng vẫn tạo cảm giác giật nếu quá trình hiệu chỉnh làm thay đổi gia tốc đột ngột. Ngược lại, một phương pháp tốt cần cân bằng giữa độ chính xác vị trí và độ mượt chuyển động, trong đó jerk là một chỉ số quan trọng để phản ánh mức độ ổn định của chuyển động theo thời gian.

2.3 Dead Reckoning

2.3.1 Giới thiệu

Dead Reckoning (DR) là kỹ thuật ước lượng trạng thái được sử dụng trong các hệ thống phân tán thời gian thực, đặc biệt là game trực tuyến, nhằm duy trì chuyển động liên tục của các thực thể khi dữ liệu mạng đến không đều hoặc bị trễ. Thay vì chỉ cập nhật vị trí của đối tượng khi nhận được gói tin mới từ Server, Client sử dụng trạng thái gần nhất đã nhận được để dự đoán trạng thái hiện tại của đối tượng.

Trong game trực tuyến, dữ liệu trạng thái từ Server thường được gửi theo từng tick hoặc từng gói tin rời rạc. Khoảng thời gian giữa hai lần nhận gói tin có thể lớn hơn nhiều so với thời gian giữa hai khung hình hiển thị trên Client. Nếu Client chỉ cập nhật vị trí đối tượng khi nhận trạng thái mới, chuyển động sẽ bị giật theo nhịp mạng. Dead Reckoning giải quyết vấn đề này bằng cách cho phép Client tự ước lượng trạng thái trung gian giữa các lần cập nhật mạng.

Về bản chất, Dead Reckoning không thay thế trạng thái chính thức của Server. Trong mô hình Server Authoritative, Server vẫn là nguồn sự thật trung tâm của hệ thống. Client chỉ sử dụng DR để hiển thị tạm thời và làm mượt chuyển động trong lúc chờ dữ liệu mới. Khi gói tin mới từ Server đến, Client sẽ so sánh trạng thái dự đoán với trạng thái chính thức và thực hiện hiệu chỉnh nếu cần thiết.

Nhờ đó, DR giúp tách biệt tần số hiển thị khỏi tần số cập nhật mạng. Client có thể hiển thị ở tốc độ cao hơn tần suất nhận trạng thái, trong khi các đối tượng vẫn duy trì được chuyển động tương đối liên tục. Đây là lý do DR thường được sử dụng cho các đối tượng có quỹ đạo dễ dự đoán như xe, tên lửa, vật thể bay, Non-Player Character (NPC) di chuyển theo quy luật hoặc các vật thể chịu ảnh hưởng rõ ràng của vật lý.

2.3.2 Nguyên lý hoạt động

Dead Reckoning thường bao gồm ba giai đoạn chính, bao gồm lưu trạng thái mới nhất từ Server, dự đoán trạng thái hiện tại trên Client và hiệu chỉnh sai lệch khi nhận được trạng thái mới [16]. Ba giai đoạn này tạo thành một vòng lặp đồng bộ liên tục, trong đó Client không chờ Server gửi trạng thái ở mọi khung hình mà chủ động ước lượng trạng thái trung gian của đối tượng. Nhờ đó, chuyển động của các đối tượng trong game có thể được duy trì liên tục ngay cả khi gói tin mạng đến không đều hoặc có độ trễ.

Khi Client nhận được một gói tin từ Server, gói tin này thường chứa các thông tin quan trọng như vị trí, vận tốc, hướng di chuyển, thời điểm tạo gói tin hoặc tick tương ứng. Client lưu lại gói tin này làm mốc dự đoán. Đây là trạng thái đáng tin cậy gần nhất do Server cung cấp và được sử dụng làm cơ sở để tính toán trạng thái tiếp theo của đối tượng. Trong thời gian chưa nhận được gói tin tiếp theo, Client sử dụng các thông tin đã có để ước lượng vị trí hiện tại của đối tượng.

Với dạng đơn giản nhất, DR giả thiết đối tượng tiếp tục chuyển động thẳng đều với vận tốc gần nhất nhận được từ Server. Khi đó, vị trí dự đoán được tính như sau:

$$\vec{P}_{predicted}(t) = \vec{P}_0 + \vec{V}_0 \Delta t, \quad (2.17)$$

trong đó $\vec{P}_0$ là vị trí cuối cùng nhận được từ Server, $\vec{V}_0$ là vận tốc tại thời điểm đó và Δt là khoảng thời gian đã trôi qua kể từ thời điểm gói tin được tạo hoặc được quy đổi sang thời gian cục bộ của Client.

Khi thực hiện dự đoán, Client cần kiểm tra sai lệch giữa trạng thái đang hiển thị và trạng thái dự đoán. Sai số này không chỉ thể hiện sai lệch giữa vị trí hiện tại và dự đoán mà còn thể hiện độ lệch của trạng thái hiện tại với trạng thái mới nhất

từ Server. Điều này là do DR luôn sử dụng trạng thái mới nhất từ Server để dự đoán, cho nên thao tác dự đoán ngay sau khi Client nhận trạng thái từ Server sẽ cho ra trạng thái gần nhất với trạng thái thực từ Server. Sai số vị trí có thể được tính như sau:

$$e = \|\vec{P}_{target} - \vec{P}_{current}\|, \quad (2.18)$$

trong đó $\vec{P}_{target}$ là vị trí đích cần hướng tới, còn $\vec{P}_{current}$ là vị trí hiện tại đang được hiển thị trên Client. Trong trường hợp này, $\vec{P}_{target}$ có thể là vị trí dự đoán từ trạng thái Server mới nhất, hoặc vị trí hiệu chỉnh được xác định sau khi Client nhận được gói tin cập nhật mới. Giá trị sai số e được dùng để quyết định Client nên tiếp tục làm mượt chuyển động hay cần dịch chuyển tức thời để đảm bảo đối tượng không lệch quá xa so với trạng thái hợp lệ.

Nếu sai số nhỏ hơn ngưỡng cho phép ϵ , Client có thể hiệu chỉnh dần bằng nội suy để tránh hiện tượng nhảy vị trí đột ngột. Nội suy tuyến tính thường được biểu diễn như sau:

$$\vec{P}_{new} = (1 - \alpha)\vec{P}_{current} + \alpha\vec{P}_{target}, \quad (2.19)$$

với $\alpha \in [0, 1]$ là hệ số nội suy. Giá trị α càng lớn thì đối tượng càng tiến nhanh về vị trí đích; giá trị α càng nhỏ thì chuyển động càng mượt nhưng thời gian hội tụ lâu hơn. Cơ chế này giúp che giấu một phần sai lệch nhỏ giữa trạng thái hiển thị trên Client và trạng thái do Server cung cấp, từ đó làm cho chuyển động có cảm giác liên tục hơn đối với người chơi.

Nếu sai số vượt quá ngưỡng cho phép, hệ thống có thể thực hiện hard snap, tức là dịch chuyển tức thời đối tượng về ngay vị trí đúng theo Server. Cách này đảm bảo độ chính xác nhưng có thể gây hiện tượng giật, đặc biệt khi khoảng cách giữa vị trí hiện tại và vị trí cần hiệu chỉnh quá lớn. Rubber banding là hiện tượng nhân vật bị kéo về vị trí cũ như là dây thun. Việc hard snap cũng có thể gây rubber banding nếu như Client nhận được gói tin vị trí trong quá khứ và hard snap tới đó. Vì vậy, ngưỡng sai số cần được lựa chọn cẩn thận để cân bằng giữa độ mượt và độ chính xác. Ngưỡng quá nhỏ có thể khiến hệ thống hiệu chỉnh liên tục, làm chuyển động dễ bị giật. Ngược lại, ngưỡng quá lớn có thể khiến đối tượng bị lệch xa khỏi trạng thái thực tế trong thời gian dài.

2.3.3 Mã giả thuật toán

Thuật toán Dead Reckoning cơ bản có thể được mô tả như sau:

Thuật toán 1 Dead Reckoning

- 1: Input:** *latestServerState* - trạng thái mới nhất nhận được từ Server; *currentTime* - thời gian hiện tại trên Client; *currentPosition* - vị trí hiện tại đang hiển thị; *errorThreshold* - ngưỡng sai số; *deltaTime* - thời gian giữa hai frame; *smoothTime* - thời gian làm mượt; *lastTimeRecievedFromServer* - thời điểm gần nhất nhận được thông tin từ Server
 - 2: Output:** *currentPosition* - vị trí mới sau khi xử lý Dead Reckoning
 - 3:** $serverPosition \leftarrow latestServerState.position$
 - 4:** $serverVelocity \leftarrow latestServerState.velocity$
 - 5:** $serverTimestamp \leftarrow latestServerState.timestamp$
 - 6:** $predictTime \leftarrow currentTime - lastTimeRecievedFromServer$
 - 7:** $targetPosition \leftarrow serverPosition + serverVelocity \cdot predictTime$
 - 8:** $error \leftarrow ||targetPosition - currentPosition||$
 - 9: if** $error > errorThreshold$ **then**
 - 10:** $currentPosition \leftarrow targetPosition$
 - 11: else**
 - 12:** $\alpha \leftarrow \frac{deltaTime}{predictTime}$
 - 13:** $currentPosition \leftarrow (1 - \alpha) \cdot currentPosition + \alpha \cdot targetPosition$
 - 14: return** $currentPosition$
-

Trong thuật toán 1, Client trước hết lấy vị trí và vận tốc mới nhất đã nhận được từ Server. Sau đó, hệ thống tính $predictTime$ dựa trên chênh lệch giữa thời gian hiện tại và thời điểm cuối cùng nhận được gói tin từ Server. Tiếp theo, Client

tính *targetPosition* bằng công thức chuyển động thẳng đều. Đây là vị trí mà Client cho rằng đối tượng nên ở tại thời điểm hiện tại. Sau đó, sai số giữa *targetPosition* và *currentPosition* được so sánh với ngưỡng *errorThreshold*. Nếu sai số lớn hơn ngưỡng, hệ thống thực hiện dịch chuyển tức thời để đưa đối tượng về vị trí dự đoán. Trường hợp này thường xảy ra khi đối tượng đổi hướng mạnh, vận tốc thay đổi đột ngột, dữ liệu mạng bị mất hoặc dự đoán đã sai lệch quá xa. Nếu sai số nhỏ hơn ngưỡng, Client không cập nhật tức thời mà nội suy dần từ vị trí hiện tại đến vị trí đích. Cách này giúp chuyển động mượt hơn và hạn chế hiện tượng nhảy vị trí do các sai lệch nhỏ.

2.3.4 Ưu điểm và hạn chế

Ưu điểm

Ưu điểm lớn nhất của Dead Reckoning là duy trì tính liên tục của chuyển động trong điều kiện dữ liệu mạng đến rời rạc. Client không cần chờ gói tin mới từ Server ở mỗi khung hình, mà có thể tự dự đoán trạng thái hiện tại dựa trên thông tin đã có. Điều này giúp chuyển động của các đối tượng từ xa trở nên mượt hơn.

DR tương đối đơn giản và dễ triển khai. Với các đối tượng có chuyển động ổn định, chỉ cần vị trí và vận tốc gần nhất là Client đã có thể dự đoán trạng thái trong một khoảng thời gian ngắn. Điều này làm DR phù hợp với các game có nhiều thực thể cần đồng bộ nhưng không phải thực thể nào cũng cần độ chính xác tuyệt đối ở mọi thời điểm. Tuy nhiên, DR không phải kỹ thuật toàn năng. DR cơ bản vẫn tồn tại nhiều hạn chế.

Nhược điểm

Sai số dự đoán có thể tăng cao. DR dựa trên giả thiết rằng đối tượng tiếp tục di chuyển theo xu hướng hiện tại. Khi đối tượng đổi hướng, tăng tốc, giảm tốc hoặc dừng đột ngột, giả thiết này không còn đúng. Sai số dự đoán khi đó có thể tăng nhanh theo thời gian.

Phụ thuộc vào chất lượng dữ liệu đầu vào. Nếu dữ liệu từ Server không chính xác, kết quả dự đoán trên Client cũng sẽ bị sai. Sai lệch thời gian cũng có thể làm Δt bị tính sai, khiến đối tượng bị dự đoán quá xa hoặc quá gần.

Chuyển động thiếu tự nhiên. Khi sử dụng nội suy tuyến tính, vận tốc hiển thị có thể thay đổi không đều nếu vị trí đích liên tục thay đổi. Điều này có thể tạo jerk lớn, khiến chuyển động có cảm giác cứng hoặc giật nhẹ.

Có thể xuất hiện rubber banding. Khi sai số vượt quá ngưỡng cho phép, Client phải đồng bộ mạnh về trạng thái đúng. Nếu việc hiệu chỉnh diễn ra tức thời, người chơi có thể thấy đối tượng bị kéo ngược hoặc nhảy vị trí đột ngột [17].

Tương tác đôi khi không chính xác. Trạng thái mà Client hiển thị là trạng thái dự đoán, không nhất thiết trùng với trạng thái chính thức trên Server. Vì vậy, các tương tác như va chạm, bắn trúng hoặc né tránh có thể bị lệch nếu chỉ dựa trên trạng thái hiển thị cục bộ.

2.3.5 Ứng dụng

Dead Reckoning phù hợp nhất với các đối tượng có chuyển động tương đối ổn định và dễ dự đoán. Ví dụ điển hình gồm xe đua, máy bay, tên lửa, vật thể bay theo quỹ đạo, NPC đi theo đường định sẵn hoặc các vật thể vật lý có quán tính rõ ràng. Trong game đua xe, DR thường phát huy hiệu quả tốt vì phương tiện có quán tính lớn, vận tốc và hướng di chuyển thường thay đổi liên tục nhưng không quá đột ngột. Khi Server gửi vị trí và vận tốc của xe theo từng gói tin, Client có thể dự đoán vị trí hiện tại của xe trong thời gian ngắn. Điều này giúp giảm hiện tượng giật hình, hạn chế va chạm ảo và duy trì cảm giác chuyển động ổn định trong điều kiện mạng không lý tưởng [16].

Đối với các đối tượng như tên lửa hoặc đạn bay chậm, quỹ đạo thường đơn giản và ít thay đổi sau khi được phóng ra. Vì vậy, Client có thể dự đoán vị trí của chúng khá chính xác bằng công thức chuyển động. Trong trường hợp này, DR giúp giảm lượng dữ liệu cần gửi qua mạng mà vẫn giữ được độ chính xác chấp nhận được.

Tuy nhiên, DR kém phù hợp hơn với các đối tượng do người chơi điều khiển trực tiếp và có khả năng đổi hướng liên tục. Ví dụ, trong các game bắn súng góc nhìn thứ nhất, nhân vật có thể đổi hướng, dừng lại hoặc tăng tốc rất nhanh theo input của người chơi. Khi đó, DR có thể dự đoán sai thường xuyên nếu chỉ dựa trên vận tốc hiện tại. Với các đối tượng này, hệ thống thường cần kết hợp thêm Client-side Prediction, Server Reconciliation và Lag Compensation.

2.3.6 Một số trò chơi sử dụng Dead Reckoning

Rocket League là game được vinh danh là game indie của năm 2015 do The Game Award bình chọn. Game này là ví dụ tiêu biểu cho việc sử dụng các kỹ thuật dự đoán và hiệu chỉnh trạng thái trong môi trường vật lý phức tạp. Trò chơi cần đồng bộ chuyển động của xe và quả bóng với độ chính xác cao, đồng thời vẫn phải duy trì cảm giác điều khiển mượt mà. Trong bối cảnh này, các kỹ thuật như DR, dự đoán và sửa lỗi được kết hợp để giảm tác động của độ trễ mạng [18, 19].

Missile Rally là dự án mã nguồn mở sử dụng Unity Netcode for GameObjects, trong đó có triển khai cơ chế đồng bộ chuyển động cho xe đua. Do đặc trưng chuyển động của xe có quán tính và vận tốc tương đối liên tục, *Missile Rally* phù hợp để làm nền tảng thử nghiệm cho DR. Trong khóa luận, *Missile Rally* được sử dụng làm môi trường thực nghiệm để đánh giá thuật toán DR, so sánh các bộ tham số đồng bộ và phân tích ảnh hưởng của độ trễ mạng đến độ chính xác cũng như độ mượt của chuyển động [20].

2.3.7 Ý nghĩa của Dead Reckoning trong khóa luận

DR được xem là một kỹ thuật trọng tâm để xử lý đồng bộ chuyển động trong môi trường mạng có độ trễ. Kỹ thuật này cho phép Client duy trì hiển thị liên tục cho các đối tượng từ xa bằng cách dự đoán vị trí dựa trên gói tin mới nhất từ Server.

Các yếu tố như Δt , vận tốc, timestamp, ngưỡng sai số và phương pháp làm mượt đều ảnh hưởng trực tiếp đến hiệu quả của DR. Nếu Δt được tính không chính xác, đối tượng có thể bị dự đoán sai vị trí. Nếu ngưỡng sai số không phù hợp, hệ thống có thể dịch chuyển tức thời quá thường xuyên hoặc cho phép sai lệch tồn tại quá lâu. Nếu phương pháp làm mượt không tốt, chuyển động có thể vẫn bị giật dù sai số dự đoán không lớn.

Do đó, việc nghiên cứu Dead Reckoning không chỉ dừng lại ở công thức dự đoán vị trí, mà còn bao gồm cách xử lý thời gian, cách hiệu chỉnh sai số và cách đánh giá độ mượt của chuyển động. Đây là cơ sở để khóa luận tiếp tục phân tích các cải tiến như ngưỡng sai số thích ứng, làm mượt, Server Reconciliation và các chỉ số đánh giá như độ chính xác và jerk.

Chương 3

Phương pháp đề xuất

3.1 Phân tích hạn chế của Dead Reckoning

DR là kỹ thuật có khả năng giảm ảnh hưởng của độ trễ mạng bằng cách dự đoán và nội suy trạng thái của các thực thể trong game. Tuy nhiên, kỹ thuật này vẫn tồn tại một số hạn chế khiến độ chính xác đồng bộ, độ mượt chuyển động và trải nghiệm điều khiển của người chơi chưa được đảm bảo. Các hạn chế này có thể được chia thành ba nhóm chính bao gồm **(1) Thời gian và lưu trữ trạng thái**, **(2) Dự đoán và hiệu chỉnh chuyển động**, và **(3) Xử lý input cũng như các tương tác quan trọng**.

Trước hết, DR phụ thuộc trực tiếp vào việc xác định chính xác khoảng thời gian dự đoán. Trong trường hợp không có cơ chế gắn nhãn thời gian, Client có thể xử lý trạng thái nhận được từ Server như thể trạng thái đó diễn ra tại thời điểm nhận gói tin. Tuy nhiên, do độ trễ mạng, trạng thái này thực chất đã được tạo ra ở một thời điểm trước đó trên Server. Việc sử dụng sai mốc thời gian làm cho giá trị Δt trong công thức dự đoán bị sai lệch, từ đó khiến trạng thái dự đoán không phản ánh chính xác trạng thái hiện tại của đối tượng. Bên cạnh đó, các kỹ thuật như hồi phục trạng thái hoặc phát lại input cũng yêu cầu Client và Server có cùng một hệ quy chiếu thời gian. Nếu input và trạng thái không được lưu trữ theo những mốc thời gian thống nhất, quá trình đối chiếu và đồng bộ có thể xảy ra sai lệch. Ngoài ra, hệ thống cũng không thể lưu trữ vô hạn toàn bộ lịch sử trạng thái và input trong suốt quá trình chơi, do điều này sẽ gây tốn kém lớn về bộ nhớ.

Nhóm hạn chế thứ hai nằm ở quá trình dự đoán và hiệu chỉnh của DR. Mô hình DR cơ bản thường dựa trên giả định chuyển động thẳng đều, tức trạng thái tương lai được ước lượng chủ yếu từ vị trí và vận tốc hiện tại. Giả định này chưa phù hợp với nhiều tình huống trong game, nơi vật thể có thể liên tục tăng tốc, giảm tốc hoặc thay đổi hướng di chuyển. Khi mô hình dự đoán không phản ánh đúng đặc điểm chuyển động, sai số giữa trạng thái dự đoán trên Client và trạng thái xác thực trên Server có thể tăng nhanh.

Quá trình hiệu chỉnh sai lệch cũng gặp hạn chế khi sử dụng một ngưỡng sai số cố định. Trong điều kiện mạng ổn định, ngưỡng cố định có thể hoạt động tương đối tốt. Tuy nhiên, khi độ trễ và jitter thay đổi, sai số phát sinh cũng thay đổi theo trạng thái mạng và trạng thái chuyển động của vật thể. Nếu ngưỡng quá thấp, hệ thống có thể hiệu chỉnh liên tục và gây hiện tượng giạt hoặc rubber banding. Ngược lại, nếu ngưỡng quá cao, trạng thái hiển thị trên Client có thể lệch đáng kể so với trạng thái xác thực trên Server. Bên cạnh đó, việc sử dụng nội suy tuyến tính để hiệu chỉnh tuy có thể giảm dịch chuyển tức thời, nhưng chưa xét đến quán tính và vận tốc hiện tại của vật thể, khiến chuyển động sau hiệu chỉnh có thể thiếu tự nhiên.

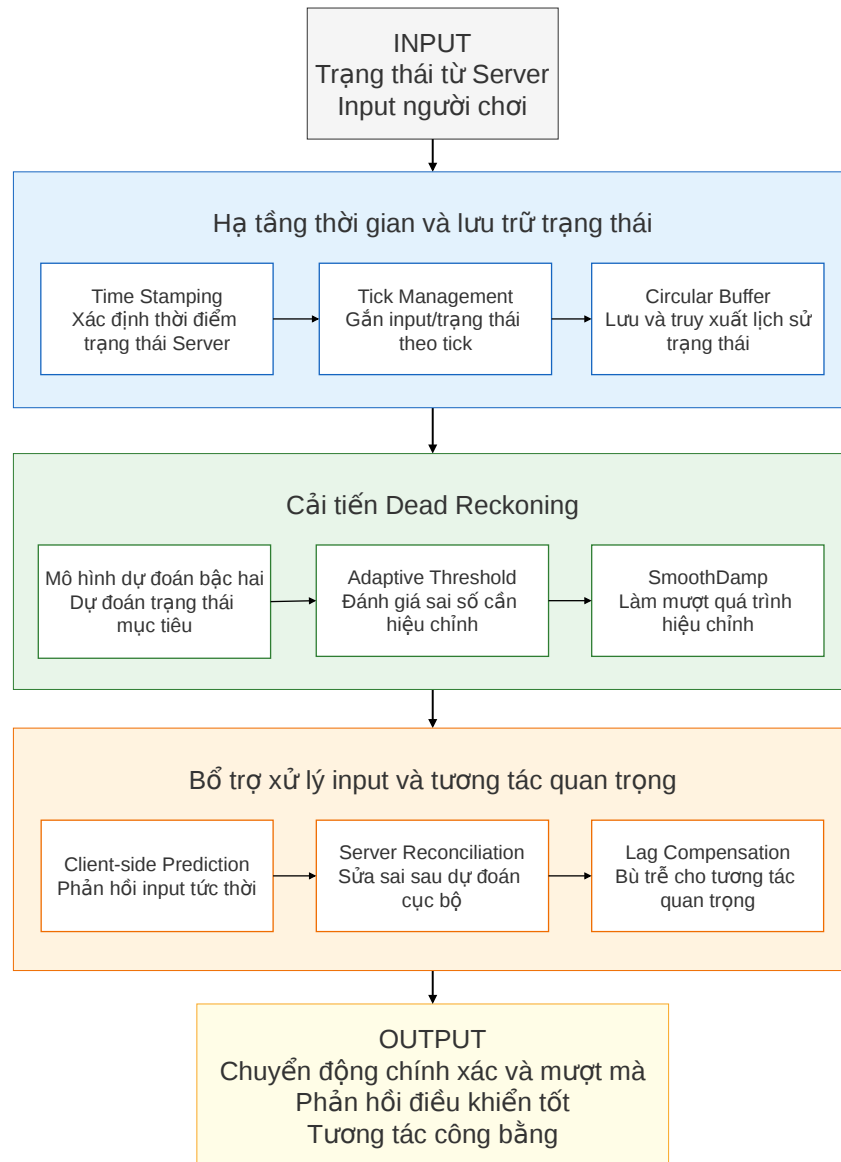
Nhóm hạn chế cuối cùng liên quan đến các tình huống mà DR cơ bản khó xử lý triệt để. Đối với thực thể do người chơi trực tiếp điều khiển, hành vi di chuyển phụ thuộc vào input tức thời của con người nên rất khó dự đoán chính xác chỉ bằng mô hình toán học. Nếu chỉ dựa vào DR, người chơi có thể cảm nhận rõ độ trễ hoặc thấy nhân vật không phản hồi đúng với thao tác điều khiển của mình. Tương tự, DR cũng không giải quyết đầy đủ các tương tác quan trọng như bắn, va chạm hoặc sử dụng kỹ năng. Do độ trễ mạng, input của người chơi có thể được Server xử lý trên một trạng thái đã khác với trạng thái mà người chơi quan sát tại thời điểm thực hiện hành động, từ đó làm cho kết quả xử lý trở nên thiếu chính xác hoặc thiếu công bằng.

Từ các hạn chế trên, có thể thấy rằng việc cải thiện DR không nên chỉ dừng lại ở việc thay đổi công thức dự đoán. Thay vào đó, cần đặt DR trong một kiến trúc đồng bộ trạng thái hoàn chỉnh, có khả năng giải quyết đồng thời vấn đề về thời gian, lưu trữ lịch sử trạng thái, dự đoán chuyển động, hiệu chỉnh sai lệch, phản hồi input cục bộ và xử lý các tương tác nhạy cảm với độ trễ.

3.2 Kiến trúc tổng thể của phương pháp đề xuất

Dựa trên các hạn chế đã phân tích, khóa luận đề xuất một kiến trúc đồng bộ trạng thái kết hợp giữa DR cải tiến và các cơ chế hỗ trợ trong mô hình Server Authoritative. Mục tiêu của kiến trúc này là xây dựng một quy trình đồng bộ thống nhất, trong đó mỗi thành phần đảm nhiệm một vai trò cụ thể thay vì tồn tại như các kỹ thuật rời rạc.

Kiến trúc tổng thể được tổ chức thành ba lớp chính, bao gồm lớp **Hạ tầng thời gian và lưu trữ trạng thái**, **Cải tiến DR**, và **Bổ trợ xử lý input** cũng như các tương tác quan trọng.



Hình 3.1: Luồng xử lý của phương pháp đề xuất

Hình 3.1 thể hiện luồng xử lý của phương pháp đề xuất. Đầu vào sẽ là trạng thái từ Server và input từ người chơi. Dữ liệu đầu vào sẽ được xử lý thông qua ba lớp tuần tự. Kết quả đầu ra là vật thể có chuyển động chính xác và mượt mà, phản hồi điều khiển tốt với input từ người chơi và tương tác công bằng hơn đối với người chơi có độ trễ mạng cao.

Lớp thứ nhất là hạ tầng thời gian và lưu trữ trạng thái. Lớp này cung cấp cơ sở cho toàn bộ quá trình đồng bộ bằng cách xác định mốc thời gian chính xác, tổ chức nhịp mô phỏng thống nhất và lưu trữ lịch sử trạng thái trong một phạm vi giới hạn. Trong lớp này, **Time Stamping** được sử dụng để giúp Client xác định đúng thời điểm mà trạng thái từ Server được tạo ra, từ đó tính toán chính xác hơn khoảng thời gian dự đoán. **Tick Management** hỗ trợ đồng bộ các bước mô phỏng giữa Client và Server, giúp input và trạng thái có thể được đối chiếu theo cùng một hệ quy chiếu. **Circular Buffer** được sử dụng để lưu trữ lịch sử input và trạng thái trong một khoảng thời gian gần nhất, phục vụ cho quá trình hiệu chỉnh, phát lại input và xử lý các tương tác cần truy xuất trạng thái quá khứ.

Lớp thứ hai tập trung cải tiến quá trình dự đoán và hiệu chỉnh của DR. Đối với quá trình dự đoán, **Mô hình dự đoán bậc hai** được sử dụng nhằm bổ sung thành phần gia tốc vào công thức ước lượng trạng thái tương lai, giúp phản ánh tốt hơn các chuyển động có thay đổi vận tốc. Đối với quá trình đánh giá sai lệch, **Adaptive Threshold** được áp dụng để điều chỉnh ngưỡng hiệu chỉnh theo điều kiện mạng và trạng thái chuyển động của vật thể, từ đó cân bằng giữa độ chính xác đồng bộ và độ mượt thị giác. Khi cần hiệu chỉnh trạng thái, **SmoothDamp** được sử dụng thay cho nội suy tuyến tính nhằm tạo ra quá trình dịch chuyển mềm mại hơn, có xét đến vận tốc và quán tính của chuyển động.

Lớp thứ ba bổ sung các cơ chế xử lý những tình huống mà DR cơ bản khó giải quyết triệt để. Đối với thực thể do người chơi điều khiển, **Client-side Prediction** được sử dụng để phản hồi input ngay lập tức trên Client, giúp giảm cảm giác trễ khi điều khiển. Do trạng thái dự đoán cục bộ có thể sai lệch so với trạng thái xác thực từ Server, **Server Reconciliation** được sử dụng để hiệu chỉnh lại trạng thái bằng cách so sánh với dữ liệu Server và phát lại các input chưa được xác nhận. Đối với các tương tác quan trọng như bắn hoặc va chạm, **Lag Compensation** được sử dụng để Server có thể đánh giá hành động dựa trên trạng thái quá khứ gần với thời điểm người chơi thực hiện input, từ đó cải thiện tính chính xác và công bằng trong điều kiện mạng có độ trễ.

Như vậy, kiến trúc đề xuất không xem DR là một kỹ thuật độc lập, mà đặt kỹ thuật này trong một quy trình đồng bộ trạng thái hoàn chỉnh. Các thành phần trong kiến trúc có vai trò bổ trợ lẫn nhau, bao gồm hạ tầng thời gian và lưu trữ trạng thái cung cấp nền tảng cho việc đối chiếu dữ liệu, các cải tiến DR giúp nâng

cao độ chính xác dự đoán và độ mượt chuyển động, trong khi các cơ chế hỗ trợ giúp xử lý phản hồi input và các tương tác nhạy cảm với độ trễ. Sự kết hợp này hướng tới mục tiêu cân bằng giữa độ chính xác đồng bộ, độ mượt hiển thị và trải nghiệm điều khiển của người chơi trong game trực tuyến thời gian thực.

3.3 Hạ tầng thời gian và lưu trữ trạng thái

3.3.1 Time Stamping

Trong DR, giá trị Δt đóng vai trò quan trọng vì nó quyết định khoảng thời gian mà Client cần dự đoán từ trạng thái cuối cùng nhận được từ Server đến thời điểm hiện tại. Nếu Δt được tính không chính xác, trạng thái dự đoán cũng sẽ bị sai lệch, khiến đối tượng có thể di chuyển nhanh, chậm hoặc rung giật bất thường.

Trong trường hợp không sử dụng Time Stamping, Client thường chỉ có thể lấy thời điểm nhận gói tin làm mốc xử lý trạng thái mới. Tuy nhiên, do ảnh hưởng của độ trễ mạng, trạng thái trong gói tin không phải là trạng thái tại thời điểm Client nhận được, mà là trạng thái đã được tạo ra ở một thời điểm trước đó trên Server. Vì vậy, nếu Client dùng thời điểm nhận gói tin để tính toán, DR sẽ bỏ qua khoảng thời gian truyền qua mạng và dẫn đến sai lệch trong dự đoán.

Để khắc phục vấn đề này, mỗi gói tin trạng thái do Server gửi đi cần được gắn kèm một nhãn thời gian *timestamp*. Timestamp này biểu diễn thời điểm trạng thái được tạo ra trong hệ thời gian của Server. Một gói tin trạng thái gửi từ Server có thể bao gồm các thông tin chính như sau:

$$StatePacket = \{position, velocity, serverTimestamp\} \quad (3.1)$$

Trong đó, *serverTimestamp* là thời điểm trạng thái được tạo ra trên Server. Khi nhận được gói tin, Client không sử dụng trực tiếp thời điểm nhận gói tin để dự đoán, mà sử dụng *serverTimestamp* kết hợp với thời gian Server hiện tại do Client ước lượng.

Thời gian thực tế trên Server và Client thường không giống nhau. Điều này xảy ra do bộ đếm trên máy hoặc chương trình đang chạy không hoàn toàn là đồng nhất. Ví dụ, trong Unity có hàm *Time.time* sẽ trả về thời gian chạy của chương trình, nếu chương trình khác nhau sẽ cho ra kết quả thời gian khác nhau. Kể cả khi lấy

thời gian trên máy tính cũng không thể đảm bảo các máy hoàn toàn đồng nhất về thời gian. Vậy nên Client sẽ cần ước lượng thời gian của Server thông qua độ lệch thời gian θ .

Để ước lượng thời gian Server hiện tại, Client cần xác định độ lệch giữa đồng hồ cục bộ và đồng hồ Server. Giả sử độ trễ truyền theo hai chiều gần đối xứng, độ trễ một chiều có thể được ước lượng bởi:

$$L_{latency} \approx \frac{RTT}{2} \quad (3.2)$$

Từ đó, độ lệch giữa đồng hồ Server và đồng hồ Client được xác định như sau:

$$\theta \approx (T_{packet} + L_{latency}) - T_{client} \quad (3.3)$$

Trong đó, θ là độ lệch thời gian cần cộng vào thời gian cục bộ của Client để ước lượng thời gian hiện tại trên Server.

Thời gian hiện tại trên Server được ước lượng qua công thức sau:

$$T_{server_now} = T_{client_now} + \theta \quad (3.4)$$

Trong thực tế, giá trị θ không nên được thay đổi đột ngột sau mỗi lần ping, vì bản thân kết quả đo cũng có thể bị nhiễu bởi jitter. Do đó, Client có thể làm mượt giá trị độ lệch thời gian bằng hệ số α :

$$\theta_{new} = (1 - \alpha)\theta_{old} + \alpha\theta_{measured} \quad (3.5)$$

Trong đó, $\theta_{measured}$ là độ lệch vừa đo được từ gói tin đồng bộ mới, còn α là hệ số làm mượt. Giá trị α nhỏ giúp độ lệch thời gian thay đổi ổn định hơn, tránh việc thời gian Server ước lượng bị dao động mạnh khi mạng xuất hiện jitter.

Sau khi có θ , quá trình tính thời gian dự đoán cho DR được thực hiện như sau. Khi Client nhận một gói tin trạng thái có timestamp là T_{packet} , Client ước lượng thời gian hiện tại của Server bằng $T_{server_now} = T_{client_now} + \theta$. Khi đó, khoảng thời gian dự đoán được xác định bởi:

$$\Delta t = T_{server_now} - T_{packet} \quad (3.6)$$

Giá trị Δt cũng nên được giới hạn bởi một ngưỡng Δt_{max} . Việc giới hạn này nhằm tránh trường hợp gói tin trạng thái đã quá cũ nhưng vẫn được sử dụng để dự đoán trạng thái hiện tại của đối tượng trong một khoảng thời gian quá dài. Khi mô hình dự đoán phải dự đoán tới một thời điểm quá xa so với thời điểm tạo gói tin, sai số có thể tăng nhanh, đặc biệt nếu đối tượng đã thay đổi vận tốc, đổi hướng hoặc chịu tác động từ môi trường. Do đó, ngưỡng Δt_{max} giúp hạn chế sai lệch lớn và giảm nguy cơ làm mô hình dự đoán mất ổn định. Giá trị Δt sau khi được tính toán và giới hạn sẽ được đưa vào mô hình dự đoán của DR. Với mô hình dự đoán bậc một, vị trí hiện tại của đối tượng có thể được ước lượng bởi:

$$\vec{P}(t) = \vec{P}_{packet} + \vec{V}_{packet}\Delta t \quad (3.7)$$

Trong đó, $\vec{P}_{packet}$ và $\vec{V}_{packet}$ lần lượt là vị trí và vận tốc được gửi trong gói tin trạng thái từ Server. Công thức trên cho thấy vị trí hiện tại của đối tượng được ước lượng bằng cách lấy vị trí tại thời điểm gói tin được tạo ra, sau đó cộng thêm phần dịch chuyển dự đoán dựa trên vận tốc và khoảng thời gian Δt . Nhờ cách tính này, Client có thể dự đoán trạng thái của đối tượng dựa trên thời điểm thật sự mà trạng thái được tạo ra tại Server, thay vì chỉ dựa trên thời điểm gói tin được nhận ở phía Client.

Như vậy, Time Stamping giúp DR sử dụng đúng mốc thời gian của trạng thái Server khi tính toán dự đoán. Kỹ thuật này không loại bỏ hoàn toàn độ trễ mạng, vì dữ liệu vẫn cần một khoảng thời gian nhất định để truyền từ Server đến Client. Tuy nhiên, Time Stamping giúp giảm sai lệch do việc sử dụng nhầm thời điểm nhận gói tin làm thời điểm tạo trạng thái. Điều này đặc biệt quan trọng trong môi trường mạng có độ trễ hoặc jitter, khi thời điểm gói tin đến Client có thể dao động và không phản ánh chính xác thời điểm trạng thái được sinh ra trên Server.

Trong các cơ chế cần đối chiếu hoặc hồi phục trạng thái như Client-side Prediction, Server Reconciliation và Lag Compensation, hệ thống không chỉ cần biết thời gian liên tục mà còn cần một mốc logic rời rạc để gắn input và trạng thái. Vì vậy, bên cạnh timestamp, khóa luận sử dụng thêm Tick Management để tổ chức quá trình mô phỏng theo các bước thời gian cố định.

3.3.2 Tick Management

Tick Management là cơ chế chia thời gian mô phỏng thành các bước rời rạc có độ dài cố định. Thay vì để logic game phụ thuộc trực tiếp vào tốc độ khung hình, hệ thống sử dụng một tần số cập nhật logic riêng gọi là *serverTickRate*. Khi đó, khoảng thời gian giữa hai tick liên tiếp được xác định bởi:

$$\Delta t_{tick} = \frac{1}{serverTickRate} \quad (3.8)$$

Mỗi tick đại diện cho một bước mô phỏng logic của hệ thống. Tại mỗi tick, Client có thể đọc input, lưu input vào bộ đệm, gửi input lên Server và tự mô phỏng trạng thái cục bộ. Tương tự, Server xử lý các input nhận được, cập nhật trạng thái xác thực và gửi gói tin trạng thái về Client. Nhờ đó, input và trạng thái không chỉ được xác định theo thời gian thực, mà còn được gắn với một chỉ số tick cụ thể để phục vụ đối chiếu và hồi phục sau này. Cách tổ chức này giúp quá trình theo dõi và tái lập trạng thái mô phỏng trở nên rõ ràng hơn.

Bên cạnh đó, Tick Management còn giúp tách biệt quá trình mô phỏng logic khỏi tốc độ hiển thị khung hình. Trong thực tế, tốc độ khung hình trên Client có thể thay đổi tùy theo hiệu năng phần cứng hoặc tải xử lý tại từng thời điểm. Nếu logic mô phỏng phụ thuộc trực tiếp vào tốc độ khung hình, cùng một hành động có thể tạo ra kết quả khác nhau trên các thiết bị khác nhau. Việc sử dụng tick với khoảng thời gian cố định giúp hệ thống duy trì quá trình cập nhật ổn định hơn, đồng thời tạo điều kiện thuận lợi cho việc đồng bộ, ghi lại và phát lại trạng thái mô phỏng.

Việc gắn input và trạng thái với tick giúp Client và Server có cùng một hệ quy chiếu thời gian khi so sánh kết quả mô phỏng. Điều này đặc biệt quan trọng trong Client-side Prediction và Server Reconciliation, vì Client cần xác định input nào đã được Server xử lý, trạng thái xác thực thuộc tick nào và cần mô phỏng lại từ thời điểm nào khi xảy ra sai lệch. Nhờ đó, quá trình hiệu chỉnh có thể dựa trên lịch sử input và trạng thái theo từng tick, thay vì chỉ dựa vào vị trí hiện tại, giúp hạn chế hiệu chỉnh sai thời điểm trên Client.

Về mặt triển khai, hệ thống cần duy trì một số biến chính gồm *currentTick*, *timer* và Δt_{tick} . Biến *timer* được dùng để tích lũy thời gian trôi qua giữa các lần cập nhật khung hình. Khi *timer* lớn hơn hoặc bằng Δt_{tick} , hệ thống thực hiện một bước cập nhật logic, tăng *currentTick* lên một đơn vị, sau đó trừ Δt_{tick} khỏi *timer*.

Quy trình hoạt động của Tick Management được mô tả như sau:

Thuật toán 2 Tick Management

- 1: Input:** $\delta\text{eltaTime}$ - Thời gian trôi qua giữa hai lần cập nhật, Δt_{tick} - Khoảng thời gian giữa hai tick, currentTick - Tick hiện tại, timer - Thời gian tích lũy chưa xử lý
 - 2: Output:** currentTick - Tick sau khi cập nhật, timer - Thời gian còn dư sau khi xử lý
 - 3:** $\text{timer} \leftarrow \text{timer} + \delta\text{eltaTime}$
 - 4: while** $\text{timer} \geq \Delta t_{\text{tick}}$ **do**
 - 5:** $\text{currentTick} \leftarrow \text{currentTick} + 1$
 - 6:** thực hiện cập nhật logic của game tại tick currentTick
 - 7:** $\text{timer} \leftarrow \text{timer} - \Delta t_{\text{tick}}$
 - 8: end while**
 - 9: return** $\text{currentTick}, \text{timer}$
-

Trong thuật toán 2, $\delta\text{eltaTime}$ là thời gian thực trôi qua giữa hai lần cập nhật khung hình. Trong vòng lặp while, nếu chưa đủ thời gian cho một tick mới, hệ thống chưa cập nhật logic mà tiếp tục tích lũy thời gian ở các khung hình sau.

Khi áp dụng vào đồng bộ mạng, mỗi gói input từ Client cần được gắn với tick tại thời điểm input được tạo ra:

$$\text{InputPacket} = \{\text{tick}, \text{input}\} \quad (3.9)$$

Tương tự, mỗi gói trạng thái từ Server cũng cần chứa tick tương ứng với trạng thái được mô phỏng:

$$\text{StatePacket} = \{\text{tick}, \text{position}, \text{velocity}\} \quad (3.10)$$

Nhờ có tick, Server biết input của Client thuộc về bước mô phỏng nào, còn Client có thể xác định trạng thái Server gửi về tương ứng với trạng thái cục bộ nào đã được lưu trước đó. Đây là cơ sở quan trọng để triển khai Server Reconciliation. Khi nhận một *StatePacket* tại tick t_s , Client truy xuất trạng thái đã lưu tại tick t_s trong bộ đệm, so sánh với trạng thái xác thực từ Server, sau đó quyết định có cần hiệu chỉnh và phát lại các input sau tick đó hay không.

Tick Management cũng hỗ trợ Lag Compensation. Server có thể lưu lịch sử trạng thái của các thực thể theo tick. Khi nhận được một input quan trọng như bắn hoặc va chạm từ Client, Server có thể dựa vào tick của input để truy xuất trạng thái quá khứ gần với thời điểm người chơi thực hiện hành động, thay vì chỉ kiểm tra trên trạng thái hiện tại.

Tuy nhiên, Tick Management không thay thế hoàn toàn cho Time Stamping. Tick phù hợp để đánh chỉ số các bước mô phỏng rời rạc, đặc biệt hữu ích cho việc lưu input, lưu trạng thái, sửa lỗi và tua ngược. Trong khi đó, Time Stamping phù hợp hơn để tính khoảng thời gian dự đoán liên tục trong DR. Vì vậy, trong phương pháp đề xuất, tick được dùng làm mốc logic cho lưu trữ và đối chiếu trạng thái, còn timestamp được dùng để tính chính xác Δt khi dự đoán chuyển động.

Việc lựa chọn *serverTickRate* cần cân bằng giữa độ chính xác mô phỏng và chi phí xử lý. Nếu *serverTickRate* quá cao, khoảng thời gian giữa các tick nhỏ hơn, giúp hệ thống cập nhật trạng thái thường xuyên hơn nhưng làm tăng số lần xử lý logic và số lượng dữ liệu cần lưu trữ. Ngược lại, nếu *serverTickRate* quá thấp, chi phí xử lý giảm nhưng trạng thái được cập nhật thưa hơn, khiến quá trình dự đoán, reconciliation hoặc lag compensation kém chính xác hơn. Do đó, giá trị này cần được lựa chọn dựa trên đặc điểm gameplay, tốc độ chuyển động của đối tượng và khả năng xử lý của hệ thống.

Tóm lại, Tick Management đóng vai trò tạo ra một trục thời gian logic thống nhất cho Client và Server. Nhờ đó, input và trạng thái trung gian có thể được lưu trữ, truy xuất và đối chiếu theo cùng một chỉ số tick. Đây là nền tảng cần thiết giúp hệ thống đồng bộ trạng thái có tổ chức và dễ kiểm soát hơn.

Để các dữ liệu theo tick có thể được sử dụng cho quá trình đối chiếu, hiệu chỉnh hoặc bù trễ, hệ thống cần một cơ chế lưu trữ lịch sử trạng thái hiệu quả. Do đó, Circular Buffer được sử dụng để lưu trữ input và trạng thái gần nhất theo tick, cho phép hệ thống truy xuất lại dữ liệu quá khứ với chi phí thấp.

3.3.3 Circular Buffer

Circular Buffer là cấu trúc dữ liệu dạng vòng được sử dụng để lưu trữ một số lượng hữu hạn các trạng thái hoặc input gần nhất của hệ thống. Khi số lượng phần tử vượt quá kích thước bộ đệm, dữ liệu mới sẽ ghi đè lên dữ liệu cũ nhất. Cách tổ chức này phù hợp với game trực tuyến thời gian thực, vì hệ thống thường chỉ cần truy xuất một khoảng lịch sử ngắn gần thời điểm hiện tại để phục vụ đối chiếu, hiệu chỉnh hoặc bù trễ, thay vì lưu toàn bộ quá trình chơi.

Trong phương pháp đề xuất, Circular Buffer được sử dụng để lưu các dữ liệu có gắn tick, như input của người chơi, trạng thái cục bộ do Client dự đoán, gói tin xác thực từ Server và lịch sử trạng thái phục vụ Lag Compensation. Mỗi phần tử trong bộ đệm không chỉ lưu dữ liệu, mà còn cần lưu tick tương ứng để kiểm tra tính hợp lệ khi truy xuất. Một phần tử trong bộ đệm có thể bao gồm các thành phần như sau:

$$BufferEntry = \{tick, data, isValid\} \quad (3.11)$$

Trong đó, *tick* là chỉ số tick của dữ liệu được lưu, *data* là dữ liệu cần lưu tại *tick*, còn *isValid* cho biết ô dữ liệu hiện tại có đang chứa thông tin hợp lệ hay không.

Với bộ đệm có kích thước N , vị trí lưu trữ dữ liệu tại tick t được xác định bằng phép chia lấy dư:

$$index = t \bmod N \quad (3.12)$$

Khi cần lưu dữ liệu tại tick t , hệ thống tính $index$, sau đó ghi dữ liệu mới vào vị trí tương ứng trong bộ đệm. Nếu vị trí này đã chứa dữ liệu cũ, dữ liệu cũ sẽ bị ghi đè.

Khi cần truy xuất lại dữ liệu tại tick t , hệ thống cũng tính $index = t \bmod N$. Tuy nhiên, do Circular Buffer có cơ chế ghi đè, không thể chỉ dựa vào $index$ để kết luận rằng dữ liệu tại vị trí đó đúng là dữ liệu của tick cần tìm. Vì vậy, sau khi truy xuất phần tử tại $index$, hệ thống phải kiểm tra lại trường *tick* được lưu trong phần tử đó. Nếu $buffer[index].tick = t$ và $buffer[index].isValid = true$, dữ liệu được xem là hợp lệ. Ngược lại, dữ liệu tại vị trí đó đã bị ghi đè hoặc chưa tồn tại, nên không thể sử dụng cho quá trình hồi phục.

Thuật toán 3 Truy xuất dữ liệu từ Circular Buffer

1: Input: *buffer* - Bộ đệm dữ liệu, *N* - Kích thước bộ đệm, *tick* - Giá trị tick cần lấy dữ liệu

2: Output: *data* - Dữ liệu tại *tick* nếu hợp lệ

3: *data* $\leftarrow \emptyset$

4: *index* $\leftarrow tick \bmod N$

5: if *buffer*[*index*].*isValid* = *true* **and** *buffer*[*index*].*tick* == *tick* **then**

6: *data* $\leftarrow buffer[index]$

7: end if

8: return *data*

Thuật toán 3 thể hiện cơ chế giúp Circular Buffer truy xuất dữ liệu theo tick với chi phí hằng số $O(1)$, đồng thời tránh việc phải dịch chuyển hoặc cấp phát lại bộ nhớ khi thêm phần tử mới. Đây là ưu điểm quan trọng trong môi trường game thời gian thực, nơi thao tác lưu input, lưu trạng thái và đọc lại lịch sử có thể xảy ra liên tục ở mỗi tick.

Circular Buffer là thành phần cần thiết cho Server Reconciliation. Khi Client nhận được trạng thái xác thực từ Server tại tick t_s , Client có thể truy xuất trạng thái cục bộ đã dự đoán tại cùng tick đó trong bộ đệm. Sau đó, hệ thống so sánh trạng thái dự đoán với trạng thái Server. Nếu sai lệch vượt quá ngưỡng cho phép, Client khôi phục trạng thái về dữ liệu Server tại tick t_s , rồi phát lại các input đã lưu trong bộ đệm từ tick $t_s + 1$ đến tick hiện tại. Nhờ đó, Client có thể hiệu chỉnh sai lệch mà không cần mô phỏng lại toàn bộ quá trình từ đầu.

Circular Buffer cũng hỗ trợ Lag Compensation ở phía Server. Server có thể lưu trạng thái của các thực thể theo tick. Khi nhận một input quan trọng từ Client, chẳng hạn như bắn súng, Server sử dụng tick hoặc timestamp của input để truy xuất trạng thái quá khứ gần thời điểm người chơi thực hiện hành động. Từ đó, Server có thể đánh giá tương tác dựa trên trạng thái mà người chơi đã quan sát.

Việc lựa chọn kích thước buffer N cần được cân nhắc dựa trên độ trễ mạng tối đa mà hệ thống muốn hỗ trợ, tick rate của Server và một khoảng dự phòng cho jitter. Nếu $T_{history}$ là khoảng thời gian lịch sử cần lưu, kích thước buffer có thể được ước lượng bởi:

$$N \geq serverTickRate \cdot T_{history} \quad (3.13)$$

Trong đó, $T_{history}$ nên đủ lớn để bao phủ độ trễ mạng, jitter và thời gian xử lý cần thiết cho Server Reconciliation hoặc Lag Compensation. Nếu N quá nhỏ, dữ liệu tại tick cần truy xuất có thể đã bị ghi đè, khiến hệ thống không thể hồi phục hoặc kiểm tra trạng thái quá khứ. Ngược lại, nếu N quá lớn, bộ đệm sẽ tiêu tốn nhiều bộ nhớ hơn và có thể làm tăng chi phí xử lý khi hệ thống phải duyệt hoặc phát lại quá nhiều input.

Tóm lại, Circular Buffer đóng vai trò là vùng lưu trữ lịch sử ngắn hạn cho hệ thống đồng bộ trạng thái. Bằng cách ánh xạ tick vào vị trí lưu trữ thông qua phép chia lấy dư và kiểm tra lại tick khi truy xuất, hệ thống có thể lưu và đọc input hoặc trạng thái một cách nhanh chóng, ổn định và tiết kiệm bộ nhớ. Đây là nền tảng quan trọng cho các kỹ thuật cần truy xuất dữ liệu quá khứ như Server Reconciliation và Lag Compensation.

Nhóm giải pháp hạ tầng gồm Time Stamping, Tick Management và Circular Buffer tạo ra nền tảng thời gian và dữ liệu cho toàn bộ quá trình đồng bộ trạng thái. Trong đó, timestamp hỗ trợ tính toán thời gian dự đoán chính xác hơn, tick cung cấp mốc logic để gắn input và trạng thái, còn Circular Buffer cho phép lưu trữ và truy xuất lịch sử gần nhất. Trên nền tảng này, khóa luận tiếp tục cải tiến trực tiếp quá trình dự đoán và hiệu chỉnh của DR nhằm nâng cao độ chính xác và độ mượt chuyển động.

3.4 Cải tiến DR

3.4.1 Mô hình dự đoán bậc hai

Mô hình dự đoán bậc hai được sử dụng nhằm cải thiện khả năng dự đoán trạng thái của DR trong các tình huống vật thể có vận tốc thay đổi theo thời gian. Trong mô hình DR cơ bản, Client thường dự đoán trạng thái tương lai của vật thể dựa

trên vị trí và vận tốc gần nhất nhận được từ Server. Cách tiếp cận này phù hợp với chuyển động thẳng đều, nhưng chưa phản ánh tốt các chuyển động thường gặp trong game như tăng tốc, giảm tốc hoặc đổi hướng.

Để khắc phục hạn chế trên, mô hình dự đoán bậc hai bổ sung thêm thành phần gia tốc vào công thức dự đoán. Khi Client nhận được một gói trạng thái từ Server, gói tin cần cung cấp các thông tin cơ bản gồm vị trí, vận tốc, gia tốc và thời điểm trạng thái được tạo ra:

$$StatePacket = \{\vec{P}_0, \vec{V}_0, \vec{A}_0, T_{packet}\} \quad (3.14)$$

Trong đó, $\vec{P}_0$, $\vec{V}_0$ và $\vec{A}_0$ lần lượt là vị trí, vận tốc và gia tốc của vật thể tại thời điểm T_{packet} . Sau khi nhận gói tin, Client sử dụng cơ chế Time Stamping để tính khoảng thời gian dự đoán:

$$\Delta t = T_{server_now} - T_{packet} \quad (3.15)$$

Khi đó, vị trí và vận tốc dự đoán tại thời điểm hiện tại được xác định bởi:

$$\vec{P}_{predicted} = \vec{P}_0 + \vec{V}_0\Delta t + \frac{1}{2}\vec{A}_0\Delta t^2 \quad (3.16)$$

$$\vec{V}_{predicted} = \vec{V}_0 + \vec{A}_0\Delta t \quad (3.17)$$

Việc bổ sung gia tốc giúp mô hình phản ánh tốt hơn các chuyển động có vận tốc biến thiên. Trong các tình huống vật thể tăng tốc, giảm tốc hoặc đổi hướng, mô hình bậc hai thường cho kết quả dự đoán gần với trạng thái thực hơn so với mô hình tuyến tính chỉ sử dụng vận tốc [21]. Khi trạng thái dự đoán gần hơn với trạng thái xác thực từ Server, sai số tích lũy sẽ tăng chậm hơn, từ đó giảm số lần phải hiệu chỉnh mạnh hoặc dịch chuyển tức thời.

Tuy nhiên, việc tăng bậc mô hình dự đoán không phải lúc nào cũng mang lại hiệu quả tốt hơn. Về tổng quát, chuyển động của vật thể có thể được biểu diễn bằng khai triển Taylor theo thời gian:

$$\vec{P}(t) = \vec{P}_0 + \vec{V}_0\Delta t + \frac{1}{2}\vec{A}_0\Delta t^2 + \frac{1}{6}\vec{J}_0\Delta t^3 + \dots \quad (3.18)$$

Trong đó, $\vec{J}_0$ là jerk, tức đạo hàm theo thời gian của gia tốc. Biểu thức trên cho thấy khi bậc của mô hình tăng lên, các số hạng bậc cao phụ thuộc mạnh hơn vào Δt . Nếu Δt nhỏ, đóng góp của các thành phần bậc cao thường không đáng kể. Ngược lại, nếu Δt lớn, các thành phần bậc cao có thể khuếch đại sai số nếu vận tốc, gia tốc hoặc jerk không được ước lượng ổn định.

Trong môi trường game trực tuyến, dữ liệu nhận được qua mạng có thể chịu ảnh hưởng của jitter, độ trễ mạng hoặc sai lệch đo đạc. Gia tốc và jerk cũng thường thay đổi đột ngột do input của người chơi, va chạm hoặc thay đổi trạng thái vật lý. Do đó, việc mở rộng mô hình lên các bậc cao hơn như jerk có thể làm tăng độ nhạy với nhiễu và khiến chuyển động dự đoán thiếu ổn định. Ngoài ra, mô hình bậc cao còn làm tăng chi phí tính toán và lượng dữ liệu cần đồng bộ [22].

Vì vậy, trong phạm vi khóa luận, mô hình dự đoán bậc hai được lựa chọn như một điểm cân bằng giữa độ chính xác, độ ổn định và chi phí triển khai. Mô hình này bổ sung đủ thông tin để cải thiện dự đoán trong các chuyển động biến đổi vận tốc, nhưng vẫn giữ được công thức đơn giản, dễ tích hợp vào DR và không yêu cầu lưu trữ quá nhiều dữ liệu lịch sử.

Dù mô hình dự đoán được cải thiện, sai số giữa trạng thái dự đoán và trạng thái xác thực từ Server vẫn có thể xuất hiện do độ trễ mạng, jitter, sai lệch đo đạc hoặc các thay đổi chuyển động đột ngột. Vì vậy, hệ thống cần một cơ chế đánh giá sai số phù hợp để quyết định khi nào cần hiệu chỉnh và mức độ hiệu chỉnh nên mạnh hay nhẹ. Đây là vai trò của Adaptive Threshold.

3.4.2 Adaptive Threshold

Trong DR, ngưỡng sai số ϵ được sử dụng để quyết định cách hệ thống hiệu chỉnh sai lệch giữa trạng thái hiện tại và trạng thái dự đoán trên Client hay trạng thái Server mới nhất. Nếu sai số nhỏ, Client có thể tiếp tục hiệu chỉnh để tránh gây giật hình. Ngược lại, nếu sai số quá lớn, hệ thống cần thực hiện hiệu chỉnh mạnh hơn để đưa trạng thái hiển thị về gần trạng thái chính xác.

Với ngưỡng sai số cố định, hệ thống khó thích ứng với các điều kiện mạng và trạng thái chuyển động khác nhau. Trong điều kiện độ trễ thấp và chuyển động chậm, ngưỡng nhỏ có thể giúp trạng thái Client bám sát Server. Tuy nhiên, khi độ trễ hoặc jitter tăng cao, sai số dự đoán có thể tăng nhanh hơn. Nếu vẫn giữ ngưỡng

quá thấp, Client sẽ thường xuyên hiệu chỉnh, gây hiện tượng rung giật hoặc rubber banding. Ngược lại, nếu đặt ngưỡng quá cao để phù hợp với điều kiện mạng xấu, trạng thái hiển thị trên Client có thể bị lệch đáng kể trong điều kiện mạng tốt.

Để giải quyết vấn đề này, khóa luận sử dụng cơ chế *Adaptive Threshold*, trong đó ngưỡng sai số được điều chỉnh dựa trên trạng thái mạng và trạng thái chuyển động của vật thể. Ý tưởng chính là cho phép ngưỡng sai số tăng hoặc giảm dựa trên việc vật thể di chuyển nhanh hay chậm, thay đổi vận tốc nhiều hay ít, độ trễ mạng lớn hay nhỏ.

Ngưỡng sai số thích ứng được xác định như sau:

$$\epsilon(t) = \epsilon_{base} + k_v \cdot \|\vec{v}(t)\| \cdot L(t) + \frac{1}{2} \cdot k_a \cdot \|\vec{a}(t)\| \cdot L(t)^2 \quad (3.19)$$

Trong đó, ϵ_{base} là ngưỡng sai số cơ bản, $\|\vec{v}(t)\|$ là độ lớn vận tốc của vật thể, $\|\vec{a}(t)\|$ là độ lớn gia tốc của vật thể, $L(t)$ là độ trễ mạng ước lượng tại thời điểm hiện tại, còn k_v và k_a là các hệ số điều chỉnh.

Thành phần $k_v \cdot \|\vec{v}(t)\| \cdot L(t)$ phản ánh sai số có thể phát sinh do vận tốc trong khoảng thời gian trễ. Khi vật thể di chuyển càng nhanh hoặc độ trễ càng lớn, khoảng cách mà vật thể có thể đi được trong thời gian đó càng tăng. Thành phần $\frac{1}{2} \cdot k_a \cdot \|\vec{a}(t)\| \cdot L(t)^2$ phản ánh ảnh hưởng của gia tốc, vì sai số do gia tốc tăng theo bình phương thời gian.

Trong triển khai thực tế, giá trị $\epsilon(t)$ không nên được sử dụng trực tiếp mà cần được giới hạn trong một khoảng cho phép:

$$\epsilon(t) = clamp(\epsilon(t), \epsilon_{min}, \epsilon_{max}) \quad (3.20)$$

Trong đó, ϵ_{min} đảm bảo hệ thống không quá nhạy với các sai số rất nhỏ, còn ϵ_{max} giới hạn mức sai lệch tối đa được phép chấp nhận trước khi cần hiệu chỉnh mạnh. Việc giới hạn này giúp tránh trường hợp ngưỡng tăng quá lớn trong điều kiện mạng xấu, khiến Client hiển thị trạng thái lệch xa so với Server.

Sau khi tính được ngưỡng thích ứng, Client tính sai số giữa trạng thái trạng thái hiện tại với trạng thái mục tiêu dự đoán:

$$e(t) = \|\vec{P}_{current}(t) - \vec{P}_{target}(t)\| \quad (3.21)$$

Trong đó, $\vec{P}_{current}(t)$ là vị trí được Client đang hiển thị, còn $\vec{P}_{target}(t)$ là vị trí mục tiêu được dự đoán. Nếu $e(t) \leq \epsilon(t)$, sai số được xem là chấp nhận được, Client thực hiện nội suy giữa hai trạng thái để đảm bảo độ mượt chuyển động. Nếu $e(t) > \epsilon(t)$, sai số vượt quá giới hạn cho phép, Client cần đưa trạng thái về trạng thái dự đoán bằng cơ chế dịch chuyển tức thời.

Cơ chế Adaptive Threshold cho phép hệ thống thay đổi mức độ nhạy của quá trình hiệu chỉnh theo điều kiện thực tế. Khi mạng ổn định và vật thể chuyển động chậm, ngưỡng sai số nhỏ giúp Client bám sát trạng thái Server. Khi độ trễ, vận tốc hoặc gia tốc tăng cao, ngưỡng sai số được mở rộng để giảm số lần hiệu chỉnh đột ngột, từ đó cải thiện độ mượt thị giác. Tuy nhiên, do Adaptive Threshold là cơ chế đánh đổi giữa độ chính xác và độ mượt, các tham số ϵ_{base} , k_v , k_a cần được hiệu chỉnh thực nghiệm dựa trên đặc điểm gameplay và điều kiện mạng mục tiêu.

Việc xác định rằng một trạng thái cần được hiệu chỉnh mới chỉ giải quyết câu hỏi khi nào cần sửa sai, chưa giải quyết cách sửa sai sao cho mượt mà. Nếu đối tượng bị đưa trực tiếp về trạng thái mục tiêu, chuyển động có thể xuất hiện hiện tượng giật hoặc dịch chuyển tức thời. Vì vậy, sau bước đánh giá sai số bằng Adaptive Threshold, khóa luận sử dụng SmoothDamp để làm mượt quá trình hiệu chỉnh trạng thái.

3.4.3 Làm mượt giảm chấn tối hạn SmoothDamp

Sau khi Client tính được trạng thái mục tiêu từ DR, đối tượng hiển thị không nên luôn được đưa ngay lập tức đến vị trí mục tiêu. Việc gán trực tiếp vị trí có thể gây hiện tượng dịch chuyển tức thời hoặc rung giật khi trạng thái từ Server thay đổi liên tục. Vì vậy, khóa luận sử dụng phương pháp làm mượt giảm chấn tối hạn *SmoothDamp* để hiệu chỉnh vị trí của đối tượng từ trạng thái hiện tại về trạng thái mục tiêu một cách mềm mại hơn.

Khác với *Lerp*, vốn chỉ nội suy tuyến tính giữa hai vị trí, *SmoothDamp* duy trì thêm một biến vận tốc hiệu chỉnh. Nhờ đó, chuyển động không chỉ phụ thuộc vào khoảng cách giữa vị trí hiện tại và vị trí mục tiêu, mà còn phụ thuộc vào vận tốc hiện tại của quá trình làm mượt. Điều này giúp quá trình hiệu chỉnh có tính quán tính hơn, tránh thay đổi trạng thái một cách đột ngột, bên cạnh đó là giúp chuyển động trông mượt mà và tự nhiên hơn.

Về mặt vật lý, *SmoothDamp* có thể được liên hệ với hệ lò xo giảm chấn ở trạng thái giảm chấn tới hạn [23, 24]. Xét sai số x giữa vị trí hiện tại và vị trí mục tiêu, lực kéo về mục tiêu và lực cản vận tốc có thể được mô tả bởi:

$$ma = -kx - bv \quad (3.22)$$

trong đó, m là khối lượng, k là hệ số đàn hồi, b là hệ số giảm chấn, x là sai số so với mục tiêu, và v là vận tốc vật thể. Khi hệ ở trạng thái giảm chấn tới hạn, là trạng thái mà hệ lò xo tiêu tán năng lượng nhanh nhất có thể để trở về vị trí cân bằng mà không xảy ra hiện tượng dao động lặp lại, hệ số giảm chấn được xác định bởi:

$$b = 2\sqrt{mk} \quad (3.23)$$

Phương trình chuyển động khi đó có dạng:

$$mx'' + bx' + kx = 0 \quad (3.24)$$

Đặt:

$$\omega = \sqrt{\frac{k}{m}} \quad (3.25)$$

ng nghiệm của hệ giảm chấn tới hạn có thể được biểu diễn dưới dạng [23]:

$$x(t) = (x_0 + (v_0 + \omega x_0)t)e^{-\omega t} \quad (3.26)$$

$$v(t) = (v_0 + \omega x_0 - \omega(x_0 + (v_0 + \omega x_0)t))e^{-\omega t} \quad (3.27)$$

Trong bối cảnh hiệu chỉnh trạng thái của DR, $x(t)$ không phải là vị trí tuyệt đối của đối tượng, mà là sai số giữa vị trí hiển thị hiện tại và vị trí mục tiêu. Gọi $\vec{P}_{current}$ là vị trí hiện tại đang hiển thị trên Client, $\vec{P}_{target}$ là vị trí mục tiêu được tính từ DR, sai số ban đầu được xác định bởi:

$$\vec{e}_0 = \vec{P}_{current} - \vec{P}_{target} \quad (3.28)$$

Sau một khoảng thời gian Δt , sai số mới là:

$$\vec{e}(\Delta t) = (\vec{e}_0 + (\vec{v}_0 + \omega \vec{e}_0)\Delta t) e^{-\omega \Delta t} \quad (3.29)$$

Từ đó, vị trí mới của đối tượng được xác định bởi:

$$\vec{P}_{new} = \vec{P}_{target} + \vec{e}(\Delta t) \quad (3.30)$$

Vận tốc hiệu chỉnh mới được tính bởi:

$$\vec{v}_{new} = (\vec{v}_0 + \omega \vec{e}_0 - \omega(\vec{e}_0 + (\vec{v}_0 + \omega \vec{e}_0)\Delta t)) e^{-\omega \Delta t} \quad (3.31)$$

Trong đó, $\vec{v}_0$ là vận tốc vật lý hiện tại của vật.

Vị trí sau khi nội suy SmoothDamp có thể được tính thông qua hàm như sau:

$$\vec{P}_{new} \leftarrow \text{SmoothDamp}(\vec{P}_{current}, \vec{P}_{target}, \vec{v}_0, \text{smoothTime}, \Delta t) \quad (3.32)$$

Trong đó, $\vec{P}_{target}$ là vị trí được tính từ mô hình dự đoán DR, smoothTime là tham số điều khiển tốc độ hội tụ về mục tiêu với công thức $\text{smoothTime} = \frac{2}{\omega}$. smoothTime càng nhỏ thì ω càng lớn, vật sẽ tiến tới $\vec{P}_{target}$ nhanh hơn.

So với Lerp, SmoothDamp phù hợp hơn cho hiệu chỉnh trạng thái trong DR vì nó không chỉ xét vị trí hiện tại và vị trí mục tiêu, mà còn duy trì vận tốc hiệu chỉnh qua thời gian.

Công thức của Lerp có dạng như sau:

$$\vec{P}(t) = (1 - \alpha)\vec{P}_{start} + \alpha\vec{P}_{end} \quad (3.33)$$

Công thức này chỉ mô tả nội suy hình học giữa hai điểm, không lưu lại trạng thái vận tốc của quá trình hiệu chỉnh. Do đó, khi mục tiêu thay đổi liên tục, vận tốc ngầm của chuyển động nội suy có thể thay đổi đột ngột, làm chuyển động trông máy móc và thiếu tự nhiên.

Ngược lại, SmoothDamp mô phỏng quá trình sai số suy giảm theo hàm mũ trong một hệ động lực học bậc hai giúp chuyển động ổn định, mềm mại và có quán tính hơn. Việc mô phỏng lại hệ lò xo cũng giúp vật chuyển động nhanh khi ở xa mục

tiêu và ngược lại, chậm dần khi gần tới đích. Điều này giúp cho chuyển động trông tự nhiên hơn so với khi sử dụng Lerp.

Tóm lại, trong phương pháp đề xuất, SmoothDamp được sử dụng ở bước hiệu chỉnh sau khi Client đã tính được trạng thái mục tiêu bằng DR. Kỹ thuật này giúp giảm hiện tượng giật cục khi đồng bộ trạng thái, đồng thời cải thiện độ mượt thị giác của chuyển động.

Các cải tiến trên giúp nâng cao hiệu quả của DR trong việc dự đoán, đánh giá sai số và làm mượt chuyển động của các thực thể từ xa. Tuy nhiên, DR vẫn không phù hợp để xử lý toàn bộ các tình huống trong mô hình Server Authoritative. Đặc biệt, đối với thực thể do người chơi trực tiếp điều khiển, hệ thống cần phản hồi input ngay lập tức để giảm cảm giác trễ. Ngoài ra, các tương tác quan trọng như bắn, va chạm hoặc sử dụng kỹ năng cũng cần được xử lý dựa trên thời điểm người chơi thực hiện hành động. Vì vậy, khóa luận bổ sung thêm nhóm giải pháp hỗ trợ gồm Client-side Prediction, Server Reconciliation và Lag Compensation.

3.5 Phương pháp hỗ trợ trong mô hình Server Authoritative

3.5.1 Client-side Prediction

Client-side Prediction là kỹ thuật cho phép Client chủ động mô phỏng trạng thái cục bộ ngay sau khi người chơi thực hiện input, thay vì phải chờ Server xử lý và phản hồi. Trong mô hình Server Authoritative, Server vẫn là nguồn xác thực cuối cùng, nhưng nếu mọi hành động đều phải đợi Server xác nhận rồi mới hiển thị, người chơi sẽ cảm nhận rõ độ trễ khi điều khiển. Vì vậy, Client-side Prediction được sử dụng để cải thiện độ phản hồi của thực thể do người chơi trực tiếp điều khiển [25].

Trong phương pháp đề xuất, Client-side Prediction được thực hiện theo từng tick mô phỏng. Tại mỗi tick, Client đọc input của người chơi, gắn input đó với tick hiện tại, lưu input vào bộ đệm, tự mô phỏng trạng thái cục bộ, sau đó gửi input lên Server để xử lý xác thực. Một gói input gửi từ Client có thể được biểu diễn như sau:

$$InputPacket = \{tick, input\} \quad (3.34)$$

Trong đó, *tick* là chỉ số tick tại thời điểm input được tạo ra, còn *input* là dữ liệu điều khiển của người chơi, chẳng hạn như hướng di chuyển, tăng tốc, phanh hoặc rẽ. Việc gắn tick vào input giúp Server biết input đó thuộc về bước mô phỏng nào, đồng thời giúp Client đối chiếu lại trạng thái khi nhận phản hồi từ Server.

Quy trình triển khai Client-side Prediction tại Client có thể được mô tả như sau:

Thuật toán 4 Client-side Prediction

- 1: Input:** *currentTick* - Tick hiện tại của Client, *localState* - Trạng thái cục bộ của Client, *inputBuffer* - Bộ đệm input của Client, *stateBuffer* - Bộ đệm trạng thái của Client
 - 2: Output:** *localState* - Trạng thái cục bộ của Client sau khi dự đoán
 - 3:** *input* $\leftarrow$ *readInputFromPlayer()*
 - 4:** *inputPacket* $\leftarrow$ {*currentTick*, *input*}
 - 5:** lưu *inputPacket* vào *inputBuffer*[*currentTick*]
 - 6:** *localState* $\leftarrow$ *simulate*(*localState*, *input*)
 - 7:** lưu *localState* vào *stateBuffer*[*currentTick*]
 - 8:** *sendInputToServer*(*inputPacket*)
 - 9: return** *localState*
-

Trong thuật toán 4, *currentTick* được cung cấp bởi cơ chế Tick Management. Client không tự tăng tick trong Client-side Prediction, mà chỉ sử dụng tick hiện tại để gắn nhãn cho input và trạng thái. Sau khi đọc input, Client mô phỏng ngay trạng thái mới bằng hàm *simulate()* và cập nhật trực tiếp kết quả lên đối tượng hiển thị. Nhờ đó, người chơi cảm nhận được phản hồi gần như tức thời, thay vì phải chờ gói tin xác nhận từ Server.

Một điểm quan trọng khi triển khai là hàm *simulate()* trên Client cần sử dụng cùng logic gameplay với phía Server ở mức cần thiết. Nếu Client và Server áp dụng hai logic mô phỏng khác nhau, trạng thái dự đoán tại Client sẽ nhanh chóng lệch khỏi

trạng thái xác thực trên Server. Vì vậy, các thành phần như xử lý input, vận tốc, gia tốc, va chạm hoặc giới hạn chuyển động cần được thiết kế nhất quán giữa hai phía. Tuy nhiên, trạng thái do Client tạo ra vẫn chỉ là trạng thái tạm thời, không có quyền thay thế trạng thái xác thực của Server.

Bên cạnh việc lưu input vào *inputBuffer*, Client cũng cần lưu trạng thái đã dự đoán vào *stateBuffer*. Khi Server gửi về trạng thái xác thực tại tick t_s , Client có thể truy xuất trạng thái đã dự đoán tại cùng tick đó trong *stateBuffer*, sau đó so sánh với trạng thái Server. Bằng cách đó, Client có thể ước lượng được độ chính xác trong chuyển động của vật thể.

Client-side Prediction chỉ có mô phỏng tức thời với input của người chơi mà không có cơ chế hiệu chỉnh với Server. Vì vậy, độ chính xác của Client sẽ càng ngày càng giảm xuống. Điều này có thể dẫn tới tình trạng những gì người chơi đang thấy và tương tác chưa chắc là trạng thái chính xác so với Server. Việc này có thể dẫn tới những tương tác không chuẩn xác, dẫn tới kết quả game không đúng như mong muốn của người chơi.

Do đó, Client-side Prediction không hoạt động độc lập, mà phải được kết hợp với Server Reconciliation. Client-side Prediction giúp cải thiện cảm giác phản hồi tức thời, trong khi Server Reconciliation đảm bảo trạng thái cục bộ cuối cùng vẫn hội tụ về trạng thái xác thực của Server.

Kỹ thuật này thường chỉ được áp dụng cho thực thể do người chơi trực tiếp điều khiển, vì đây là nhóm đối tượng yêu cầu độ phản hồi cao nhất. Đối với các thực thể từ xa, Client thường không có input gốc của chúng, nên hệ thống sẽ ưu tiên sử dụng nội suy hoặc DR để dự đoán chuyển động.

Tóm lại, trong phương pháp đề xuất, Client-side Prediction đóng vai trò là lớp phản hồi tức thời ở phía Client. Tại mỗi tick, Client đọc input, lưu input, mô phỏng trạng thái cục bộ, lưu trạng thái dự đoán và gửi input lên Server. Cơ chế này giúp giảm cảm giác độ trễ khi điều khiển, đồng thời tạo dữ liệu đầu vào cần thiết cho Server Reconciliation ở bước tiếp theo.

Trạng thái mô phỏng từ Client-side Prediction chỉ là trạng thái dự đoán tạm thời và có thể sai lệch so với trạng thái xác thực được Server tính toán. Nếu không có cơ chế hiệu chỉnh, sai số giữa Client và Server có thể tích lũy dần theo thời gian. Vì vậy, Server Reconciliation được sử dụng để so sánh trạng thái dự đoán của Client

với trạng thái xác thực từ Server, sau đó hiệu chỉnh và phát lại các input chưa được xác nhận khi cần thiết.

3.5.2 Server Reconciliation

Server Reconciliation là cơ chế hiệu chỉnh trạng thái cục bộ của Client dựa trên trạng thái xác thực do Server gửi về. Kỹ thuật này thường được sử dụng cùng với Client-side Prediction trong mô hình Server Authoritative. Trong đó, Client-side Prediction giúp người chơi nhận phản hồi tức thời sau khi nhập input, còn Server Reconciliation đảm bảo rằng trạng thái cục bộ cuối cùng vẫn hội tụ về trạng thái chính thức được Server xác nhận [25].

Trong quá trình hoạt động, Client liên tục gửi input lên Server và đồng thời tự mô phỏng trạng thái cục bộ. Mỗi input được gắn với một tick cụ thể và được lưu trong *inputBuffer*. Đồng thời, trạng thái sau khi Client tự mô phỏng cũng được lưu trong *stateBuffer* tại tick tương ứng. Khi Server xử lý input, Server gửi lại trạng thái xác thực cho Client dưới dạng một gói trạng thái:

$$ServerState = \{tick, position, rotation, velocity, acceleration\} \quad (3.35)$$

Trong đó, *tick* là tick mà trạng thái được Server xác nhận. Khi nhận được gói tin này, Client không thay thế trực tiếp trạng thái hiện tại bằng trạng thái Server, vì cách làm đó dễ gây dịch chuyển tức thời. Thay vào đó, Client truy xuất lại trạng thái đã dự đoán tại cùng tick trong *stateBuffer*, so sánh với trạng thái Server, sau đó quyết định có cần thực hiện hiệu chỉnh hay không.

Sai số giữa trạng thái dự đoán và trạng thái xác thực có thể được tính bằng khoảng cách vị trí:

$$error = \|\vec{P}_{predicted}(t_s) - \vec{P}_{server}(t_s)\| \quad (3.36)$$

trong đó, $\vec{P}_{predicted}(t_s)$ là vị trí mà Client đã dự đoán tại tick t_s , còn $\vec{P}_{server}(t_s)$ là vị trí xác thực do Server gửi về tại cùng tick đó.

Nếu sai số lớn hơn ngưỡng cho phép, Client cần thực hiện dịch chuyển tức thời ngay lập tức để đảm bảo độ chính xác. Ngược lại, nếu sai số nhỏ hơn ngưỡng, Client cần Server Reconciliation để hiệu chỉnh dần. Khi đó, Client khôi phục trạng

thái về trạng thái Server tại tick t_s , sau đó phát lại toàn bộ các input đã lưu từ tick $t_s + 1$ đến tick hiện tại. Kết quả sau khi replay sẽ trở thành trạng thái cục bộ mới của Client.

Quy trình cơ bản của Server Reconciliation có thể được mô tả như sau:

Thuật toán 5 Server Reconciliation

```

1: Input: serverState - Trạng thái xác nhận từ Server, tickRewindLimit - Giới
             hạn tick tua ngược, localState - Trạng thái cục bộ hiện tại, inputBuffer - Bộ
             đệm input, stateBuffer - Bộ đệm trạng thái, errorThreshold - Ngưỡng sai số

2: Output: localState - Trạng thái cục bộ sau khi hiệu chỉnh

3: if localState.tick - serverState.tick < tickRewindLimit then

4:   localState  $\leftarrow$  serverState

5:   if error(stateBuffer[serverState.tick], serverState) < errorThreshold then

6:     for each input  $\in$  inputBuffer (bắt đầu từ serverState.tick) do

7:       localState  $\leftarrow$  simulate(localState, input)

8:     end for

9:   end if

10: end if

11: return localState

```

Trong thuật toán 5, nếu như tick của *serverState* cách tick hiện tại của *localState* một khoảng nhỏ hơn giới hạn *tickRewindLimit* thì sẽ tiến hành tua ngược trạng thái. Trạng thái hiện tại của Client là *localState* được khôi phục về trạng thái xác nhận từ Server là *serverState*. Sai lệch giữa *localState* và *serverState* được tính dựa trên khoảng cách, nếu sai lệch lớn hơn *errorThreshold* thì không tua ngược nữa, chấp nhận *serverState* ngay lập tức, ngược lại quy trình tua ngược được tiếp tục. Sau đó, Client liên tục tính toán trạng thái mới tại từng tick có lưu input tại *inputBuffer*. Nhờ đó, Client vừa duy trì được phản hồi tức thời, vừa đảm bảo

trạng thái cuối cùng gần với trạng thái chính thức của Server.

Cơ chế này giúp giảm cảm giác giật so với việc gán trực tiếp trạng thái hiện tại về trạng thái Server, vì Client không bị đưa về quá khứ rồi dừng lại ở đó. Thay vào đó, hệ thống tái tạo lại trạng thái hiện tại bằng cách bắt đầu từ mốc Server đã xác nhận và áp dụng lại các input còn hiệu lực.

Tuy nhiên, Server Reconciliation vẫn tồn tại một số hạn chế. Nếu *tickRewindLimit* quá lớn, Client có thể phải replay nhiều input, làm tăng chi phí xử lý và có nguy cơ tạo ra sai lệch nếu mô phỏng không hoàn toàn ổn định. Ngược lại, nếu *tickRewindLimit* quá nhỏ, Client có thể bỏ qua nhiều trạng thái xác thực từ Server, khiến sai lệch tích lũy lâu hơn. Tương tự, *errorThreshold* cũng cần được lựa chọn phù hợp. Ngưỡng quá thấp khiến Client reconciliation quá thường xuyên và gây rung giật, ngưỡng quá cao khiến Client chấp nhận sai lệch lớn so với Server.

Tóm lại, Server Reconciliation là bước bổ sung cần thiết cho Client-side Prediction. Client-side Prediction tạo phản hồi tức thời bằng cách mô phỏng trước input ở phía Client, còn Server Reconciliation sử dụng trạng thái xác thực từ Server để kiểm tra, sửa sai và phát lại các input chưa được xác nhận. Nhờ đó, hệ thống vừa duy trì được cảm giác điều khiển nhanh, vừa đảm bảo trạng thái cục bộ không lệch quá xa so với trạng thái chính thức của Server.

Server Reconciliation chủ yếu xử lý sai lệch trạng thái của thực thể do người chơi điều khiển, chưa giải quyết đầy đủ các tương tác nhạy cảm với thời điểm xử lý như bắn, va chạm hoặc kiểm tra trúng đích. Trong các trường hợp này, Server cần đánh giá hành động dựa trên trạng thái gần với thời điểm người chơi thực hiện input, thay vì chỉ dựa trên trạng thái hiện tại khi gói tin đến nơi. Vì vậy, Lag Compensation được sử dụng như một lớp bổ sung để xử lý các tương tác quan trọng trong điều kiện mạng có độ trễ.

3.5.3 Lag Compensation

Lag Compensation là kỹ thuật được sử dụng để giảm ảnh hưởng của độ trễ mạng đối với các tương tác nhạy cảm theo thời gian, chẳng hạn như bắn súng [26]. Server là phía quyết định kết quả cuối cùng của các tương tác, nhưng nếu Server luôn kiểm tra tương tác tại thời điểm nhận input, kết quả có thể không khớp với trạng thái mà người chơi đã quan sát khi thực hiện hành động.

Ví dụ, người chơi có thể bắn khi mục tiêu đang nằm trong tâm ngắm trên màn hình Client. Do độ trễ mạng, khi gói input đến Server, mục tiêu có thể đã di chuyển sang vị trí khác. Nếu Server chỉ kiểm tra va chạm tại trạng thái hiện tại, phát bắn có thể bị xem là trượt mặc dù tại thời điểm người chơi thực hiện hành động, mục tiêu đang ở đúng vị trí bị bắn. Điều này làm giảm cảm giác công bằng, đặc biệt đối với người chơi có độ trễ mạng cao.

Để xử lý vấn đề trên, Lag Compensation cho phép Server đánh giá hành động dựa trên trạng thái quá khứ gần với thời điểm input được tạo ra trên Client. Một gói input quan trọng gửi từ Client có thể được biểu diễn như sau:

$$ActionPacket = \{tick, inputMovement, inputAction\} \quad (3.37)$$

Trong đó, *tick* là tick tại thời điểm Client tạo input, *inputMovement* là input di chuyển cơ bản, *inputAction* là input hành động cần kiểm tra. Input hành động thường đa dạng theo thể loại game, có thể là hành động bắn trong game bắn súng hoặc là va chạm trong game đua xe.

Để hỗ trợ Lag Compensation, Server cần lưu lịch sử trạng thái của các thực thể có thể tương tác theo tick. Lịch sử này có thể được tổ chức bằng Circular Buffer:

$$HistoryState = \{tick, objectId, state\} \quad (3.38)$$

Khi Server nhận được *ActionPacket*, Server kiểm tra xem tick của input có nằm trong giới hạn được phép tua ngược hay không:

$$currentTick - input.tick \leq tickRewindLimit \quad (3.39)$$

Nếu input quá cũ, Server có thể từ chối xử lý theo cơ chế Lag Compensation để tránh việc người chơi có độ trễ quá cao nhận được lợi thế không công bằng. Ngược lại, nếu input còn nằm trong cửa sổ lịch sử cho phép, Server truy xuất trạng thái của các đối tượng liên quan tại tick đó và thực hiện kiểm tra tương tác trên trạng thái quá khứ. Từ đó, đảm bảo người chơi có độ trễ cao cũng có thể tương tác với game một cách chính xác. Việc này giúp đảm bảo tính công bằng khi tương tác của người chơi có độ trễ cao vẫn có thể được chấp thuận trong khi người chơi có độ trễ thấp vẫn không cảm thấy Server đang ưu tiên độ trễ cao hơn.

Quy trình cơ bản của Lag Compensation có thể được mô tả như sau:

Thuật toán 6 Lag Compensation

```
1: Input: input - Input từ Client, currentState - Trạng thái hiện tại của hệ thống,
           stateBuffer - Bộ đệm trạng thái của Server, inputBuffer - Bộ đệm input cho
           các vật thể của hệ thống, tickRewindLimit - Giới hạn tick được tua ngược,

2: Output: currentState - Trạng thái mới sau khi kiểm tra input tại thời điểm
           trong quá khứ

3: if currentState.tick - input.tick < tickRewindLimit then

4:   currentState  $\leftarrow$  stateBuffer[input.tick]

5:   inputBuffer[input.tick].pendInput(input)

6:   currentState  $\leftarrow$  simulateSystemAtTick(input.tick)

7: end if

8: return currentState
```

Trong thuật toán 6, Server cần duy trì lịch sử trạng thái của trò chơi trong một khoảng thời gian nhất định. Khi nhận được một hành động từ Client, Server sẽ xác định thời điểm mà hành động đó được tạo ra, sau đó truy xuất trạng thái tương ứng trong lịch sử để thực hiện kiểm tra. Cụ thể, hệ thống sẽ kiểm tra xem *input* gửi tới có quá xa so với thời điểm hiện tại không, nếu không thì sẽ thực hiện tua ngược. Khi tua ngược, hệ thống sẽ đưa trạng thái về tick của *input*, sau đó lưu *input* vào *inputBuffer*. Tiếp theo, hệ thống sẽ giả lập lại tick đó với dữ liệu *input* mới thông qua hàm *simulateSystemAtTick*(). Hàm này sẽ trả về trạng thái hiện tại khi đã công nhận *input* mới tại tick trong quá khứ.

Về bản chất, Lag Compensation là cơ chế quay ngược thời gian ở phía Server. Hệ thống tạm thời phục hồi trạng thái quá khứ đối với các đối tượng liên quan, thực hiện các phép kiểm tra cần thiết như nhận biết va chạm hoặc nhận biết tương tác trong *simulateSystemAtTick*. Tùy vào mục đích mà cách cài đặt *simulateSystemAtTick* sẽ khác nhau. Nếu chỉ muốn kiểm tra đạn bắn trong game bắn súng góc nhìn thứ

nhất thì chỉ cần kiểm tra người chơi có nằm trên đường đạn không. Đối với các tương tác vật lý phức tạp hơn, chẳng hạn như va chạm tốc độ cao, nếu muốn kiểm tra va chạm thì cần thực hiện áp dụng lại lần lượt input như là tua lại từ quá khứ để đảm bảo không bỏ sót va chạm.

Lag Compensation cũng cần được giới hạn bởi *tickRewindLimit*. Nếu giới hạn này quá nhỏ, nhiều input hợp lệ của người chơi có ping cao có thể không được bù trễ. Ngược lại, nếu giới hạn này quá lớn, người chơi có độ trễ cao có thể tạo ra các kết quả không công bằng đối với người chơi khác, ví dụ bắn trúng mục tiêu dù mục tiêu đã né tránh từ lâu trên trạng thái hiện tại. Vì vậy, *tickRewindLimit* cần được lựa chọn dựa trên độ trễ tối đa mà hệ thống muốn hỗ trợ và yêu cầu cân bằng gameplay.

Ngoài ra, để giảm chi phí xử lý, hệ thống không nên tua ngược toàn bộ thực thể trong thế giới game. Server chỉ nên truy xuất và phục hồi trạng thái của các đối tượng thật sự liên quan đến hành động đang được kiểm tra. Ví dụ, trong kiểm tra bắn trúng, hệ thống chỉ cần xét người bắn và mục tiêu tiềm năng. Cách làm này giúp Lag Compensation có thể hoạt động hiệu quả hơn trong môi trường có nhiều thực thể mạng.

Tóm lại, trong phương pháp đề xuất, Lag Compensation đóng vai trò xử lý các tương tác quan trọng chịu ảnh hưởng mạnh bởi độ trễ mạng. Kỹ thuật này sử dụng lịch sử trạng thái được lưu theo tick để Server kiểm tra hành động tại thời điểm gần với lúc người chơi thực hiện input, sau đó khôi phục trạng thái hiện tại và áp dụng kết quả nếu tương tác hợp lệ. Nhờ đó, hệ thống có thể cải thiện tính chính xác và công bằng của các hành động như bắn, va chạm hoặc sử dụng kỹ năng trong game trực tuyến thời gian thực.

Chương 4

Thực nghiệm và đánh giá

4.1 Câu hỏi nghiên cứu

Chương này được xây dựng nhằm đánh giá hiệu quả của kỹ thuật Dead Reckoning trong việc đồng bộ chuyển động của đối tượng trong môi trường mạng nhiều người chơi. Bên cạnh đó là xem xét các bộ tham số phù hợp cho các nhu cầu sử dụng khác nhau. Cụ thể, các câu hỏi nghiên cứu được đặt ra như sau:

- **RQ1:** Các kỹ thuật cải tiến Dead Reckoning, bao gồm mô hình dự đoán bậc hai và Time Stamping, có giúp cải thiện độ chính xác đồng bộ chuyển động giữa Client và Server hay không?
- **RQ2:** Các kỹ thuật hỗ trợ làm mượt và điều khiển phản hồi, bao gồm Smooth-Damp, Adaptive Threshold, Client-side Prediction và Server Reconciliation, có giúp chuyển động của đối tượng và thao tác điều khiển của người chơi trở nên mượt mà, tự nhiên và ổn định hơn hay không?
- **RQ3:** Lag Compensation có giúp hệ thống xử lý chính xác hơn các thao tác quan trọng của người chơi trong điều kiện độ trễ mạng cao hay không?
- **RQ4:** Các tham số của Adaptive Threshold, Server Reconciliation và Lag Compensation nên được lựa chọn như thế nào để đạt hiệu quả phù hợp trong từng điều kiện mạng và từng trường hợp sử dụng?

4.2 Môi trường và thiết lập thực nghiệm

4.2.1 Tổng quan

Để minh họa và đánh giá trực quan kỹ thuật Dead Reckoning đã trình bày, báo cáo sử dụng dự án thực nghiệm *Multiplayer Videogame Relay Netcode* với tựa game *RocketRally* [20].

Đây là một ứng dụng trò chơi trực tuyến nhiều người chơi được xây dựng trên nền tảng Unity Engine. Mục tiêu của dự án là triển khai một kiến trúc mạng Client-Host hoàn chỉnh, cho phép người chơi kết nối qua Internet thông qua dịch vụ trung gian mà không yêu cầu cấu hình mạng phức tạp.

4.2.2 Công nghệ cốt lõi

Dựa trên tài liệu kỹ thuật của dự án, hệ thống sử dụng các công nghệ chính sau:

Game Engine: Unity 2022.3 LTS.

Unity Netcode for GameObjects (NGO): thư viện mạng chính thức của Unity, hỗ trợ đồng bộ biến, đồng bộ đối tượng mạng và triển khai các cơ chế đồng bộ chuyển động.

Unity Relay: dịch vụ chuyển tiếp giúp các Client kết nối với Host.

4.2.3 Các tính năng chính của hệ thống

Hệ thống cung cấp các chức năng cơ bản gồm tạo Lobby, tham gia Lobby và bắt đầu trận đấu. Bên cạnh đó, hệ thống còn hỗ trợ làm mượt chuyển động ở mức cơ bản để hạn chế cảm giác giật khi độ trễ mạng tăng cao.

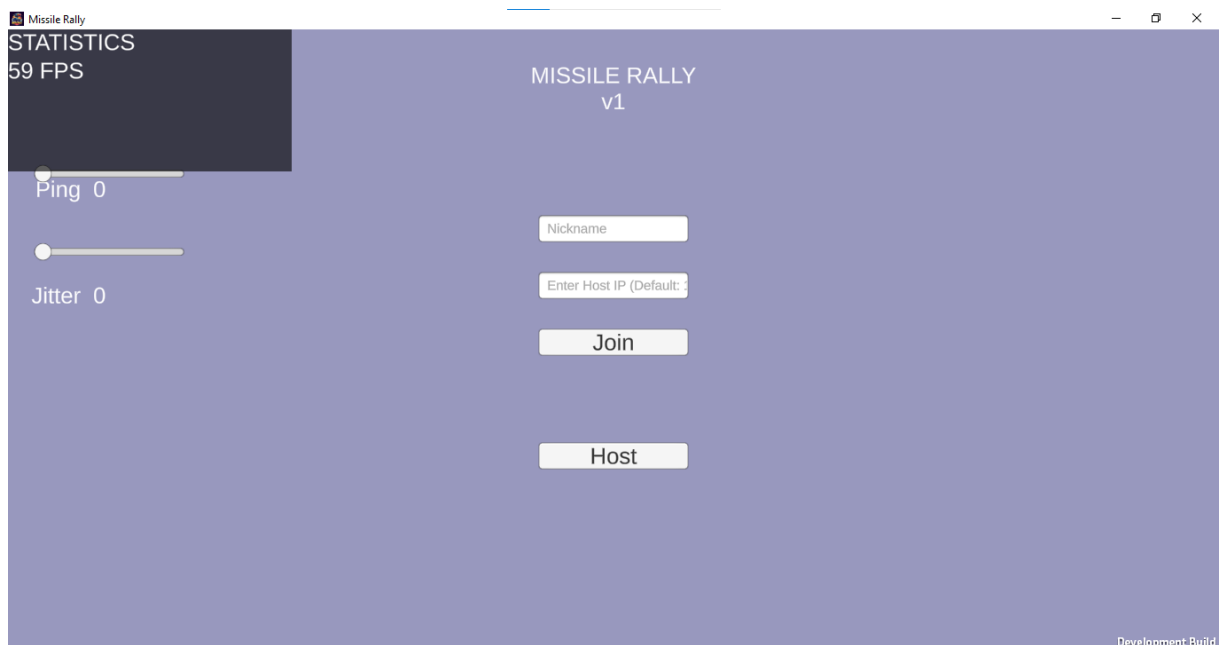
Các thành phần được bổ sung nhằm phục vụ minh họa và thực nghiệm Dead Reckoning và giải pháp đề xuất bao gồm:

- Chức năng thay đổi giữa mô hình dự đoán bậc một và bậc hai của DR.
- Chức năng thay đổi giữa mô hình nội suy tuyến tính Lerp và nội suy giảm chấn tối hạn SmoothDamp.
- Chức năng bật tắt tùy chọn sử dụng Time Stamping và Adaptive Threshold của DR.
- Chức năng bật tắt tùy chọn Dead Reckoning, Client-side Prediction và Server Reconciliation đối với xe của người chơi.
- Chức năng bật tắt Lag Compensation ở phía Server và bắn laser tự động để kiểm thử.

- Khả năng tùy chỉnh các tham số cho Adaptive Threshold, Server Reconciliation và Lag Compensation.
- Giao diện thể hiện thông số đo đạc bao gồm Độ chính xác, Jerk và bộ đếm hit của Lag Compensation giúp theo dõi độ hiệu quả của phương pháp đề xuất.

4.2.4 Quy trình vận hành demo

Để phục vụ thực nghiệm, đầu tiên khởi động các bản sao của dự án bằng thư viện ParrelSync. Bên cạnh đó, khóa luận cũng tạo ra bản Build của dự án để có thể chạy thông qua file .exe mà không cần phải mở Unity Editor.

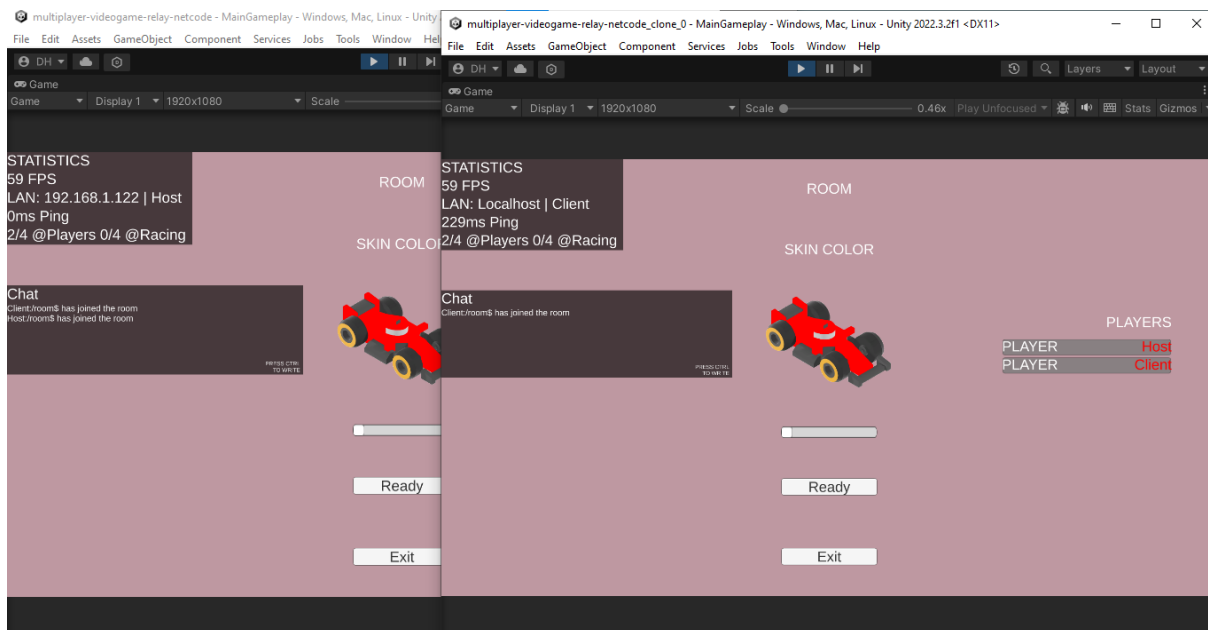


Hình 4.1: Giao diện menu chính trong game

Hình 4.1 thể hiện menu chính của game. Tại đây, người chơi có thể chọn làm Host hoặc trở thành Guest. Người chơi sẽ trở thành Host khi nhấn nút “Host”, khi đó game sẽ tạo phòng để các người chơi Guest tham gia. Trước khi tạo phòng, người chơi Host có thể tùy chỉnh cài đặt mạng bao gồm độ trễ Ping và độ biến thiên trễ Jitter. Người chơi Guest sẽ tham gia vào phòng đã được tạo thông qua nút “Join”. Khi đó, người chơi Guest sẽ được đưa vào phòng chờ mà người chơi Host đã tạo trước đó.

Việc đưa các tham số Ping và Jitter vào ngay trong giao diện khởi tạo giúp quá trình thực nghiệm được thực hiện thuận tiện hơn. Người thử nghiệm có thể thay

điều kiện mạng trước mỗi lần chạy, từ đó so sánh hành vi của hệ thống trong nhiều môi trường mạng khác nhau. Đây là cơ sở để đánh giá mức độ ổn định, độ chính xác và độ mượt của các phương pháp đồng bộ được triển khai trong game.



Hình 4.2: Giao diện Lobby trong game

Hình 4.2 thể hiện giao diện phòng chờ của trò chơi trước khi cuộc đua bắt đầu. Tại giao diện này, người chơi có thể quan sát danh sách các người chơi đang tham gia phòng, đồng thời tùy chỉnh màu sắc của xe nhằm phân biệt phương tiện của mình với các người chơi khác trong quá trình đua. Sau khi hoàn tất quá trình chuẩn bị, người chơi có thể nhấn nút *Ready* để xác nhận trạng thái sẵn sàng. Hệ thống sẽ chỉ bắt đầu cuộc đua khi tất cả người chơi trong phòng đều đã chuyển sang trạng thái *Ready*.

Khi cuộc đua bắt đầu, người chơi điều khiển xe bằng các phím mũi tên để di chuyển trên đường đua. Mục tiêu chính của người chơi là hoàn thành một vòng đua trong thời gian ngắn nhất có thể. Ngoài thao tác di chuyển, người chơi có thể sử dụng phím *F* để bắn về phía đối thủ. Khi bị bắn trúng, xe của đối thủ sẽ rơi vào trạng thái choáng trong một khoảng thời gian ngắn và tạm thời không thể di chuyển. Cơ chế này tạo thêm yếu tố cạnh tranh và tương tác trực tiếp giữa các người chơi, thay vì chỉ tập trung vào khả năng điều khiển xe.

Bên cạnh đó, trò chơi cũng cung cấp chức năng *Respawn* để xử lý các tình huống bất lợi trong quá trình đua như khi xe bị lật, mắc kẹt hoặc văng ra khỏi đường

đua. Chức năng này giúp duy trì tính liên tục của phiên chơi, hạn chế trường hợp người chơi không thể tiếp tục tham gia cuộc đua do lỗi vật lý hoặc do các va chạm ngoài ý muốn.



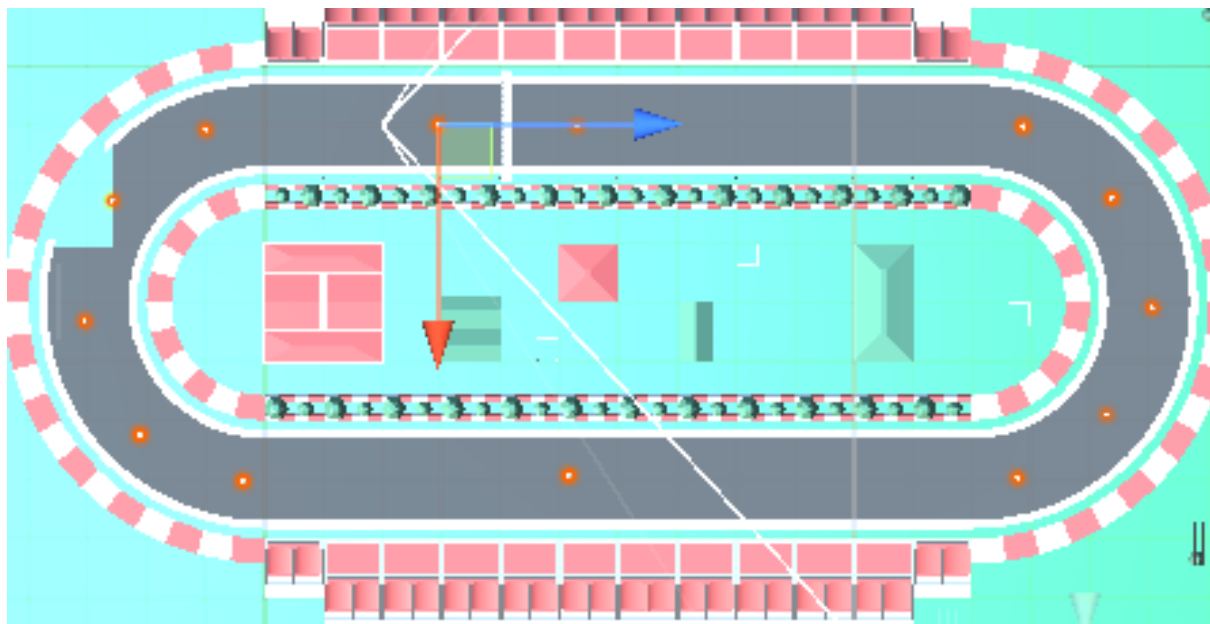
Hình 4.3: Giao diện gameplay

Hình 4.3 thể hiện giao diện của trò chơi trong quá trình cuộc đua diễn ra. Bên cạnh các thành phần gameplay cơ bản, giao diện này còn được thiết kế nhằm hỗ trợ quá trình thực nghiệm và đánh giá các phương pháp được đề xuất trong khóa luận. Thông qua giao diện, người dùng có thể trực tiếp lựa chọn và bật tắt các kỹ thuật đồng bộ khác nhau, từ đó quan sát sự thay đổi về độ chính xác, độ mượt chuyển động và khả năng phản hồi của hệ thống trong từng trường hợp.

Đối với xe do người chơi trực tiếp điều khiển, giao diện cung cấp các tùy chọn bật hoặc tắt *Client-side Prediction*, *Server Reconciliation* và *Dead Reckoning*. Đối với các xe của người chơi khác được hiển thị trên màn hình, giao diện cho phép tùy chỉnh các thành phần liên quan đến *Dead Reckoning*. Cụ thể, người dùng có thể bật hoặc tắt *Dead Reckoning* đối với những xe đó, lựa chọn mô hình dự đoán, lựa chọn mô hình nội suy, đồng thời bật hoặc tắt các giải pháp cải tiến *Time Stamping* và *Adaptive Threshold*. Đối với phía Server, hay người chơi đóng vai trò *Host*, giao diện cung cấp tùy chọn bật hoặc tắt *Lag Compensation*.

Ngoài các tùy chọn điều khiển phương pháp, giao diện còn hiển thị các chỉ số phục vụ đánh giá thực nghiệm. Cụ thể, hệ thống hỗ trợ theo dõi độ chính xác và giá trị

Jerk của từng xe trong game. Bên cạnh đó, giao diện cũng hiển thị số lần người chơi bắn và được Server xác nhận là trúng đích.



Hình 4.4: Các waypoint mà bot sẽ di chuyển tới trong quá trình thực nghiệm

Hình 4.4 thể hiện các điểm mốc mà bot sẽ lần lượt di chuyển tới. Để đảm bảo tính khách quan trong quá trình đo đạc, hệ thống hỗ trợ bot tự động di chuyển theo các điểm waypoint được đánh dấu trên bản đồ. Tính năng này có thể được bật/tắt thông qua tickbox *Turn On Bot*. Cơ chế di chuyển của bot có sự khác biệt so với người chơi thông thường. Trong quá trình điều khiển, người chơi thường chủ động giảm tốc tại các khúc cua bằng cách nhả phím tăng tốc. Tuy nhiên, bot không trực tiếp nhận biết được hình dạng của khúc cua mà chỉ xác định hướng di chuyển dựa trên vị trí của điểm mốc tiếp theo. Do đó, khóa luận cài đặt cơ chế giảm dần giá trị input tiến về phía trước dựa trên độ lớn của góc xoay cần thiết để hướng tới điểm mốc kế tiếp. Nhờ đó, bot có thể giảm tốc độ khi vào cua, giúp chuyển động trở nên ổn định và tự nhiên hơn. Do giá trị input tiến thẳng của bot biến thiên liên tục thay vì chỉ nhận các giá trị rời rạc như 0 hoặc 1 giống người chơi, chuyển động của bot đôi khi có thể xuất hiện hiện tượng giật nhẹ. Vì vậy, để đảm bảo tính nhất quán và công bằng trong quá trình thực nghiệm, khóa luận cài đặt tất cả các xe tham gia thử nghiệm đều được điều khiển bởi bot.

Đối với Lag Compensation, hệ thống cung cấp nút "Auto Shoot Bot" và khả năng bắn laser tự động vào đối thủ. Khi bấm, hệ thống sẽ đưa xe tới vị trí nhất định,

xe sẽ đóng vai trò bắn khi có xe khác tới gần. Độ hiệu quả của Lag Compensation được thể hiện thông qua việc hành động bắn có được chấp thuận không.

4.3 Chỉ số đánh giá

4.3.1 Chỉ số Jerk

Như đã nói ở Chương 2, trong vật lý và điều khiển chuyển động, Jerk (j) là đạo hàm bậc ba của vị trí theo thời gian, hay đạo hàm của gia tốc theo thời gian. Đại lượng này phản ánh mức độ thay đổi đột ngột của gia tốc.

$$j(t) = \frac{d\vec{a}(t)}{dt} = \frac{d^3\vec{p}(t)}{dt^3} \quad (4.1)$$

Jerk là chỉ số quan trọng để định lượng độ mượt của chuyển động và mức độ xuất hiện của hiện tượng Rubber Banding. Nếu Jerk cao, chuyển động có xu hướng giật cục; nếu Jerk thấp, quỹ đạo chuyển động mượt và tự nhiên hơn.

Trong thực nghiệm này, Jerk được tính theo đơn vị vị trí của Unity trên giây lập phương (u/s^3).

$$J_{avg} = \frac{1}{N} \sum_{i=1}^N \left| \frac{\vec{a}_i - \vec{a}_{i-1}}{\Delta t} \right| \quad (4.2)$$

Jerk sẽ được sử dụng cho giải pháp liên quan tới độ mượt, bao gồm Adaptive Threshold, Nội suy SmoothDamp, Server Reconciliation.

4.3.2 Tỷ lệ chính xác

Đây là chỉ số thể hiện tỷ lệ phần trăm số khung hình mà sai số vị trí giữa Client và Server nhỏ hơn một ngưỡng chấp nhận được (δ_{pos}).

$$Accuracy(\%) = \frac{N_{success}}{N_{total}} \times 100 \quad (4.3)$$

Trong đó $N_{success}$ là số khung hình thỏa mãn điều kiện:

$$\|P_{client} - P_{server}\| < \delta_{pos} \quad (4.4)$$

Tỷ lệ chính xác cao cho thấy trạng thái hiển thị trên Client bám sát với trạng thái thực tế do Server tính toán. Ngược lại, tỷ lệ thấp cho thấy thuật toán dự đoán chưa đủ hiệu quả hoặc cơ chế hiệu chỉnh còn gây lệch quỹ đạo.

Khóa luận sẽ sử dụng chiều dài xe làm ngưỡng sai số. Nếu xe di chuyển lệch khoảng bằng với chiều dài xe thì lúc đó được coi là không chính xác.

Đối với Lag Compensation, độ chính xác sẽ dựa trên tỉ lệ số lần hành động bắn của xe Client trên số lần xe thực hiện hành động bắn thành công trên Server.

4.3.3 Mức sử dụng CPU

Bên cạnh độ chính xác và độ mượt chuyển động, chi phí tính toán của thuật toán cũng là một yếu tố quan trọng trong hệ thống mạng thời gian thực. Trong thực nghiệm này, mức sử dụng CPU được sử dụng để đánh giá lượng tài nguyên xử lý mà các giải pháp đề xuất tiêu tốn trong quá trình hoạt động.

Chỉ số này được đo bằng khoảng thời gian xử lý của CPU dành cho mỗi khung hình. Giá trị này càng cao thì tức là CPU càng tốn nhiều thời gian để xử lý khung hình đó. Khi đó, giá trị CPU cao cho thấy thuật toán yêu cầu nhiều phép tính hơn, đặc biệt trong các trường hợp sử dụng mô hình dự đoán bậc cao, nội suy phức tạp hoặc cơ chế hiệu chỉnh liên tục.

Việc đo mức sử dụng CPU giúp đánh giá khả năng mở rộng của hệ thống khi số lượng đối tượng mạng hoặc số lượng người chơi tăng lên. Một giải pháp có độ chính xác cao nhưng tiêu tốn quá nhiều tài nguyên xử lý có thể gây ảnh hưởng đến tốc độ khung hình và trải nghiệm tổng thể của trò chơi.

Mức độ sử dụng CPU sẽ được dùng để đo hiệu quả của Lag Compensation. Lí do là vì Lag Compensation là một kỹ thuật tiêu tốn nhiều tài nguyên của hệ thống. Cốt lõi của kỹ thuật này dựa trên việc quay ngược về thời điểm của input trên toàn bộ Server, tức là Server sẽ phải mô phỏng lại toàn bộ hoặc một phần game trong một khoảng thời gian ngắn. Điều này dẫn tới việc Server sẽ phải tiêu tốn rất nhiều tài nguyên để xử lí. Vậy nên mức sử dụng CPU sẽ được dùng để kiểm tra xem việc đánh đổi tài nguyên hệ thống với Lag Compensation có đem lại hiệu quả không.

4.3.4 Mức sử dụng bộ nhớ RAM

Mức sử dụng RAM được dùng để đánh giá lượng bộ nhớ mà hệ thống cần sử dụng trong quá trình lưu trữ và xử lý dữ liệu mạng.

Trong các kỹ thuật như Lag Compensation, hệ thống cần lưu trữ lịch sử trạng thái và input của đối tượng trong các cấu trúc như Circular Buffer. Điều này làm tăng lượng bộ nhớ cần thiết, đặc biệt khi tần suất khung hình cao hoặc thời gian lưu lịch sử dài.

Chỉ số RAM được đo bằng dung lượng bộ nhớ mà ứng dụng sử dụng trong quá trình thực nghiệm. Giá trị RAM càng lớn cho thấy hệ thống cần lưu trữ nhiều dữ liệu hơn để phục vụ đồng bộ và hiệu chỉnh trạng thái.

Việc đánh giá mức sử dụng RAM giúp xác định khả năng triển khai của hệ thống trên các thiết bị có cấu hình khác nhau, đồng thời đánh giá sự đánh đổi giữa hiệu năng bộ nhớ và chất lượng đồng bộ mạng.

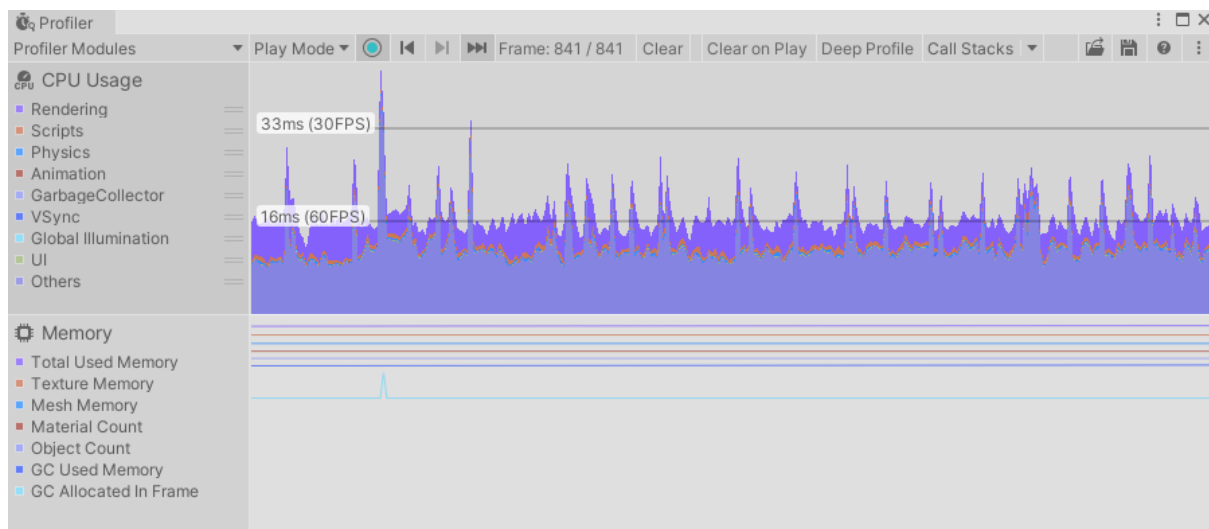
Mức độ sử dụng RAM sẽ được dùng để đo hiệu quả của Lag Compensation. Lí do cũng tương tự như đối với mức độ sử dụng CPU đó là Lag Compensation tốn rất nhiều tài nguyên hệ thống. Lag Compensation yêu cầu hệ thống có một lượng lưu trữ input và trạng thái lớn trên toàn Server để có thể quay ngược một cách hiệu quả. Vậy nên mức sử dụng RAM sẽ được dùng để kiểm tra xem việc đánh đổi tài nguyên hệ thống với Lag Compensation có đem lại hiệu quả không.

4.3.5 Thu thập dữ liệu CPU và RAM

Các chỉ số về mức sử dụng CPU và RAM trong quá trình thực nghiệm được thu thập thông qua công cụ giám sát hiệu năng *Unity Profiler* của Unity. Unity Profiler cho phép theo dõi thời gian xử lý của CPU, mức sử dụng bộ nhớ và chi phí xử lý của từng thành phần trong hệ thống theo thời gian thực. Nhờ đó, khóa luận có thể đánh giá mức tài nguyên mà các kỹ thuật đồng bộ mạng tiêu tốn khi được triển khai trong game.

Trong phạm vi thực nghiệm, chỉ số CPU được sử dụng để đánh giá thời gian xử lý trung bình trên mỗi khung hình, đặc biệt khi áp dụng các kỹ thuật có chi phí tính toán cao như Lag Compensation. Bên cạnh đó, chỉ số RAM được theo dõi nhằm xác định lượng bộ nhớ phát sinh khi hệ thống lưu trữ lịch sử trạng thái phục vụ

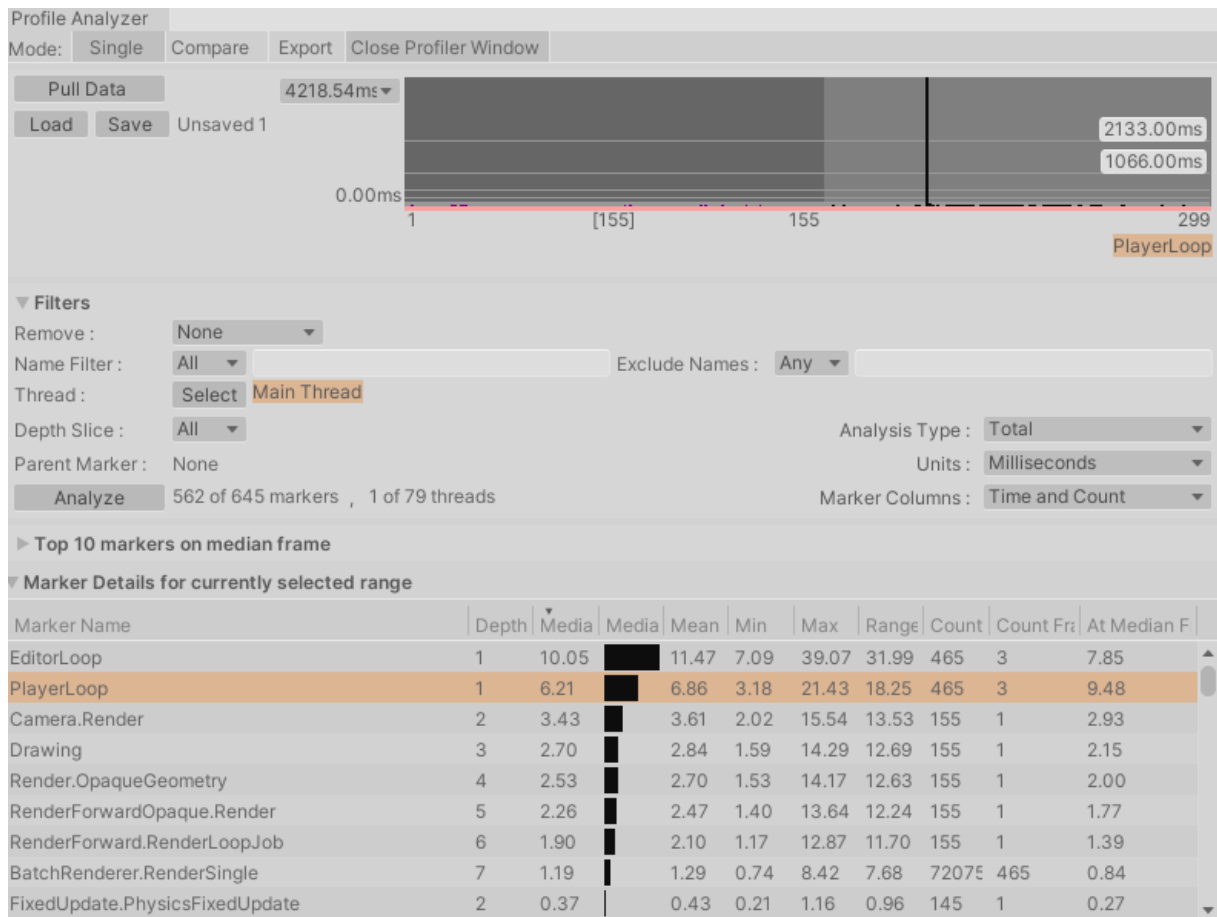
quá trình quay ngược. Việc kết hợp hai chỉ số này giúp phân tích sự đánh đổi giữa hiệu quả đồng bộ và chi phí tài nguyên của phương pháp đề xuất.



Hình 4.5: Công cụ Unity Profiler

Hình 4.5 thể hiện giao diện công cụ *Unity Profiler*. Công cụ này sẽ thể hiện mức tài nguyên CPU và bộ nhớ đã sử dụng trong quá trình vận hành game theo từng khung hình. Cần lưu ý rằng công cụ này sẽ không trực tiếp đo mức tài nguyên sử dụng của kỹ thuật riêng lẻ như Lag Compensation. Thay vào đó, công cụ đưa ra dữ liệu đo được của toàn bộ quá trình vận hành game, bao gồm cả hệ thống Unity Editor. Bên cạnh đó, công cụ cũng không thể hiện mức CPU trung bình của toàn bộ quá trình vận hành mà chỉ thể hiện dữ liệu theo khung hình. Ngược lại, tài nguyên RAM được sử dụng trong quá trình vận hành game có giá trị tương đương nhau trên mọi khung hình. Điều này là do kể từ khi bắt đầu chạy thử nghiệm, game đã đăng ký sẵn lượng RAM sẽ sử dụng trong quá trình vận hành. Việc này giúp theo dõi RAM trên *Unity Profiler* dễ hơn nhiều so với theo dõi mức sử dụng CPU.

Để phân tích chi tiết mức tài nguyên sử dụng trong quá trình kiểm thử, bao gồm cả thời gian xử lý CPU trung bình, khóa luận sử dụng công cụ *Unity Profile Analyzer*. Đây là công cụ hỗ trợ phân tích dữ liệu được ghi lại từ *Unity Profiler*, cho phép tổng hợp, so sánh và đánh giá các chỉ số hiệu năng một cách trực quan. Dữ liệu sau khi được thu thập từ Unity Profiler sẽ được đưa vào Profile Analyzer để quan sát mức tiêu thụ tài nguyên trung bình, từ đó hỗ trợ đánh giá chi phí xử lý của các kỹ thuật được triển khai. Giao diện của công cụ được thể hiện trong Hình 4.6.



Hình 4.6: Công cụ Unity Profile Analyzer

Hình 4.6 thể hiện giao diện của công cụ *Unity Profile Analyzer*. Công cụ này hỗ trợ phân tích và tính toán mức sử dụng CPU dựa trên dữ liệu được thu thập từ *Unity Profiler*. Thông qua đó, người thực nghiệm có thể quan sát thời gian xử lý trung bình và chi phí CPU của các thành phần trong quá trình chạy game.

Trong phạm vi khóa luận, quá trình đánh giá sẽ tập trung vào giá trị của dòng *PlayerLoop*. Lý do là vì *PlayerLoop* phản ánh phần lớn thời gian xử lý trực tiếp của game, bao gồm cập nhật logic, xử lý input, mô phỏng vật lý và cập nhật đối tượng. Các thành phần khác có thể bao gồm chi phí của Unity Editor hoặc các tác vụ không trực tiếp thuộc logic game. Vì vậy, việc tập trung vào *PlayerLoop* giúp kết quả đo phản ánh chính xác hơn mức CPU mà game sử dụng trong quá trình kiểm thử.

4.4 Kết quả thực nghiệm

4.4.1 Tổng quan

Phần này trình bày kết quả thực nghiệm của các phương pháp cải tiến và bổ trợ đã được triển khai trong hệ thống. Các thực nghiệm được thực hiện trên cùng một môi trường kiểm thử nhằm đảm bảo tính nhất quán khi so sánh giữa các cấu hình.

Trong toàn bộ kịch bản, hệ thống sử dụng bot tự động điều khiển phương tiện để duy trì quỹ đạo di chuyển và thời điểm thực hiện thao tác ổn định giữa các lần đo. Mỗi cấu hình được chạy nhiều lần trong cùng điều kiện mạng, sau đó lấy giá trị trung bình nhằm giảm ảnh hưởng của nhiễu hệ thống và dao động trong quá trình xử lý.

4.4.2 Đánh giá mô hình dự đoán và Time Stamping

Kịch bản đầu tiên đánh giá hiệu quả của mô hình dự đoán bậc một, mô hình dự đoán bậc hai và cơ chế Time Stamping đối với độ chính xác của Dead Reckoning. Kịch bản này giúp trả lời câu hỏi **RQ1**.

Bảng 4.1: Kết quả thực nghiệm mô hình dự đoán và Time Stamping

Ping (ms)	Bậc 1		Bậc 2	
	Không Time Stamping	Có Time Stamping	Không Time Stamping	Có Time Stamping
0	79.1	93.4	86.1	98.2
100	72.7	91.6	83.8	96.5
500	69.6	84.2	76.5	89.8
1000	57.8	74.5	71.8	81.7

Dựa trên Bảng 4.1, cấu hình sử dụng mô hình dự đoán bậc hai kết hợp với Time Stamping đạt kết quả tốt nhất ở tất cả các mức ping. Accuracy đạt 98.2% tại ping 0 ms, 96.5% tại ping 100 ms, 89.8% tại ping 500 ms và 81.7% tại ping 1000 ms. Tính trung bình trên bốn mức ping, cấu hình cơ bản sử dụng mô hình bậc một và không có Time Stamping đạt 69.8%, trong khi cấu hình bậc hai có Time Stamping đạt khoảng 91.6%.

Kết quả này cho thấy cả Time Stamping và mô hình dự đoán bậc hai đều cải thiện độ chính xác của quá trình dự đoán trạng thái. Time Stamping giúp Client xác định chính xác hơn khoảng thời gian đã trôi qua kể từ khi Server tạo trạng thái,

nhờ đó giá trị Δt được sử dụng trong dự đoán phản ánh đúng hơn ảnh hưởng của độ trễ mạng. Trong khi đó, mô hình dự đoán bậc hai xét thêm thành phần gia tốc, phù hợp hơn với các chuyển động có tăng tốc, giảm tốc hoặc đổi hướng như trong game đua xe.

Như vậy, việc kết hợp Time Stamping với mô hình dự đoán bậc hai là hướng cải tiến hiệu quả cho Dead Reckoning, đặc biệt trong các điều kiện mạng có độ trễ cao. Tuy nhiên, mô hình bậc hai cũng yêu cầu thêm thông tin gia tốc hoặc phải ước lượng gia tốc từ các gói tin liên tiếp, do đó cần cân nhắc giữa mức cải thiện độ chính xác và độ phức tạp triển khai.

4.4.3 Đánh giá độ mượt chuyển động

Nhóm thực nghiệm này đánh giá ảnh hưởng của các kỹ thuật làm mượt chuyển động, bao gồm Lerp, SmoothDamp và Adaptive Threshold. Mục tiêu chính là phân tích sự thay đổi của chỉ số Jerk trong các điều kiện mạng khác nhau. Nhóm thực nghiệm này giúp trả lời câu hỏi **RQ2** và **RQ4** đối với Adaptive Threshold.

Nội suy và Adaptive Threshold

Bảng 4.2: Kết quả thực nghiệm Nội suy và Adaptive Threshold

Ping (ms)	Không nội suy	Lerp		SmoothDamp	
		Không Adaptive	Có Adaptive	Không Adaptive	Có Adaptive
0	79.6	53.4	31.3	40.5	23.0
100	75.4	41.7	24.3	35.0	10.7
500	59.8	88.9	33.5	64.9	7.6
1000	431.6	359.4	128.6	219.0	107.3

Dựa trên Bảng 4.2, cấu hình SmoothDamp kết hợp Adaptive Threshold cho Jerk thấp nhất trong phần lớn các trường hợp. Tại ping 0 ms, 100 ms và 500 ms, Jerk lần lượt giảm từ 79.6 xuống 23.0, từ 75.4 xuống 10.7 và từ 59.8 xuống 7.6 u/s^3 so với trường hợp không sử dụng nội suy và Adaptive Threshold. Điều này tương ứng với mức giảm khoảng 71.1%, 85.8% và 87.3%.

Khi so sánh giữa Lerp và SmoothDamp, SmoothDamp nhìn chung cho chuyển động ổn định hơn. Khi cùng sử dụng Adaptive Threshold, Jerk trung bình của

SmoothDamp đạt khoảng $37.2 u/s^3$, thấp hơn so với $54.4 u/s^3$ của Lerp. Điều này cho thấy SmoothDamp phù hợp hơn với quá trình hiệu chỉnh trạng thái do chuyển động tiến về trạng thái mục tiêu mềm hơn, hạn chế thay đổi đột ngột so với nội suy tuyến tính.

Adaptive Threshold cũng góp phần giảm Jerk rõ rệt, đặc biệt trong điều kiện ping cao. Tại ping 1000 ms, khi sử dụng SmoothDamp, việc bổ sung Adaptive Threshold giúp Jerk giảm từ 219.0 xuống còn $107.3 u/s^3$. Kết quả này cho thấy Adaptive Threshold giúp hạn chế các hiệu chỉnh quá mạnh khi sai số giữa trạng thái dự đoán và trạng thái Server thay đổi lớn.

Adaptive Threshold trong điều kiện jitter

Bảng 4.3: Kết quả thực nghiệm Adaptive Threshold trong điều kiện Jitter

Ping (ms)	Jitter (ms)	Không Adaptive Accuracy	Jerk	Có Adaptive Accuracy	Jerk
100	50	71.5	142.4	71.8	121.8
100	100	72.9	133.1	72.8	128.3
500	50	72.5	191.0	74.2	173.4
500	100	73.9	199.4	72.5	183.7
1000	50	80.1	316.4	74.0	240.5
1000	100	76.8	244.35	72.3	182.6

Bảng 4.3 cho thấy Adaptive Threshold tiếp tục có tác dụng giảm Jerk trong điều kiện mạng có jitter. Tính trung bình trên các kịch bản thử nghiệm, Jerk giảm từ 204.4 xuống còn $171.7 u/s^3$, tương ứng giảm khoảng 16.0%. Mức giảm thể hiện rõ hơn ở ping cao, chẳng hạn tại ping 1000 ms và jitter 50 ms, Jerk giảm từ 316.4 xuống $240.5 u/s^3$; tại ping 1000 ms và jitter 100 ms, Jerk giảm từ 244.35 xuống $182.6 u/s^3$.

Tuy nhiên, độ chính xác không cải thiện rõ rệt khi sử dụng Adaptive Threshold. Accuracy trung bình giảm nhẹ từ 74.6% xuống còn 72.9%. Điều này phản ánh sự đánh đổi giữa độ mượt và độ chính xác: khi ngưỡng sai số được nới rộng linh hoạt hơn, hệ thống có thể giảm số lần hiệu chỉnh không cần thiết, nhưng đồng thời cho phép trạng thái Client lệch khỏi Server trong phạm vi lớn hơn.

Như vậy, Adaptive Threshold phù hợp với mục tiêu cải thiện cảm giác chuyển động trong điều kiện mạng không ổn định. Tuy nhiên, phương pháp này không nên được xem là giải pháp tối ưu tuyệt đối về độ chính xác, mà đóng vai trò cân bằng giữa việc duy trì chuyển động mượt và hạn chế sai lệch quá lớn so với trạng thái Server.

Bảng 4.4: Kết quả thực nghiệm tham số Adaptive Threshold

Ping (ms)	Bộ tham số ($ThresholdBase, k_v, k_a$)					
	(1.5, 0.10, 0.15)		(2.5, 0.15, 0.25)		(4.0, 0.25, 0.40)	
	Accuracy	Jerk	Accuracy	Jerk	Accuracy	Jerk
0	97.4	35.3	96.2	31.0	96.0	31.8
100	96.7	102.2	94.1	41.3	92.1	13.3
500	95.8	212.6	92.1	83.8	89.3	44.5
1000	95.5	266.5	90.3	139.7	87.9	54.8

Bảng 4.4 cho thấy các bộ tham số Adaptive Threshold tạo ra sự khác biệt rõ rệt giữa Accuracy và Jerk. Bộ tham số ($ThresholdBase, k_v, k_a$) = (1.5, 0.10, 0.15) đạt Accuracy trung bình cao nhất, khoảng 96.35%, nhưng cũng có Jerk trung bình cao nhất, khoảng 154.15 u/s^3 . Ngược lại, bộ (4.0, 0.25, 0.40) cho Jerk trung bình thấp nhất, khoảng 36.10 u/s^3 , nhưng Accuracy trung bình giảm xuống khoảng 91.33%.

Kết quả này cho thấy ngưỡng sai số thấp giúp đối tượng bám sát Server hơn, nhưng làm tăng số lần hiệu chỉnh và khiến chuyển động dễ bị giật. Ngược lại, ngưỡng cao giúp chuyển động mượt hơn, nhưng cho phép sai lệch vị trí tồn tại lâu hơn trước khi sửa lỗi. Trong ba bộ tham số, bộ (2.5, 0.15, 0.25) là lựa chọn cân bằng hơn, với Accuracy trung bình khoảng 93.18% và Jerk trung bình khoảng 73.95 u/s^3 .

Do đó, việc lựa chọn tham số Adaptive Threshold cần phụ thuộc vào yêu cầu của trò chơi. Nếu ưu tiên độ chính xác trong các tương tác quan trọng như va chạm hoặc bắn trúng, nên chọn ngưỡng thấp hơn. Nếu ưu tiên độ mượt trong điều kiện mạng kém ổn định, có thể chọn ngưỡng cao hơn. Trong phạm vi thực nghiệm này, bộ tham số trung gian (2.5, 0.15, 0.25) là cấu hình hợp lý để cân bằng giữa độ chính xác và độ mượt.

4.4.4 Đánh giá Client-side Prediction và Server Reconciliation

Nhóm thực nghiệm này đánh giá hiệu quả của Client-side Prediction kết hợp Server Reconciliation trong bài toán đồng bộ chuyển động của đối tượng do người chơi điều khiển. Khác với Dead Reckoning, phương pháp này cho phép Client dự đoán phản hồi từ input cục bộ, sau đó hiệu chỉnh lại dựa trên trạng thái xác thực từ Server. Nhóm thực nghiệm này giúp trả lời câu hỏi **RQ2** và **RQ4** đối với Server Reconciliation.

Hiệu quả của Client-side Prediction và Server Reconciliation

Bảng 4.5: Kết quả thực nghiệm Client-side Prediction và Server Reconciliation

Ping (ms)	Không kỹ thuật		Dead Reckoning		CSP + SR	
	Accuracy	Jerk	Accuracy	Jerk	Accuracy	Jerk
0	63.8	64.0	27.6	15.4	89.8	18.6
100	48.5	53.3	42.2	7.2	92.5	8.8
500	11.9	25.7	68.3	7.9	96.4	10.7
1000	21.6	25.3	28.4	12.9	85.7	9.1

Dựa trên Bảng 4.5, Client-side Prediction và Server Reconciliation (CSP + SR) đạt Accuracy cao nhất ở tất cả các mức ping. Accuracy lần lượt đạt 89.8%, 92.5%, 96.4% và 85.7% tại các mức ping 0 ms, 100 ms, 500 ms và 1000 ms. Tính trung bình, CSP + SR đạt Accuracy khoảng 91.1%, cao hơn đáng kể so với 36.5% của trường hợp không xử lý và 41.6% của Dead Reckoning.

Xét về độ mượt, Dead Reckoning có Jerk trung bình thấp nhất, khoảng $10.9 u/s^3$, trong khi CSP + SR đạt khoảng $11.8 u/s^3$. Tuy nhiên, sự chênh lệch này không lớn, trong khi độ chính xác của CSP + SR cao hơn rõ rệt. Điều này cho thấy Dead Reckoning có thể làm mượt chuyển động tốt, nhưng không phù hợp để xử lý đối tượng do người chơi trực tiếp điều khiển vì không phản ánh chính xác các input thay đổi liên tục.

Kết quả thực nghiệm cho thấy CSP + SR phù hợp hơn với bài toán đồng bộ chuyển động của người chơi trong mô hình Server Authoritative. Client-side Prediction giúp phản hồi input gần như tức thời, còn Server Reconciliation giúp đưa trạng

thái dự đoán về gần trạng thái xác thực từ Server. Nhờ đó, phương pháp này cân bằng tốt hơn giữa cảm giác điều khiển, độ chính xác và tính nhất quán của hệ thống.

Ảnh hưởng của `errorThreshold` trong Server Reconciliation

Bảng 4.6: Kết quả thực nghiệm Server Reconciliation

Ping (ms)	<code>errorThreshold</code> = 1.5		<code>errorThreshold</code> = 2.5		<code>errorThreshold</code> = 4.0	
	Accuracy	Jerk	Accuracy	Jerk	Accuracy	Jerk
0	94.6	13.9	90.8	10.8	88.3	8.7
100	96.2	14.3	90.6	12.5	83.5	10.4
500	87.3	16.2	79.4	15.6	65.0	14.2
1000	67.4	21.1	45.8	18.5	34.6	16.8

Bảng 4.6 cho thấy giá trị *errorThreshold* ảnh hưởng trực tiếp đến sự cân bằng giữa Accuracy và Jerk. Với *errorThreshold* = 1.5, hệ thống đạt Accuracy trung bình cao nhất, khoảng 86.4%, nhưng Jerk trung bình cũng cao nhất, khoảng $16.4 u/s^3$. Ngược lại, *errorThreshold* = 4.0 cho Jerk trung bình thấp nhất, khoảng $12.5 u/s^3$, nhưng Accuracy trung bình giảm còn khoảng 67.9%.

Điều này xuất phát từ cơ chế hoạt động của Server Reconciliation. Khi ngưỡng sai số nhỏ, Client thực hiện hiệu chỉnh thường xuyên hơn để bám sát Server, nhờ đó Accuracy tăng nhưng chuyển động dễ xuất hiện nhiều thay đổi đột ngột hơn. Khi ngưỡng sai số lớn, số lần hiệu chỉnh giảm, giúp chuyển động mượt hơn, nhưng trạng thái Client có thể lệch xa hơn so với Server, đặc biệt trong điều kiện ping cao.

Trong ba cấu hình thử nghiệm, *errorThreshold* = 2.5 có thể được xem là lựa chọn trung gian. Cấu hình này không đạt Accuracy cao nhất, nhưng giảm Jerk so với ngưỡng 1.5 và vẫn duy trì độ chính xác tốt hơn ngưỡng 4.0. Vì vậy, tham số này phù hợp khi hệ thống cần cân bằng giữa độ chính xác đồng bộ và độ mượt hiển thị.

4.4.5 Đánh giá Lag Compensation

Nhóm thực nghiệm này đánh giá hiệu quả của Lag Compensation trong các tình huống tương tác phụ thuộc vào thời điểm, cụ thể là kiểm tra bắn trúng bằng Ray-cast. Trong kịch bản thử nghiệm, xe A di chuyển theo quỹ đạo xác định trước, còn xe B thực hiện bắn tại một thời điểm cố định. Khi không sử dụng Lag Compensation, Server kiểm tra và chạm tại thời điểm nhận được yêu cầu bắn. Khi sử dụng Lag Compensation, Server tua lại trạng thái mục tiêu về thời điểm Client thực hiện thao tác, sau đó mới kiểm tra và chạm. Nhóm thực nghiệm này giúp trả lời câu hỏi **RQ3** và **RQ4** đối với Lag Compensation.

Hiệu quả của Lag Compensation

Bảng 4.7: Kết quả thực nghiệm Lag Compensation

Ping (ms)	Hit Accuracy (%)	
	Không Lag Compensation	Có Lag Compensation
0	100.0	100.0
100	100.0	100.0
500	0	100
1000	0	100

Dựa trên Bảng 4.7, Lag Compensation cho thấy hiệu quả rõ rệt trong điều kiện ping cao. Ở mức ping 0 ms và 100 ms, cả hai trường hợp có và không sử dụng Lag Compensation đều đạt Hit Accuracy 100%, cho thấy độ trễ thấp chưa tạo ra sai lệch đủ lớn để ảnh hưởng đến kết quả kiểm tra và chạm.

Tuy nhiên, tại ping 500 ms và 1000 ms, nếu không sử dụng Lag Compensation, Hit Accuracy giảm xuống 0%. Ngược lại, khi sử dụng Lag Compensation, Hit Accuracy vẫn được duy trì ở mức 100%. Tính trung bình trên bốn mức ping, trường hợp không sử dụng Lag Compensation chỉ đạt 50%, trong khi trường hợp có Lag Compensation đạt 100%.

Kết quả này cho thấy Lag Compensation là cơ chế cần thiết đối với các tương tác yêu cầu độ chính xác theo thời điểm. Khi Server chỉ kiểm tra trạng thái hiện tại tại thời điểm nhận gói tin, mục tiêu có thể đã di chuyển khỏi vị trí mà người chơi

quan sát được khi bắn. Việc tua lại trạng thái giúp Server đánh giá va chạm theo góc nhìn của người chơi tại thời điểm thực hiện hành động, từ đó cải thiện tính công bằng và độ chính xác trong điều kiện mạng có độ trễ cao.

Ảnh hưởng của `tickRewindLimit`

Bảng 4.8: Kết quả thực nghiệm tham số Lag Compensation

Ping (ms)	<code>tickRewindLimit = 25</code>			<code>tickRewindLimit = 50</code>			<code>tickRewindLimit = 100</code>		
	Hit	CPU	RAM	Hit	CPU	RAM	Hit	CPU	RAM
0	100	65.9	2.0	100	69.3	3.3	100	73.3	3.6
100	100	68.0	3.1	100	72.2	3.5	100	82.1	4.0
500	100	72.9	3.3	100	85.1	4.2	100	92.8	4.5
1000	0	81.1	4.6	100	90.6	4.8	100	105.1	5.3

Bảng 4.8 cho thấy *tickRewindLimit* ảnh hưởng trực tiếp đến khả năng duy trì Hit Accuracy trong điều kiện ping cao. Ở các mức ping 0 ms, 100 ms và 500 ms, cả ba cấu hình 25, 50 và 100 đều đạt Hit Accuracy 100%. Tuy nhiên, tại ping 1000 ms, cấu hình *tickRewindLimit* = 25 không còn đủ khả năng tua lại trạng thái, khiến Hit Accuracy giảm xuống 0%. Trong khi đó, hai cấu hình 50 và 100 vẫn duy trì Hit Accuracy 100%.

Khi xét thêm chi phí tài nguyên, *tickRewindLimit* = 50 là cấu hình cân bằng hơn so với 100. Cả hai đều đạt Hit Accuracy 100% ở tất cả các mức ping, nhưng cấu hình 50 có CPU trung bình khoảng 79.3 ms/frame và RAM trung bình khoảng 3.95 GB, thấp hơn so với 88.3 ms/frame và 4.35 GB của cấu hình 100.

Như vậy, *tickRewindLimit* cần đủ lớn để bao phủ khoảng thời gian trễ giữa lúc Client thực hiện hành động và lúc Server xử lý yêu cầu. Nếu giới hạn quá nhỏ, Server không thể truy xuất trạng thái quá khứ phù hợp trong điều kiện ping cao. Ngược lại, nếu giới hạn quá lớn, hệ thống tiêu tốn thêm CPU và RAM mà không nhất thiết cải thiện kết quả. Trong phạm vi thực nghiệm này, *tickRewindLimit* = 50 là lựa chọn phù hợp nhất.

4.4.6 Tổng hợp kết quả

Từ các kết quả thực nghiệm, có thể rút ra một số nhận xét chính. Thứ nhất, mô hình dự đoán bậc hai kết hợp Time Stamping giúp cải thiện rõ rệt độ chính xác của Dead Reckoning, đặc biệt trong các điều kiện mạng có độ trễ cao. Thứ hai, SmoothDamp và Adaptive Threshold giúp giảm Jerk, qua đó cải thiện độ mượt của chuyển động hiển thị trên Client. Tuy nhiên, Adaptive Threshold cũng tạo ra sự đánh đổi giữa độ mượt và độ chính xác, do ngưỡng sai số lớn hơn có thể cho phép trạng thái Client lệch khỏi Server lâu hơn.

Thứ ba, đối với các đối tượng do người chơi trực tiếp điều khiển, Client-side Prediction kết hợp Server Reconciliation cho kết quả phù hợp hơn Dead Reckoning truyền thống. Phương pháp này đạt độ chính xác cao trong khi vẫn duy trì Jerk ở mức chấp nhận được. Cuối cùng, Lag Compensation cho thấy vai trò quan trọng trong các tương tác phụ thuộc thời điểm như bắn trúng hoặc va chạm. Việc tua lại trạng thái giúp duy trì Hit Accuracy trong điều kiện ping cao, nhưng cần lựa chọn *tickRewindLimit* phù hợp để cân bằng giữa độ chính xác và chi phí tài nguyên.

Nhìn chung, các kết quả thực nghiệm cho thấy nhóm phương pháp cải tiến giúp nâng cao hiệu quả dự đoán và làm mượt của Dead Reckoning, trong khi nhóm phương pháp bổ trợ giúp khắc phục các hạn chế của Dead Reckoning trong các tình huống liên quan đến input người chơi và tương tác quan trọng. Điều này chứng minh rằng việc kết hợp các kỹ thuật dự đoán, hiệu chỉnh và tua lại trạng thái là cần thiết để nâng cao chất lượng đồng bộ trong game trực tuyến thời gian thực.

Kết luận

Khóa luận đã nghiên cứu và đề xuất các phương pháp cải tiến, hỗ trợ cho kỹ thuật Dead Reckoning nhằm nâng cao hiệu quả đồng bộ hóa trạng thái trong trò chơi trực tuyến thời gian thực. Các phương pháp cải tiến được xây dựng dựa trên cơ sở vật lý và toán học, tập trung vào việc nâng cao độ chính xác dự đoán và độ mượt trong quá trình nội suy chuyển động. Bên cạnh đó, các phương pháp hỗ trợ như Client-side Prediction, Server Reconciliation và Lag Compensation được sử dụng nhằm khắc phục những hạn chế cố hữu của Dead Reckoning, đặc biệt trong các tình huống liên quan đến chuyển động của người chơi, bắn súng, va chạm và các thao tác có ảnh hưởng trực tiếp đến trải nghiệm gameplay.

Thông qua kết quả thực nghiệm, khóa luận đã chứng minh được hiệu quả của các phương pháp được đề xuất trong nhiều điều kiện mạng khác nhau. Mô hình dự đoán bậc hai và Time Stamping cho thấy khả năng cải thiện độ chính xác dự đoán so với Dead Reckoning cơ bản. Trong khi đó, SmoothDamp và Adaptive Threshold giúp nâng cao độ mượt chuyển động so với mô hình nội suy tuyến tính ban đầu. Đặc biệt, Adaptive Threshold thể hiện khả năng hoạt động hiệu quả trong môi trường có độ trễ mạng biến thiên, giúp giảm hiện tượng giật cục trong chuyển động, dù phải đánh đổi bằng việc độ chính xác có thể giảm nhẹ trong một số trường hợp.

Ngoài các cải tiến trực tiếp trên Dead Reckoning, khóa luận cũng chứng minh hiệu quả của các phương pháp hỗ trợ. Client-side Prediction kết hợp với Server Reconciliation cho kết quả tốt hơn đáng kể trong trường hợp đồng bộ chuyển động của người chơi so với việc không sử dụng phương pháp hỗ trợ hoặc chỉ sử dụng Dead Reckoning. Lag Compensation cũng cho thấy vai trò quan trọng trong việc xác nhận các input quan trọng như bắn súng hoặc va chạm, giúp người chơi vẫn có thể tương tác tương đối chính xác ngay cả trong điều kiện độ trễ cao, chẳng hạn như 1000 ms.

Bên cạnh việc đánh giá hiệu quả, khóa luận cũng đưa ra một số gợi ý lựa chọn tham số cho từng phương pháp. Đối với Adaptive Threshold, việc lựa chọn tham số cần dựa trên mức độ ưu tiên giữa độ chính xác và độ mượt của chuyển động. Với Server Reconciliation, ngưỡng hiệu chỉnh cần được điều chỉnh phù hợp với yêu cầu gameplay, tránh tình trạng hiệu chỉnh quá mạnh gây giật cục hoặc quá yếu làm

sai lệch trạng thái kéo dài. Đối với Lag Compensation, khóa luận đề xuất cấu hình cân bằng nhằm đảm bảo khả năng xác nhận input của người chơi mà vẫn kiểm soát được chi phí tài nguyên hệ thống. Trong các trường hợp yêu cầu độ chính xác cao hơn, nhà phát triển có thể tăng giới hạn lưu trữ và quay ngược trạng thái, miễn là vẫn đảm bảo tính công bằng và khả năng xử lý của hệ thống.

Mặc dù đạt được những kết quả tích cực, khóa luận vẫn còn một số hạn chế nhất định. Trước hết, phạm vi thực nghiệm mới chủ yếu tập trung vào một số kịch bản chuyển động cụ thể, chưa đánh giá đầy đủ hiệu quả của phương pháp đề xuất trên các thể loại game khác như game nhập vai trực tuyến nhiều người chơi hoặc game đối kháng. Bên cạnh đó, các phương pháp dự đoán và nội suy hiện tại phù hợp hơn với môi trường không gian mở hoặc ít vật cản. Trong các không gian hẹp, phức tạp hoặc có nhiều vật thể va chạm, hiệu quả dự đoán có thể bị suy giảm do hệ thống chưa xét đầy đủ đến tương tác giữa đối tượng với môi trường xung quanh.

Từ những hạn chế trên, một hướng phát triển tiềm năng trong tương lai là mở rộng Dead Reckoning theo hướng có khả năng nhận biết môi trường. Cụ thể, hệ thống có thể kết hợp thêm thông tin về vật cản, địa hình, va chạm hoặc hành vi của các thực thể xung quanh để nâng cao độ chính xác dự đoán. Hướng tiếp cận này có thể giúp phương pháp hoạt động hiệu quả hơn trong các không gian hẹp, phức tạp, đồng thời hỗ trợ tốt hơn cho những trò chơi có số lượng lớn thực thể như game nhập vai trực tuyến nhiều người chơi.

Tài liệu tham khảo

- [1] Uswatun Hasanah. Classification of game genres based on interaction patterns and popularity in the virtual world of roblox. *International Journal Research on Metaverse*, 2025.
- [2] World Bank. Population, total - thailand. <https://data.worldbank.org/indicator/SP.POP.TOTL?locations=TH>, 2024. World Development Indicators. Accessed: 2026-05-16.
- [3] Saad Aldousari, A. Almarzouq, Abdelkareem Hassan, A. Shahin, S. Bubishate, and Basmah Bahbahani. The era of telesurgery: insights from ultra-long-distance asia to middle east human telesurgery robotic assisted radical prostatectomy. *Journal of Robotic Surgery*, 19, 2025.
- [4] Shengmei Liu, Mark Claypool, Atsuo Kuwahara, James J. Scovell, and Jamie Sherman. The effects of network latency on competitive first-person shooter game players. *2021 13th International Conference on Quality of Multimedia Experience (QoMEX)*, pages 151–156, 2021.
- [5] Grand View Research. Online Gaming Market Size, Share & Trends Analysis Report, 2030, 2025.
- [6] Edgegap. Independent Survey on Latency in Multiplayer Games and the Impacts for Game Developers, 2022.
- [7] Lothar Pantel and L. Wolf. On the suitability of dead reckoning schemes for games. pages 79–84, 2002.
- [8] S. A. Hamid, Rana Alaudeen, and Dr.Ruaa Ali Khamees. What is client-server system: Architecture, issues and challenge of client -server system (review). 2020.
- [9] Mee Lan Han, Byung Il Kwak, and H. Kim. Cheating and detection method in massively multiplayer online role-playing game: Systematic literature review. *IEEE Access*, PP:1–1, 2022.
- [10] Joshua E. Glazer and S. Madhav. Multiplayer game programming: Architecting networked games. 2015.

- [11] Gonzalo Martinez, J.A. Hernandez, P. Reviriego, and Paul Reinheimer. Round trip time delay in the internet: Analysis and trends. *IEEE Network*, 38:280–285, 2023.
- [12] Aline Normoyle, Gina Guerrero, and S. Jörg. Player perception of delays and jitter in character responsiveness. *Proceedings of the ACM Symposium on Applied Perception*, 2014.
- [13] Florian Metzger, Stefan Geissler, Alexej Grigorjew, Frank Loh, Christian Moldovan, Michael Seufert, and T. Hossfeld. An introduction to online video game qos and qoe influencing factors. *IEEE Communications Surveys Tutorials*, 24:1894–1925, 2022.
- [14] T. Koshtybayev, A. Akzholova, A. Tatenov, and M. Aliyeva. Mathematical principles of mechanical movements. *Bulletin of the National Engineering Academy of the Republic of Kazakhstan*, 2024.
- [15] D. Eager, Ann-Marie Pendrill, and N. Reistad. Beyond velocity and acceleration: jerk, snap and higher derivatives. *European Journal of Physics*, 37, 2016.
- [16] Youfu Chen and Elvis S. Liu. Comparing dead reckoning algorithms for distributed car simulations. *Proceedings of the 2018 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*, 2018.
- [17] Y. W. Bernier. Latency compensating methods in client/server in-game protocol design and optimization. *Game Developers Conference (GDC)*, 2001.
- [18] Psyonix. Rocket league. <https://www.rocketleague.com>, 2015.
- [19] Jared Cone. It is rocket science!: The physics of rocket league. https://media.gdcvault.com/gdc2018/presentations/Cone_Jared_It_Is_Rocket.pdf, 2018.
- [20] aaseper. Multiplayer videogame relay netcode. <https://github.com/aaseper/multiplayer-videogame-relay-netcode>, 2024.
- [21] Peter John Ryan. Effectiveness of dead reckoning algorithms in advanced distributed simulation. 2004.

- [22] D. Cacuci. Perspective on predictive modeling: Current status, new high-order methodology and outlook for energy systems. *Energies*, 2023.
- [23] Thomas Lowe. Critically damped ease-in/ease-out smoothing. In Andrew Kirmse, editor, *Game Programming Gems 4*, pages 97–98. Charles River Media, 2004.
- [24] Unity Technologies. Unity scripting api: Mathf.smoothdamp. <https://docs.unity3d.com/ScriptReference/Mathf.SmoothDamp.html>, 2025.
- [25] Takato Motoo, Jiei Kawasaki, T. Fujihashi, S. Saruwatari, and Takashi Watanabe. Client-side network delay compensation for online shooting games. *IEEE Access*, 9:125678–125690, 2021.
- [26] Steven W. K. Lee and R. Chang. Enhancing the experience of multiplayer shooter games via advanced lag compensation. *Proceedings of the 9th ACM Multimedia Systems Conference*, 2018.